Metode Numerice @ CS-UPB

Echipa MN CS - 2015:

Profesori:

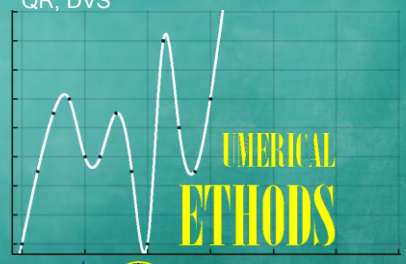
- Pantelimon George Popescu - Seria CA;
- Florin Pop - Seria CB, CC;

Asistenti:

PG Popescu, Clementin Cercel, Sorin Ciolofan, Bogdan Tiganoaia, Alexandru Tifrea, David Iancu, Bogdan Cristian Marchis, Radu Poenaru, Madalina Grosu, Andrei Vlad Postoaca, Radu Constantinescu, Roxana Istrate, Madalina Hristache, Adelina Vidovici;

Vectori si valori proprii

QR, DVS



Vectori si Valori proprii

Metoda QR

Algoritmul QR - O metoda matriciala de reducere folosita pentru calculul simultan al valorilor proprii ale unei matrici simetrice; (top 10 Computational Alg.)

Aplicam QR unei matrici simetrice tridiagonale, iar daca matricea nu este in forma tridiagonala, mai intai aplicam Householder si o aducem intr-o forma asemanatoare tridiagonala.

Vectori si Valori proprii

Metoda QR

Matrice simetrica tridiagonala

$$A = \begin{bmatrix} a_1 & b_2 & 0 & \cdots & 0 \\ b_2 & a_2 & b_3 & & \vdots \\ 0 & b_3 & a_3 & \ddots & 0 \\ \vdots & & \ddots & & b_n \\ 0 & \cdots & 0 & b_n & a_n \end{bmatrix}$$

Daca b2=0 sau bn=0 atunci a1 sau an sunt valori proprii pentru A. Metoda se bazeaza pe aceasta observatie si la fiecare pas descreste b2 si bn.

Vectori si Valori proprii

Metoda QR

Daca exista bj=0, 2<j<n atunci se aplica QR pentru doua matrici mai mici, adica pentru

$$\begin{bmatrix} a_1 & b_2 & 0 & \cdots & 0 \\ b_2 & a_2 & b_3 & & \vdots \\ 0 & b_3 & a_3 & \ddots & 0 \\ \vdots & & \ddots & & b_{j-1} \\ 0 & \cdots & 0 & b_{j-1} & a_{j-1} \end{bmatrix} \quad \begin{bmatrix} a_j & b_{j+1} & 0 & \cdots & 0 \\ b_{j+1} & a_{j+1} & b_{j+2} & & \vdots \\ 0 & b_{j+2} & a_{j+2} & \ddots & 0 \\ \vdots & & \ddots & & b_n \\ 0 & \cdots & 0 & b_n & a_n \end{bmatrix}$$

Vectori si Valori proprii

Metoda QR

Iar daca bj!=0, oricare ar fi 2<j<n atunci QR formeaza pas cu pas un sir de matrici A1, A2,... astfel

1. A1=A si A1=Q1R1;
2. A2=R1Q1 ...

In general Ai este factorizata in QiRi, iar Ai+1=RiQi, dar cum Qi este ortogonala atunci Ri=Qi^TAi si deci

Ai+1=Qi^TAiQi, ceea ce asigura ca Ai+1 este simetrica si are aceleasi valori proprii ca si A si de asemenea asigura faptul ca Ai+1 este tridiagonala.

Algoritmul QR continua si prin inductie Ai+1 devine matrice diagonala, cu valorile proprii pe diagoanala.

Vectori si Valori proprii

Metoda QR - Accelerarea convergentei

Daca |λ1|>|λ2|>…>|λn| atunci rata convergentei faptului ca b_j+1(i+1)->0 in matricea A(i+1) este direct proportionala cu raportul |λj+1/λj|.

Aceasta determina cat de repede converge a_j(i+1) catre valoarea proprie corespunzatoare λj. Si deci daca raportul |λj+1/λj| este aproape de 1 atunci metoda converge slab.

Vectori si Valori proprii

Metoda QR - Accelerarea convergentei

Pentru a grabi convergenta se foloseste o metoda de deplasare, se alege σ aproape de o valoare proprie si la fiecare pas factorizarea QR se modifica in

$A^{(i)} - \sigma I = Q^{(i)} R^{(i)}$ si se considera $A^{(i+1)} = R^{(i)} Q^{(i)} + \sigma I$

Cu aceasta modificare raportul ce dicteaza convergenta devine |(λj+1- σ)/(λj- σ)| si deci pentru σ aproape de λj+1 dar departe de λj avem asigurata convergenta rapida pentru a_j(i+1) catre λj.

Vectori si Valori proprii

Metoda QR - Accelerarea convergentei

La fiecare pas se alege un alt σ si ne implica faptul ca bn(i+1) converge catre 0 mai repede ca oricare bj(i+1), iar atunci cand bn(i+1)~0 putem considera an(i+1)~λn. La urmatorul pas vom considera matricea simplificata, fara ultima linie si ultima coloana si se repeta procedeul pana cand il aproximam pe λn-1. Si repetam pana cand obtinem o aproximatie pentru fiecare valoare proprie.

Vectori si Valori proprii

Metoda QR - Accelerarea convergentei

Alegerea lui σ la fiecare pas se face astfel

$$E^{(i)} = \begin{bmatrix} a_{n-1}^{(i)} & b_n^{(i)} \\ b_n^{(i)} & a_n^{(i)} \end{bmatrix}$$

Se alege valoarea proprie a matricei Ei care este mai aproape de an(i).

Vectori si Valori proprii

Metoda QR - Matrici Nesimetrice

Daca A este nesimetrica atunci matricea este redusa, prin metoda Householder la forma Hessenberg(Karl Hessenberg (1904-1959), inginer german) superioara, adica la o forma de genul

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1\,(n-1)} & a_{1\,n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2\,(n-1)} & a_{2\,n} \\ 0 & a_{32} & a_{33} & \cdots & a_{3\,(n-1)} & a_{3\,n} \\ 0 & 0 & a_{43} & \cdots & a_{4\,(n-1)} & a_{4\,n} \\ 0 & 0 & 0 & \ddots & a_{5\,(n-1)} & a_{5\,n} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & a_{(n-1)\,(n-2)} & a_{(n-1)\,(n-1)} & a_{(n-1)\,n} \\ 0 & 0 & 0 & 0 & a_{n\,(n-1)} & a_{n\,n} \end{bmatrix}$$

Vectori si Valori proprii

Metoda QR - Matrici Nesimetrice

Diferenta metodei de deplasare QR la matrici nesimetrice, care este mai complicata, consta in faptul ca pot aparea valori proprii distincte de acelasi modul. Metoda de deplasare QR dubla

$$\begin{aligned} H^{(1)} - \sigma_1 I &= Q^{(1)} R^{(1)}, & H^{(2)} &= R^{(1)} Q^{(1)} + \sigma_1 I, \\ H^{(2)} - \sigma_2 I &= Q^{(2)} R^{(2)}, & H^{(3)} &= R^{(2)} Q^{(2)} + \sigma_2 I, \end{aligned}$$

unde σ1 si σ2 sunt numere complex conjugate, iar H1,H2,H3… matrici reale Hessenberg superioare.

Descompunerea Valorilor Singulare

Factorizarea A=USV^T, unde A este o matrice mxn, S este o matrice mxn ale carei singure elemente diferite de 0 sunt pe prima diagonala, U este o matrice ortogonala mxm si V este o matrice ortogonala nxn, poarta numele de DVS.

De obicei m>n, sau m>>n.

Desi aceasta tehnica de factorizare dateaza de pe la finele secolului 19, ea a fost folosita odata cu dezvoltarea algoritmilor eficienti ce o implementeaza.

Descompunerea Valorilor Singulare

Pentru a factoriza matricea A in sensul DVS, vom considera matricile patrate mxm AA^T si nxn A^TA.

Pt o matrice A mxn se defineste rangul matricei A, ca fiind Rank(A) = numerul liniilor linear independente din A, iar

Null(A) = cel mai mare set de vectori v, din R^n linear independenti pentru care Av=0.

Daca A este patratica nxn atunci A este inversabila daca si numai daca Null(A)=0 si Rank(A)=n.

Numarul de linii linear independente dintr-o matrice mxn este egal cu numarul de coloane linear independente din acea matrice.

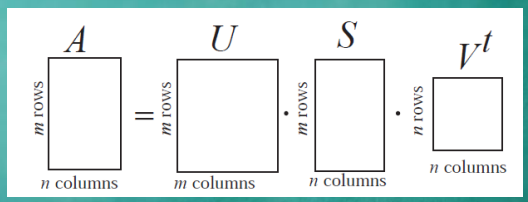
Descompunerea Valorilor Singulare

Fie A o matrice mxn, atunci:

- A^TA si AA^T sunt simetrice;
- Null(A)=Null(A^TA);
- Rank(A)=Rank(A^TA);
- Valorile proprii pt A^TA si AA^T sunt reale nenegative;
- Valorile proprii diferite de 0 pt A^TA si AA^T sunt aceleasi;

Descompunerea Valorilor Singulare

DVS



U,V ortogonale, iar Sii>=0.

Descompunerea Valorilor Singulare

Constructia lui S

Fie s_i^2 valorile proprii ale matricei simetrice nxn A^TA pe care le ordonam astfel

$$s_1^2 \geq s_2^2 \geq \cdots \geq s_k^2 > s_{k+1} = \cdots = s_n = 0.$$

Valoarile pozitive ale radicalului valorilor proprii ale matricii A^TA poarta numele de valori singulare pentru A.

Descompunerea Valorilor Singulare

Constructia lui S

$$S = \begin{bmatrix} s_1 & 0 & \cdots & 0 \\ 0 & s_2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & s_n \\ 0 & \cdots & \cdots & 0 \\ \vdots & & & \vdots \\ 0 & \cdots & \cdots & 0 \end{bmatrix}$$

Daca A este simetrica atunci A=A^T, deci A^2=A^TA si deci valorile singulare sunt chiar valorile proprii.

S contine valorile proprii ale lui A^TA.

Descompunerea Valorilor Singulare

Constructia lui V

Pentru orice matrice M simetrica avem M=VDV^T, unde D este o matrice diagonla ce contine valorile proprii ale lui M, iar V este ortogonala cu coloanele vectorii proprii ale lui M si de norma euclidiana 1.

Si deci cum A^TA este simetrica avem A^TA=VDV^T, iar cum vectorii proprii pot fi alesi oricum, inseamna ca avem mai mult varinte pentru V, matrice nxn.

Iar pentru ca valorile proprii ale lui A^TA sunt nenegative avem D=S^2.

Descompunerea Valorilor Singulare

Constructia lui U

Consideram $s_1 \geq s_2 \geq \cdots \geq s_k > 0$

si coloanele corespunzatoare din V, v1,v2,...,vk pentru care definim $\mathbf{u}_i = \frac{1}{s_i} A\mathbf{v}_i,$ i=1...k;

Folosim acesti vectori ca primele k coloane din U, care este mxm si deci ne mai lipsesc m-k vectori, pe care ii consideram astfel incat matricea sa fie ortogonala si deci coloanele linear independente.

Descompunerea Valorilor Singulare

Constructia lui U

Alegerea coloanelor ramase se face prin algoritmul Gram-Schmitd (Jørgen Pedersen Gram((1850 - 1916) - mat. danez) si Erhard Schmidt((1876 -1959) - mat. german))

Descompunerea Valorilor Singulare

Constructia lui U - Algoritmul Gram-Schmitd

$$\operatorname{proj}_{\mathbf{u}}(\mathbf{v}) = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\langle \mathbf{u}, \mathbf{u} \rangle} \mathbf{u},$$

$$\begin{aligned}
\mathbf{u}_1 &= \mathbf{v}_1, & \mathbf{e}_1 &= \frac{\mathbf{u}_1}{\|\mathbf{u}_1\|} \\
\mathbf{u}_2 &= \mathbf{v}_2 - \operatorname{proj}_{\mathbf{u}_1}(\mathbf{v}_2), & \mathbf{e}_2 &= \frac{\mathbf{u}_2}{\|\mathbf{u}_2\|} \\
\mathbf{u}_3 &= \mathbf{v}_3 - \operatorname{proj}_{\mathbf{u}_1}(\mathbf{v}_3) - \operatorname{proj}_{\mathbf{u}_2}(\mathbf{v}_3), & \mathbf{e}_3 &= \frac{\mathbf{u}_3}{\|\mathbf{u}_3\|} \\
\mathbf{u}_4 &= \mathbf{v}_4 - \operatorname{proj}_{\mathbf{u}_1}(\mathbf{v}_4) - \operatorname{proj}_{\mathbf{u}_2}(\mathbf{v}_4) - \operatorname{proj}_{\mathbf{u}_3}(\mathbf{v}_4), & \mathbf{e}_4 &= \frac{\mathbf{u}_4}{\|\mathbf{u}_4\|} \\
&\vdots & &\vdots \\
\mathbf{u}_k &= \mathbf{v}_k - \sum_{j=1}^{k-1} \operatorname{proj}_{\mathbf{u}_j}(\mathbf{v}_k), & \mathbf{e}_k &= \frac{\mathbf{u}_k}{\|\mathbf{u}_k\|}.
\end{aligned}$$

Descompunerea Valorilor Singulare

Exemplu:

$$A = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}$$

$$A = U S V^t = \begin{bmatrix} \frac{\sqrt{30}}{15} & \frac{\sqrt{6}}{3} & 0 & \frac{\sqrt{5}}{5} & 0 \\ \frac{\sqrt{30}}{15} & -\frac{\sqrt{6}}{6} & 0 & \frac{\sqrt{3}}{3} & \frac{\sqrt{2}}{2} \\ \frac{\sqrt{30}}{10} & 0 & \frac{\sqrt{2}}{2} & -\frac{\sqrt{3}}{3} & 0 \\ \frac{\sqrt{30}}{15} & -\frac{\sqrt{6}}{3} & 0 & \frac{\sqrt{5}}{5} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{30}}{10} & 0 & -\frac{\sqrt{2}}{2} & -\frac{\sqrt{3}}{3} & 0 \end{bmatrix} \begin{bmatrix} \sqrt{5} & 0 & 0 \\ 0 & \sqrt{2} & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} \frac{\sqrt{6}}{6} & \frac{\sqrt{6}}{3} & \frac{\sqrt{6}}{6} \\ \frac{\sqrt{3}}{3} & -\frac{\sqrt{3}}{3} & \frac{\sqrt{3}}{3} \\ -\frac{\sqrt{2}}{2} & 0 & \frac{\sqrt{2}}{2} \end{bmatrix}.$$

Bibliografie QR,DVS

- Richard L. Burden, J. Douglas Faires, Numerical Analysis, 9th edition, Brooks/Cole, Cengage Learning, 2011;

Metoda QR cu deplasare explicită pentru matrice simetrice

Noțiuni teoretice

Din cauza creșterii rapide a erorii obținute, metodele puterii nu se folosesc la calcularea tuturor valorilor proprii ale unei matrice.

O alternativă este algoritmul QR care determină simultan toate valorile proprii ale unei matrice simetrice tridiagonale. Pentru a putea determina valorile proprii ale oricărei matrice simetrice, vom folosi în prealabil metoda Householder, metodă care transformă o matrice simetrică într-una simetrică tridiagonală.

Matricea A, de dimensiune $n \times n$, în forma simetrică tridiagonală este:

$$A = \begin{bmatrix} a_1 & b_2 & 0 & \cdots & 0 \\ b_2 & a_2 & b_3 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & b_{n-1} & a_{n-1} & b_n \\ 0 & \cdots & 0 & b_n & a_n \end{bmatrix}$$

Dacă $b_2 = 0$, respectiv $b_n = 0$, atunci matricea A va avea o valoare proprie egală cu a_1, respectiv cu a_n. Ceea ce face metoda QR în cazul în care b_2 și b_n sunt diferite de 0 este să scadă progresiv valorea lui b_2, respectiv a lui b_n, până devin aproximativ egale cu 0.

Când $b_j = 0$ pentru o valoare a lui j care respectă condiția $2 < j < n$, problema poate fi redusă la rezolvarea a două probleme de dimensiune mai mică: o problemă de dimensiune $j - 1$ (a) și o problemă de dimensiune $n - j + 1$ (b).

$$(a)\quad \begin{bmatrix} a_1 & b_2 & 0 & \cdots & 0 \\ b_2 & a_2 & b_3 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & b_{j-1} \\ 0 & \cdots & 0 & b_{j-1} & a_{j-1} \end{bmatrix} \qquad (b)\quad \begin{bmatrix} a_j & b_{j+1} & 0 & \cdots & 0 \\ b_{j+1} & a_{j+1} & b_{j+2} & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & b_n \\ 0 & \cdots & 0 & b_n & a_n \end{bmatrix}$$

Dacă niciuna din valorile b_j nu este egală cu 0, atunci metoda QR presupune formarea unei secvențe $A = A^{(1)}, A^{(2)}, A^{(3)}...$ după cum urmează:

- $A^{(1)} = A$ este factorizată ca fiind $A^{(1)} = Q^{(1)}R^{(1)}$, unde $Q^{(1)}$ este o matrice ortogonală, iar $R^{(1)}$ este o matrice superior triunghiulară.
- $A^{(2)} = R^{(1)}Q^{(1)}...$

În general, $A^{(i)}$ este factorizat ca fiind $A^{(i)} = Q^{(i)}R^{(i)}$, unde $Q^{(i)}$ este o matrice ortogonală, iar $R^{(i)}$ este o matrice superior triunghiulară. Apoi, $A^{(i+1)} = R^{(i)}Q^{(i)}$. Deoarece $Q^{(i)}$ este ortogonală, $R^{(i)} = {Q^{(i)}}^t A^{(i)}$ şi $A^{(i+1)} = R^{(i)}Q^{(i)} = ({Q^{(i)}}^t A^{(i)})Q^{(i)} = {Q^{(i)}}^t A^{(i)}Q^{(i)}$.

Acest operații asigură faptul că $A^{(i+1)}$ este o matrice simetrică ce are aceleași valori proprii ca $A^{(i)}$ și, având în vedere că inițial $A^{(1)} = A$, înseamnă că $A^{(i+1)}$ are aceleași valori proprii ca matricea A. Tridiagonalitatea matricei $A^{(i+1)}$ este asigurată de modul în care definim $R^{(i)}$ și $Q^{(i)}$.

Matrice de rotație

Pentru a putea descrie construirea matricelor $Q^{(i)}$ și $R^{(i)}$, este necesară definirea noțiunii de *matrice de rotație*. O *matrice de rotație* P este diferită de matricea identitate în cel mult patru elemente. Aceste patru elemente sunt: $p_{ii} = p_{jj} = cos\Theta$ și $p_{ij} = -p_{ji} = sin\Theta$, pentru o valoare Θ și $i \neq j$.

Orice matrice de rotație P este ortogonală pentru că definiția implică $PP^t = I$. Pentru orice matrice de rotație P, matricea produs AP este diferită de A doar prin valorile din coloanele i și j, în timp ce matricea produs PA este diferită de A doar prin valorile din liniile i și j. Mai mult, pentru orice $i \neq j$, valoarea unghiului Θ poate fi aleasă astfel încât elementul $(PA)_{ij}$ să se anuleze.

Construcția matricelor Q și R

Pentru a obține matricea superior triunghiulară $R^{(1)}$, sunt necesare mai multe matrice de rotație aplicate asupra matricei A.

$$R^{(1)} = P_n P_{n-1} ... P_2 A^{(1)}$$

Pentru început, construim matricea de rotație P_2 cu:

$$p_{11} = p_{22} = cos\Theta_2, \quad p_{12} = -p_{21} = sin\Theta_2$$

unde

$$sin\Theta_2 = \frac{b_2}{\sqrt{{b_2}^2 + {a_1}^2}}, \quad cos\Theta_2 = \frac{a_1}{\sqrt{{b_2}^2 + {a_1}^2}}$$

Pentru verificare, vom calcula produsul:

$$(-sin\Theta_2)a_1 + (cos\Theta_2)b_2 = \frac{-b_2 a_1}{\sqrt{{b_2}^2 + {a_1}^2}} + \frac{a_1 b_2}{\sqrt{{b_2}^2 + {a_1}^2}} = 0$$

ceea ce înseamnă că elementul din poziția $(2,1)$ din matricea ${A_2}^{(1)} = P_2 A^{(1)}$ este egal cu 0. Înmulțirea lui P_2 cu $A^{(1)}$ modifică liniile 1 și 2, însă având în vedere că matricea $A^{(1)}$ este tridiagonală, doar valoarea elementului de indice (1,3) poate deveni diferit de 0 în matricea ${A_2}^{(1)}$.

În general, matricea P_k este aleasă astfel încât elementul cu indicele $(k, k-1)$ din ${A_k}^{(1)} = P_k {A_{k-1}}^{(1)}$ să fie 0. Ceea ce face ca elementul de indice $(k-1, k+1)$ să devină diferit de 0.

După construirea tuturor matricelor de rotație $P_2, P_3, ...P_n$, putem determina matricele Q și R:

$$R^{(1)} = {A_n}^{(1)} = P_n P_{n-1} ... P_2 A$$
$$Q^{(1)} = {P_2}^t {P_3}^t ... {P_n}^t$$

Ortogonalitatea matricelor de rotație implică:

$$Q^{(1)} R^{(1)} = ({P_2}^t {P_3}^t ... P_n^t)(P_n ... P_3 P_2) A^{(1)} = A^{(1)}$$

Matricea $Q^{(1)}$ este ortogonală deoarece:

$${Q^{(1)}}^t Q^{(1)} = ({P_2}^t {P_3}^t ... P_n^t)^t ({P_2}^t {P_3}^t ... P_n^t) = (P_n ... P_3 P_2)({P_2}^t {P_3}^t ... P_n^t) = I$$

Accelerarea convergenței

Dacă valorile proprii $\lambda_1, \lambda_2, ..., \lambda_n$, ale matricei A, au module diferite, rata de convergență a elementului ${b_{j+1}}^{(i+1)}$ către 0 în matricea $A^{(i+1)}$ depinde de raportul $\left|\frac{\lambda_{i+1}}{\lambda_i}\right|$. Convergența lui ${b_{j+1}}^{(i+1)}$ către 0 determină rata cu care elementul ${a_j}^{(i+1)}$ converge către valoarea proprie corespunzătoare λ_j.

Pentru a accelera convergența, vom implementa o tehnică de deplasare explicită: este aleasă o constantă σ, constantă apropiată de una din valorile proprii ale matricei A. În acest caz, factorizarea devine:

$$A^{(i)} - \sigma I = Q^{(i)} R^{(i)}$$

iar matricea $A^{(i+1)}$ este definită ca fiind:

$$A^{(i+1)} = R^{(i)} Q^{(i)} + \sigma I.$$

Cu această modificare, rata convergenței lui ${b_{j+1}}^{(i+1)}$ către 0 depinde de raportul $\left|\frac{\lambda_{i+1}-\sigma}{\lambda_i-\sigma}\right|$. Acest lucru poate aduce o îmbunătățire semnificativă asupra convergenței elementului ${a_j}^{(i+1)}$ către valoarea proprie corespunzătoare λ_j, în cazul în care σ este mai apropiat de λ_{j+1} decât de λ_j.

Vom schimba constanta σ la fiecare pas, astfel încât, atunci când A are valori proprii distincte în modul, ${b_n}^{(i+1)}$ să conveargă la 0 mai rapid decât oricare alt ${b_j}^{(i+1)}$, pentru orice j mai mic strict ca n. Când ${b_n}^{(i+1)}$ este aproape 0, tragem concluzia că λ_n este aproximativ egal cu ${a_n}^{(i+1)}$, după care eliminăm rândul n și coloana n și continuăm cu determinarea valorii proprii λ_{n-1}. Procesul se încheie în momentul în care am obținut câte o aproximare pentru fiecare valoare proprie a matricei A.

Tehnica de deplasare explicită alege la pasul i constanta σ_i ca fiind egală cu valoarea proprie cea mai apropiată de ${a_n}^{(i)}$ a matricei $E^{(i)}$:

$$E^{(i)} = \begin{bmatrix} {a_{n-1}}^{(i)} & {b_n}^{(i)} \\ {b_n}^{(i)} & {a_n}^{(i)} \end{bmatrix}$$

Probleme rezolvate

Problema 1

Să se determine matricea P cu proprietatea că PA are un element egal cu 0 în poziția $(2,1)$ (linia 2, coloana 1).

$$A = \begin{bmatrix} 3 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 3 \end{bmatrix}$$

Soluție:

Forma lui P este: $P = \begin{bmatrix} cos\Theta & sin\Theta & 0 \\ -sin\Theta & cos\Theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$

Prin urmare, $PA = \begin{bmatrix} 3cos\Theta + sin\Theta & cos\Theta + 3sin\Theta & sin\Theta \\ -3cos\Theta + cos\Theta & -sin\Theta + 3cos\Theta & cos\Theta \\ 0 & 1 & 3 \end{bmatrix}$

Unghiul Θ este ales astfel încât $-3sin\Theta + cos\Theta = 0$, ceea ce înseamnă că $tg\Theta = \frac{1}{3}$.

Rezultă că: $cos\Theta = \frac{3\sqrt{10}}{10}, sin\Theta = \frac{\sqrt{10}}{10}$.

$$PA = \begin{bmatrix} \frac{3\sqrt{10}}{10} & \frac{\sqrt{10}}{10} & 0 \\ -\frac{\sqrt{10}}{10} & \frac{3\sqrt{10}}{10} & 0 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} 3 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 3 \end{bmatrix} = \begin{bmatrix} \sqrt{10} & \frac{3\sqrt{10}}{5} & \frac{\sqrt{10}}{10} \\ 0 & \frac{4\sqrt{10}}{5} & \frac{3\sqrt{10}}{10} \\ 0 & 1 & 3 \end{bmatrix}$$

Observație: După cum putem observa, matricea PA nu este nici simetrică, nici tridiagonală.

Problema 2

Determinați prima iterație a metodei QR pentru matricea A:

$$A = \begin{bmatrix} 3 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 3 \end{bmatrix}$$

Soluție:

$A^{(1)} = A$ și P_2 reprezintă matricea de rotație determinată în cadrul problemei 1.

$$A_2{}^{(1)} = P_2 A^{(1)} = \begin{bmatrix} \frac{3\sqrt{10}}{10} & \frac{\sqrt{10}}{10} & 0 \\ -\frac{\sqrt{10}}{10} & \frac{3\sqrt{10}}{10} & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 3 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 3 \end{bmatrix} = \begin{bmatrix} \sqrt{10} & \frac{3\sqrt{10}}{5} & \frac{\sqrt{10}}{10} \\ 0 & \frac{4\sqrt{10}}{5} & \frac{3\sqrt{10}}{10} \\ 0 & 1 & 3 \end{bmatrix}$$

În continuare, calculăm:

$$sin\Theta_3 = \frac{1}{\sqrt{(1^2 + (\frac{4\sqrt{10}}{5})^2}} = 0.36761, \quad cos\Theta_3 = \frac{\frac{4\sqrt{10}}{5}}{\sqrt{(1^2 + (\frac{4\sqrt{10}}{5})^2}} = 0.92998$$

Vom determina $R^{(1)} = A_3{}^{(1)} = P_3 A_2{}^{(1)}$ și $Q^{(1)} = P_2{}^t P_3{}^t$ astfel:

$$R^{(1)} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.92998 & 0.36761 \\ 0 & -0.36761 & 0.92998 \end{bmatrix} \cdot \begin{bmatrix} \sqrt{10} & \frac{3\sqrt{10}}{5} & \frac{\sqrt{10}}{10} \\ 0 & \frac{4\sqrt{10}}{5} & \frac{3\sqrt{10}}{10} \\ 0 & 1 & 3 \end{bmatrix} =$$

$$\begin{bmatrix} \sqrt{10} & \frac{3\sqrt{10}}{5} & \frac{\sqrt{10}}{10} \\ 0 & 2.7203 & 1.9851 \\ 0 & 0 & 2.4412 \end{bmatrix}$$

$$Q^{(1)} = \begin{bmatrix} \frac{3\sqrt{10}}{10} & -\frac{\sqrt{10}}{10} & 0 \\ \frac{\sqrt{10}}{10} & \frac{3\sqrt{10}}{10} & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.92998 & -0.36761 \\ 0 & 0.36761 & 0.92998 \end{bmatrix} =$$

$$\begin{bmatrix} 0.94868 & -0.29409 & 0.11625 \\ 0.31623 & 0.88226 & -0.34874 \\ 0 & 0.36761 & 0.92998 \end{bmatrix}$$

În consecință, $A^{(2)} = R^{(1)} Q^{(1)}$

$$A^{(2)} = \begin{bmatrix} \sqrt{10} & \frac{3\sqrt{10}}{5} & \frac{\sqrt{10}}{10} \\ 0 & 2.7203 & 1.9851 \\ 0 & 0 & 2.4412 \end{bmatrix} \cdot \begin{bmatrix} 0.94868 & -0.29409 & 0.11625 \\ 0.31623 & 0.88226 & -0.34874 \\ 0 & 0.36761 & 0.92998 \end{bmatrix} =$$

$$\begin{bmatrix} 3.6 & 0.8602 & 0 \\ 0.8602 & 3.1297 & 0.8974 \\ 0 & 0.8974 & 2.2702 \end{bmatrix}$$

Elementele de sub diagonala principală din matricea $A^{(2)}$ sunt mai mici decât cele din matricea $A^{(1)}$ cu aproximativ 14%. Pentru a obține valori sub 0.001, vor fi necesare 13 iterații folosind algoritmul QR. După 13 iterații, vom obține:

$$A^{(13)} = \begin{bmatrix} 4.4139 & 0.01941 & 0 \\ 0.01941 & 3.0003 & 0.00095 \\ 0 & 0.00095 & 1.5858 \end{bmatrix}$$

Ceea ce înseamnă că am determinat aproximația unei valori proprii a matricei A, 1.5858 și că putem determina și celelalte două valori proprii calculând valorile proprii ale matricei reduse:

$$\begin{bmatrix} 4.4139 & 0.01941 \\ 0.01941 & 3.0003 \end{bmatrix}$$

Problema 3

Aplicați metoda QR cu deplasare explicită pentru matricea A:

$$A = \begin{bmatrix} 3 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 3 \end{bmatrix}$$

Soluție:

Pentru a determina factorul de accelerare σ_1, trebuie să determinăm valorile proprii ale matricei $\begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$ care sunt $\mu_1 = 4$ și $\mu_2 = 2$. Cum ambele valori sunt la fel de depărtate de valoarea lui ${a_3}^{(1)}$, alegem pe oricare dintre cele două, de exemplu, pe $\mu_2 = 2$. Prin urmare, $\sigma_1 = 2$.

$$A^{(1)} - \sigma_1 I = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

Continuăm cu determinarea lui ${A_2}^{(1)}$, ca în cazul algoritmului fără deplasare:

$${A_2}^{(1)} = \begin{bmatrix} \sqrt{2} & \sqrt{2} & \frac{\sqrt{2}}{2} \\ 0 & 0 & \frac{\sqrt{2}}{2} \\ 0 & 1 & 1 \end{bmatrix}$$

Determinăm $R^{(1)} = {A_3}^{(1)} = \begin{bmatrix} \sqrt{2} & \sqrt{2} & \frac{\sqrt{2}}{2} \\ 0 & 1 & 1 \\ 0 & 0 & -\frac{\sqrt{2}}{2} \end{bmatrix}$ și

$$A^{(2)} = R^{(1)}Q^{(1)} = \begin{bmatrix} 2 & \frac{\sqrt{2}}{2} & 0 \\ \frac{\sqrt{2}}{2} & 1 & -\frac{\sqrt{2}}{2} \\ 0 & -\frac{\sqrt{2}}{2} & 0 \end{bmatrix}$$

Am terminat o iterație a algoritmului QR. Nici ${b_2}^{(2)} = \frac{\sqrt{2}}{2}$, nici ${b_3}^{(2)} = -\frac{\sqrt{2}}{2}$ nu sunt suficient de apropiate de 0, așa că vom calcula și pasul următor al algoritmului QR. De data aceasta, determinăm valorile proprii ale matricei:

$$\begin{bmatrix} 1 & -\frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} & 0 \end{bmatrix}$$

Acestea sunt $\frac{1}{2} \pm \frac{1}{2}\sqrt{3}$. Determinăm cea mai apropiată valoare proprie de ${a_3}^{(2)} = 0$. Rezultă $\sigma_2 = \frac{1}{2} - \frac{1}{2}\sqrt{3}$.

$$A^{(3)} = \begin{bmatrix} 2.6720277 & 0.37597448 & 0 \\ 0.37597448 & 1.4736080 & 0.030396964 \\ 0 & 0.030396964 & -0.047559530 \end{bmatrix}$$

Dacă ${b_3}^{(3)} = 0.030396964$ este suficient de apropiat de 0, atunci aproximația valorii proprii λ_3 este 1.5864151, suma dintre ${a_3}^{(3)}$ și $\sigma_1 + \sigma_2 = 2 + \frac{1-\sqrt{3}}{2}$. Eliminând a treia linie și a treia coloană din $A^{(3)}$ obţinem:

$$A^{(3)} = \begin{bmatrix} 2.6720277 & 0.37597448 \\ 0.37597448 & 1.4736080 \end{bmatrix}$$

cu valorile proprii μ_1=2.7802140 și μ_2=1.3654218. Prin urmare, $\lambda_1 \approx \mu_1 + \sigma_1 + \sigma_2 = 4.4141886$ și $\lambda_2 \approx \mu_2 + \sigma_1 + \sigma_2 = 2.9993964$.

Valorile proprii exacte ale matricei A sunt $\lambda_1 = 4.41420$, $\lambda_2 = 3.00000$, și $\lambda_3 = 1.58579$, ceea ce demostrează că algoritmul QR cu deplasare explicită oferă precizie bună și în cazul unui număr mic de iterații.

Probleme propuse

Problema 1

Implementați în OCTAVE algoritmul QR fără deplasare pentru determinarea valorilor proprii ale unei matrice simetrice tridiagonale. Date de intrare: A - matricea simetrică tridiagonală; n - dimensiunea matricei; *tol* - toleranța acceptată; *maxiter* - numărul maxim de iterații. Date de ieșire: valorile proprii ale matricei A sau un mesaj de eroare în cazul în care a fost depășit *maxiter*.

Problema 2

Pornind de la programul anterior, realizați implementarea algoritmului QR cu deplasare explicită pentru a determina valorile proprii ale unei matrice simetrice tridiagonale.

Problema 3

Determinați primele două iterații ale algoritmului QR fără deplasare pentru matricile simetrice tridiagonale următoare:

$$(a)\quad \begin{bmatrix} 6 & 1 & 0 \\ 1 & 4 & 2 \\ 0 & 2 & 3 \end{bmatrix} \qquad (b)\quad \begin{bmatrix} 2 & 4 & 0 \\ 4 & 2 & -3 \\ 0 & -3 & -1 \end{bmatrix}$$

$$(c)\quad \begin{bmatrix} 2 & -3 & 0 & 0 \\ -3 & 3 & -2 & 0 \\ 0 & -2 & 4 & -1 \\ 0 & 0 & -1 & 1 \end{bmatrix} \qquad (d)\quad \begin{bmatrix} 2 & -2 & 0 & 0 \\ -2 & 2 & 4 & 0 \\ 0 & 4 & 2 & -2 \\ 0 & 0 & -2 & 3 \end{bmatrix}$$

Problema 4

Folosind algoritmul QR cu deplasare explicită, determinați valorile proprii ale matricelor de la *Problema* 3 cu o toleranță de 10^{-5}.

Metode Numerice @ CS-UPB

Echipa MN CS - 2015:

Profesori:

- Pantelimon George Popescu - Seria CA;
- Florin Pop - Seria CB, CC;

Asistenti:

PG Popescu, Clementin Cercel, Sorin Ciolofan, Bogdan Tiganoaia, Alexandru Tifrea, David Iancu, Bogdan Cristian Marchis, Radu Poenaru, Madalina Grosu, Andrei Vlad Postoaca, Radu Constantinescu, Roxana Istrate, Madalina Hristache, Adelina Vidovici;

Interpolare



Interpolare Polinomiala

O functie f:[a,b]->R este cunoscuta intr-un set de puncte x1,x2,...,xn si are valorile f1=f(x1),f2=f(x2),...,fn=f(xn).

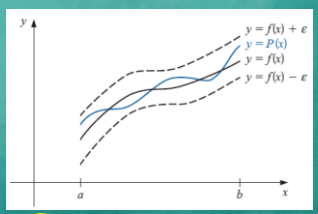
Ne propunem sa aproximama aceasta functie printr-un polinom

$$P_n(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0$$

Rezultatul care ne permite sa aproximam printr-un polinom este dat de teorema lui Weierstrass (Karl Theodor Wilhelm Weierstrass (1815-1897) mat. german) si adica

Interpolare Polinomiala

Fie f:[a,b]->R o functie. Pentru orice ε>0 exista un polinom P(x), astfel incat $|f(x) - P(x)| < \epsilon$ oricare ar fi x din [a,b].



Interpolare Polinomiala Lagrange

Fie $l_k(x) = c_k(x - x_1)\dots(x - x_{k-1}) \cdot (x - x_{k+1})\dots(x - x_n)$ cu conditia ca lk(xk)=1, atunci

$$c_k = \frac{1}{(x_k - x_0)\cdots(x_k - x_{k-1})(x_k - x_{k+1})\cdots(x_k - x_n)}$$

si deci rezulta $l_k(x) = \prod_{i=0, i\neq k}^{n} \frac{x - x_i}{x_k - x_i}$

Avem urmatoarea teorema

Interpolare Polinomiala Lagrange

Daca x0,x1,...,xn sunt n+1 puncte distincte in care o functie f este cunoscuta, atunci exista un polinom unic de grad cel mult n, P(x) care satisface P(xi)=f(xi) si are forma

$$P_n(x) = \sum_{k=0}^{n} f(x_k) \cdot \prod_{i=0, i\neq k}^{n} \frac{x - x_i}{x_k - x_i}$$

Polinomul poarta in acest caz numele de polinom de interpolare Lagrange(Joseph-Louis Lagrange (1736-1813), mat, italian)

Interpolare Polinomiala Lagrange

Polinoamele $l_k(x) = \prod_{i=0, i\neq k}^{n} \frac{x - x_i}{x_k - x_i}$ se numesc multiplicatori Lagrange.

Coeficientii polinomului de interpolare Lagrange pot fi calculati prin adunarea a n+1 polinoame de gradul n, obtinute prin dezvoltarea produselor ce apar in

$$P_n(x) = \sum_{k=0}^{n} f(x_k) \cdot \prod_{i=0, i\neq k}^{n} \frac{x - x_i}{x_k - x_i}$$

In scopul reducerii nr. de operatii se face astfel: P(x0,x1,....,xn;x)=P01...n(x) si se impart pe rand cu (x-xk), iar coeficientii polinoamelor astfel obtinute, ponderati cu f(xk) se aduna intre ei. O(n^3).

Interpolare Polinomiala Lagrange

Exercitiu: (Calculul polinomului de interpolare Lagrange intr-un punct a) Forma Matriciala, relativ eficient MATLAB

$$V = \begin{bmatrix} 1 & a - x_1 & \cdots & a - x_1 \\ a - x_2 & 1 & \cdots & a - x_2 \\ \cdots & \cdots & \cdots & \cdots \\ a - x_n & a - x_n & & 1 \end{bmatrix}$$

$$U = \begin{bmatrix} 1 & x_2 - x_1 & \cdots & x_n - x_1 \\ x_1 - x_2 & 1 & \cdots & x_n - x_2 \\ \cdots & \cdots & \cdots & \cdots \\ x_1 - x_n & x_2 - x_n & \cdots & 1 \end{bmatrix}$$

Interpolare Polinomiala Lagrange

Exercitiu: (Calculul polinomului de interpolare Lagrange intr-un punct a) Forma Matriciala, relativ eficient MATLAB

$$L = \begin{bmatrix} l_1 & l_2 & \cdots & l_n \end{bmatrix} = \text{prod}(V) \cdot / \text{prod}(U)$$

Si deci valoarea polinomului L in a este

$$b = \sum_{i=1}^{n} l_i \cdot y_i = \text{prod}(V) \cdot / \text{prod}(U) * y$$

Interpolare Polinomiala Lagrange

Exercitiu: (Calculul polinomului de interpolare Lagrange intr-un punct a) Forma Matriciala, relativ eficient MATLAB

$$U = \begin{bmatrix} x_1 & x_1 & \cdots & x_1 \\ x_2 & x_2 & \cdots & x_2 \\ \cdots & \cdots & \cdots & \cdots \\ x_n & x_n & \cdots & x_n \end{bmatrix} - \begin{bmatrix} x_1 & x_2 & \cdots & x_n \\ x_1 & x_2 & \cdots & x_n \\ \cdots & \cdots & \cdots & \cdots \\ x_1 & x_2 & \cdots & x_n \end{bmatrix} + I_n$$

$$U = \begin{bmatrix} x_1 & 0 & \cdots & 0 \\ 0 & x_2 & \cdots & 0 \\ \cdots & \cdots & \cdots & \cdots \\ 0 & 0 & \cdots & x_n \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & 1 & \cdots & 1 \\ \cdots & \cdots & \cdots & \cdots \\ 1 & 1 & \cdots & 1 \end{bmatrix} - \begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & 1 & \cdots & 1 \\ \cdots & \cdots & \cdots & \cdots \\ 1 & 1 & \cdots & 1 \end{bmatrix} \cdot \begin{bmatrix} x_1 & 0 & \cdots & 0 \\ 0 & x_2 & \cdots & 0 \\ \cdots & \cdots & \cdots & \cdots \\ 0 & 0 & \cdots & x_n \end{bmatrix} + I_n$$

```
U=diag(x)*ones(n)-ones(n)*diag(x)+eye(n)
```


Interpolare Polinomiala Lagrange

Exercitiu: (Calculul polinomului de interpolare Lagrange intr-un punct a) Forma Matriciala, relativ eficient MATLAB

$$V = \begin{bmatrix} 0 & a & \cdots & a \\ a & 0 & \cdots & a \\ \cdots & \cdots & \cdots & \cdots \\ a & a & \cdots & 0 \end{bmatrix} - \begin{bmatrix} 0 & x_1 & \cdots & x_1 \\ x_2 & 0 & \cdots & x_2 \\ \cdots & \cdots & \cdots & \cdots \\ x_n & x_n & \cdots & 0 \end{bmatrix} + I_n$$

$$V = a * \begin{bmatrix} 0 & 1 & \cdots & 1 \\ 1 & 0 & \cdots & 1 \\ \cdots & \cdots & \cdots & \cdots \\ 1 & 1 & \cdots & 0 \end{bmatrix} - \begin{bmatrix} x_1 & 0 & \cdots & 0 \\ 0 & x_2 & \cdots & 0 \\ \cdots & \cdots & \cdots & \cdots \\ 0 & 0 & \cdots & x_n \end{bmatrix} \cdot \begin{bmatrix} 0 & 1 & \cdots & 1 \\ 1 & 0 & \cdots & 1 \\ \cdots & \cdots & \cdots & \cdots \\ 1 & 1 & \cdots & 0 \end{bmatrix} + I_n$$

```
V=(a-diag(x))*~eye(n)+eye(n)
```


Interpolare Polinomiala Metoda Neville

Polinomul de interpolare de gr n poate fi calculat si prin recurenta, folosind polinoame de interpolare de grad mai mic.

Daca notam $\sigma=\{i_1, i_2, \ldots, i_p\}$ si P_σ polinomul de interpolare ce trece prin punctele

$$(x_{i1}, y_{i1}), (x_{i2}, y_{i2}), \ldots, (x_{ip}, y_{ip})$$

Atunci polinomul de interpolare pentru ansamblul extins $\sigma+j+k = \sigma \cup \{j,k\}$ se defineste astfel

$$P_{\sigma+j+k}(x) = \frac{(x - x_j)P_{\sigma+k}(x) - (x - x_k)P_{\sigma+j}(x)}{x_k - x_j}$$

Interpolare Polinomiala Metoda Neville

Se observa ca Pσ+j+k(xi)=yi, oricare ar fi i din σ+j+k.

Metoda Neville(Eric Harold Neville, (1889-1961) mat. englez) considera astfel

$$Q_{ij}(x) = \frac{(x - x_{i-j}) \cdot Q_{i,j-1}(x) - (x - x_i) \cdot Q_{i-1,j-1}(x)}{x_i - x_{i-j}}$$

unde Qij=Pi-j,...,i si trece prin punct. (xi-j,yi-j),...,(xi,yi).

Interpolare Polinomiala Dif. Divizate

Notam cu

$$F_0[x_0] = f(x_0)$$

$$F_1[x_0, x_1] = \frac{F_0[x_0] - F_0[x_1]}{x_0 - x_1}$$

$$F_p[x_0, \dots, x_p] = \frac{F_{p-1}[x_0, \dots, x_{p-1}] - F_{p-1}[x_1, \dots, x_p]}{x_0 - x_p}$$

$$F_p[x_0, \dots, x_p] = \sum_{k=0}^{p} \frac{f(x_k)}{\pi'(x_k)} = \sum_{k=0}^{p} \frac{f(x_k)}{\prod_{i=0}^{p} (x_k - x_i)}$$

ce se deduce prin inductie.

Interpolare Polinomiala Dif. Divizate

Consideram

$$F_1[x_1, x_1] = \lim_{x_0 \to x_1} F_1[x_0, x_1] = \lim_{x_0 \to x_1} \frac{f(x_0) - f(x_1)}{x_0 - x_1} = f'(x_1),$$

$$F_{p-1}[\underbrace{x_i, \dots, x_i}_{p}] = f^{(p-1)}(x_i)$$

care se deduce tot prin inductie.

Interpolare Polinomiala Dif. Divizate

Identitatea lui Newton(Isaac Newton (1642-1727) mat. si fizician englez) stabileste o alta forma pentru polinomul de interpolare Lagrange, care pune in evidenta expresia erorii.

$$(x - x_0) \cdot F_1[x, x_0] = f(x) - f(x_0)$$

$$(x - x_0) \cdot (x - x_1) \cdot F_2[x, x_0, x_1] = F_1[x, x_0] - F_1[x_0, x_1]$$

$$(x - x_0) \cdot (x - x_1) \cdot (x - x_2) \cdot F_3[x, x_0, x_1, x_2] = F_2[x, x_0, x_1] - F_2[x_0, x_1, x_2]$$

...

$$(x - x_0) \dots (x - x_{n-1})(x - x_n) F_{n+1}[x, x_0, \dots, x_n] = F_n[x, x_0, \dots, x_{n-1}] - F_n[x_0, x_1, x_n]$$

Interpolare Polinomiala Dif. Divizate

de unde sumand toate egalitatile vom obtine

$$f(x) = f(x_0) + (x - x_0) \cdot F_1[x_0, x_1] + (x - x_0) \cdot (x - x_1) \cdot F_2[x_0, x_1, x_2] + \dots$$
$$+ (x - x_0) \dots (x - x_{n-1}) \cdot F_n[x_0, \dots, x_n] + (x - x_0) \dots (x - x_n) \cdot F_{n+1}[x, x_0, \dots, x_n]$$

si notand cu Pn(x) prima parte va rezulta

$$f(x) = P_n(x) + R_n(x)$$

$$R_n(x) = (x - x_0) \dots (x - x_n) \cdot F_{n+1}[x, x_0, \dots, x_n]$$

Polinomul Rn(x) are n+1 radacini, R'n(x) va avea n radacini sasmd R^(n)_n(x) va avea o radacina.

Interpolare Polinomiala Dif. Divizate

dar R^(n)_n(x) = f^(n)(x)-P^(n)_n(x) si stim ca

$$P_n^{(n)}(x) = n! \cdot F_n[x_0, x_1, \dots, x_n]$$

atunci ecuatia $f^{(n)}(x) - n! \cdot F_n[x_0, x_1, \dots, x_n] = 0$

consideram ca are solutia ξ si deci rezulta ca

$$F_n[x_0, x_1, \dots, x_n] = \frac{f^{(n)}(\xi)}{n!} \quad \xi \in [x_0, x_n]$$

Interpolare Polinomiala Dif. Divizate

reamintim ca

$$R_n(x) = (x - x_0)\dots(x - x_n)\cdot F_{n+1}[x, x_0, x_1, \dots, x_n]$$

si aplicand modul vom obtine

$$|R_n(x)| \le |(x - x_0)\dots(x - x_n)| \cdot \frac{|f^{(n+1)}(\xi)|}{(n+1)!}$$

Interpolare Polinomiala Dif. Divizate

Fie x0,x1,…,xn un set de puncte distincte din [a,b] si f o functie de clasa C^(n+1)pe [a,b]. Atunci pentru orice x din [a,b], exista un ξ=ξ(x) intre punctele x0,x1,…,xn, adica in [a,b] astfel incat

$$f(x) = P(x) + \frac{f^{(n+1)}(\xi(x))}{(n+1)!}(x - x_0)(x - x_1)\cdots(x - x_n)$$

unde P(x) este polinomul Lagrange.

Interpolare Polinomiala Dif. Finite

Se considera puncte echidistante $x_i = x_0 + ih$ si notam:

- Diferente progresive(inainte)

$$\Delta f(x_i) = f(x_i + h) - f(x_i) = f(x_{i+1}) - f(x_i)$$
$$\Delta^k f(x_i) = \Delta^{k-1} f(x_{i+1}) - \Delta^{k-1} f(x_i)$$

- Diferente regresive(inapoi)

$$\nabla f(x_i) = f(x_i) - f(x_i - h) = f(x_i) - f(x_{i-1})$$
$$\nabla^k f(x_i) = \nabla^{k-1} f(x_i) - \nabla^{k-1} f(x_{i-1})$$

Interpolare Polinomiala Dif. Finite

- Diferente centrate

$$\delta f(x_i) = f\left(x_i + \frac{h}{2}\right) - f\left(x_i - \frac{h}{2}\right) = f\left(x_{i+\frac{1}{2}}\right) - f\left(x_{i-\frac{1}{2}}\right)$$
$$\delta^k f(x_i) = \delta^{k-1} f\left(x_{i+\frac{1}{2}}\right) - \delta^{k-1} f\left(x_{i-\frac{1}{2}}\right)$$

Trecerea de la diferente finite la diferente divizate se face astfel:

$$\Delta^n f(x_i) = \nabla^n f(x_{i+n}) = \delta^n f(x_{i+n/2}) = n!\, h^n F_n[x_i \dots x_{i+n}]$$

Interpolare Polinomiala

Formulele de interpolare Newton(se mai numesc si Newton - Gregory)

Fie f o functie cunoscuta in punctele (xi,yi), i=0:n cu xi+1=xi+h, i=0:n-1 si vrem sa evaluam functia f intr-un punct x!=xi. Vom considera x in (x0,x1) sau x in (xn-1,xn), fara a restrange generalitatea, deoarece daca x se afla la mijloc, atunci nu mai consider primele sau ultimele valori unde functia f este cunoscuta si deci reduc problema la unul din cele doua cazuri.

Interpolare Polinomiala

Formulele de interpolare Newton

Caz 1(la inceput de tablou): x=x0+uh, 0<u<1. Aplicand Identitatea lui Newton in x cu diferente divizate exprimate prin diferente progresive, adica

$$F_k[x_0, x_1, \dots, x_k] = \frac{\Delta^k f(x_0)}{k! h^k}$$

obtinem:

$$P_1(x) = P_1(x_0 + u\cdot h) = p_1(u) = f(x_0) + \frac{x - x_0}{1!h}\cdot\Delta f(x_0) + \frac{(x - x_0)\cdot(x - x_1)}{2!h^2}\cdot\Delta^2 f(x_0) + \dots + \frac{(x - x_0)\cdot(x - x_1)\dots(x - x_{n-1})}{n!h^n}\cdot\Delta^n f(x_0)$$

Interpolare Polinomiala

iar pentru ca

$$x - x_k = x - x_0 - (x_k - x_0) = u \cdot h - kh = (u - k)h$$

rezulta

$$p_1(u) = f_0 + \frac{u}{1!} \cdot \Delta f_0 + \frac{u \cdot (u-1)}{2!} \cdot \Delta^2 f_0 + \ldots + \frac{u \cdot (u-1) \ldots (u-n+1)}{n!} \cdot \Delta^n f_0$$

de unde, considerand conceptul generalizat de combinare vom avea formula de interpolare Newton 1

$$p_1(u) = f_0 + \binom{u}{1} \cdot \Delta f_0 + \binom{u}{2} \cdot \Delta^2 f_0 + \ldots + \binom{u}{n} \cdot \Delta^n f_0$$



Interpolare Polinomiala

Caz 2(la sfarsit de tablou): daca consideram x=xn-uh, cu 0<u<1, aplicand Identitatea Newton in x cu diferente divizate exprimate prin diferente regresive vom obtine

$$p_2(u) = f_0 - \binom{u}{1} \nabla f_n + \binom{u}{2} \nabla^2 f_n + \ldots + (-1)^n \binom{u}{n} \nabla^n f_n$$

ce reprezinta formula de interpolare Newton 2.

lar daca consideram x=xn+uh, cu -1<u<0 vom obtine:

$$p_3(u) = f_n + \binom{u}{1} \cdot \nabla f_n + \binom{u+1}{2} \cdot \nabla^2 f_n + \ldots + \binom{u+n-1}{n} \cdot \nabla^n f_n$$

ce reprezinta formula de interpolare Newton 3.



Interpolare Polinomiala Hermite

La conditiile ca polinomul de interpolare sa treaca prin punctele date, P2n+1(xi)=f(xi) se mai adauga si derivatele, P'2n+1(xi)=f'(xi), i=0:n de unde rezulta 2n+2 conditii de interpolare. Deci polinomul de interpolare Hermite(Charles Hermite (1822-1901) mat. francez) are gradul 2n+1 si arata astfel:

$$H_{2n+1}(x) = \sum_{j=0}^{n} f(x_j) H_{n,j}(x) + \sum_{j=0}^{n} f'(x_j) \hat{H}_{n,j}(x)$$ unde

$$H_{n,j}(x) = [1 - 2(x - x_j) L'_{n,j}(x_j)] L^2_{n,j}(x)$$ si $$\hat{H}_{n,j}(x) = (x - x_j) L^2_{n,j}(x)$$



Interpolare Polinomiala Hermite

unde Ln,j=lj sunt multiplicatorii Lagrange, adica

$$l_k(x) = \prod_{i=0, i \neq k}^{n} \frac{x - x_i}{x_k - x_i}$$

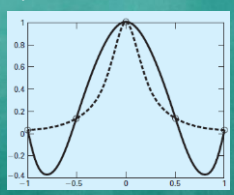
de unde

$$l'_k(x) = \prod_{i=0, i \neq k}^{n} \frac{1}{x_k - x_i}$$



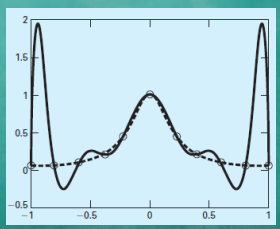
Interpolare Polinomiala Oscilatii

Daca ne propunem sa aproximam functia $f(x) = \frac{1}{1+25x^2}$ numita functia lui Carl Runge(1901), pe intervalul [-1,1] considerand 5 puncte echidistante vom obtine

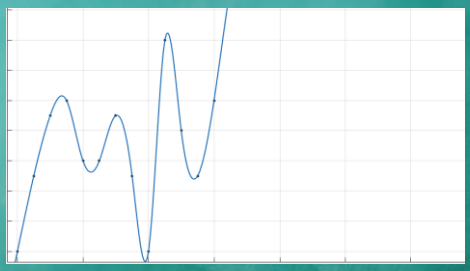


Interpolare Polinomiala Oscilatii

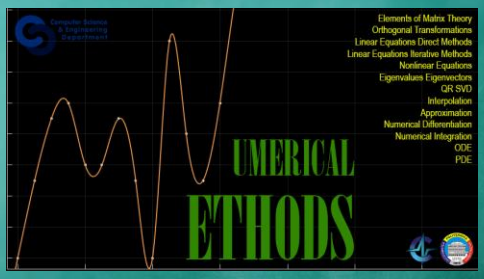
Daca ne propunem sa aproxima functia Runge, pe intervalul [-1,1] considerand 11 puncte echidistante vom obtine



Interpolare Polinomiala LogoMN



Interpolare Polinomiala LogoMN



Interpolare Spline

Curbele in general pot fi reprezentate explicit, implicit sau parametric(dependent de coordonate).

Un SPLINE este o curba parametrica definita prin punctele de control.

Un SPLINE este o functie S:[a,b]->R definita local pe mai multe intervale prin functiile Pi:[xi,xi+1)->R cu a=x0<x1<…<xk-1=b, unde

S(x)=P0(x), a<=x<x1; … S(x)=Pk-2(x), xk-2<=x<xk-1;

Interpolare Spline

De obicei:

- functiile Pi sunt polinoame de grad 3;
- Punctele xi sunt echidistante -> SPLINE uniform;
- Functiile SPLINE pot fi functii de interpolare(trec prin toate punctele de cotrol) sau de aproximare(nu trec prin toate punctele de control)

Adobe si PostScript folosesc SPLINE-uri ce au continuitatea de clasa C1;

Interpolare Spline

SPLINE de clasa C1 - polinoame lineare;

$x_0 < x_1 < \dots < x_n$
$f(x_0), f(x_1), \dots, f(x_n)$

şi vom considera funcţii de interpolare liniare, locale pe subintervalele

$[x_0, x_1], [x_1, x_2], \dots, [x_{n-1}, x_n]$

$p_i(x) = a_i x + b_i, \quad i = 0 : n-1$

în care cei $2n$ parametri se determină din *condiţiile de interpolare:*

$p_i(x_i) = f(x_i), \quad i = 0 : n-1$
$p_{n-1}(x_n) = f(x_n)$

şi a *condiţiilor de racordare* (*continuitate* în punctele interioare):

$p_i(x_{i+1}) = p_{i+1}(x_{i+1}), \quad i = 0 : n-2$

Interpolare Spline

SPLINE de clasa C1 - polinoame lineare;

$$a_i = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}, \qquad i = 0 : n-1$$

$$b_i = \frac{x_{i+1} \cdot f(x_i) - x_i \cdot f(x_{i+1})}{x_{i+1} - x_i}$$

Interpolare Spline

SPLINE de clasa C1 - polinoame de grad 3(Hermite);

Aici este necesar sa cunoastem si valoarea derivatei functiei in punctele suport.

$$S_i(x) = a_i + b_i(x - xi) + c_i(x - xi)^2 + d_i(x - xi)^3$$

sau *parametric*

$$S_i(t) = a_i + b_i h_i t + c_i h_i^2 t^2 + d_i h_i^3 t^3, \quad t \in [0,1]$$

în care s-a notat $h_i = x_{i+1} - x_i$ şi s-a efectuat schimbarea de variabilă:

$$t = \frac{x - x_i}{x_{i+1} - x_i} = \frac{x - x_i}{h_i}$$



Interpolare Spline

SPLINE de clasa C1 - polinoame de grad 3(Hermite);

De obicei se foloseste baza Bernstein(Sergei Natanovich Bernstein(1880-1968) mat. rus) pentru simplificarea volumului de calcul;

$$(1-t)^3,\ 3t(1-t)^2,\ 3t^2(1-t),\ t^3 \quad \text{cu } t \in [0,1]$$

deci s_i se rescriu astfel

$$s_i(t) = a'_i(1-t)^3 + 3b'_i t(1-t)^2 + 3c'_i t^2(1-t) + d'_i t^3$$

iar cei 4n coeficienti se deduc din urmatoarele conditii



Interpolare Spline

SPLINE de clasa C1 - polinoame de grad 3(Hermite);

Avem **2n+2** condiţii de interpolare de tip Hermite:

$$s_i(x_i) = f(x_i)$$
$$s'_i(x_i) = f'(x_i), \quad i = 0:n$$

şi **2n-2** condiţii de racordare (continuitate şi derivabilitate în punctele interioare):

$$s_i(x_{i+1}) = s_{i+1}(x_{i+1})$$
$$s'_i(x_{i+1}) = s'_{i+1}(x_{i+1}), \quad i = 0:n-2$$

$$a'_i = f(x_i) \qquad b'_i = f(x_i) + \frac{h_i}{3} f'(x_i)$$

$$d'_i = f(x_{i+1}) \qquad c'_i = f(x_{i+1}) - \frac{h_i}{3} f'(x_{i+1})$$

$$s_i(t) = y_i(1-t)^3 + (3y_i + h_i y'_i)t(1-t)^2 + (3y_{i+1} - hy'_{i+1})t^2(1-t) + y_{i+1}t^3$$



Interpolare Spline

SPLINE de clasa C2 - polinoame de grad 3;

Considerăm numai **n+1** condiţii de interpolare de tip Lagrange:

$$s_i(x_i) = f(x_i), \quad i = 0:n-1$$
$$s_{n-1}(x_n) = f(x_n)$$

rezultă:

$$a_i = f(x_i), \quad i = 0:n-1$$

$$a_{n-1} + b_{n-1}h_{n-1} + c_{n-1}h_{n-1}^2 + d_{n-1}h_{n-1}^3 = f(x_n) \equiv a_n$$

$$a_i = f(x_i), \qquad i = 0 : n$$



Interpolare Spline

SPLINE de clasa C2 - polinoame de grad 3;

Conditii de racordare:

- continuitate de ordin 0(n-1 conditii)

$$s_i(x_{i+1}) = s_{i+1}(x_{i+1})$$

$$a_{i+1} = a_i + b_i h_i + c_i h_i^2 + d_i h_i^3 \qquad i = 0 : n-2$$

$$a_n \equiv a_{n-1} + b_{n-1}h_{n-1} + c_{n-1}h_{n-1}^2 + d_{n-1}h_{n-1}^3$$



Interpolare Spline

SPLINE de clasa C2 - polinoame de grad 3;

Conditii de racordare:

- continuitate derivatelor, ordin 1(n-1 conditii)

$$s'_i(x) = b_i + 2c_i(x - x_i) + 3d_i(x - x_i)^2$$

$$s'_i(x_{i+1}) = s'_{i+1}(x_{i+1}) \qquad i = 0 : n-2$$

$$b_{i+1} = b_i + 2c_i h_i + 3d_i h_i^2$$

$$b_n \equiv b_{n-1} + 2c_{n-1}h_{n-1} + 3d_{n-1}h_{n-1}^2$$



Interpolare Spline

SPLINE de clasa C2 - polinoame de grad 3;
Conditii de racordare:

- continuitate derivatelor de ordin 2(n-1 conditii)

$$s_i''(x) = 2c_i + 6d_i(x - x_i)$$
$$s_i''(x_{i+1}) = s_{i+1}''(x_{i+1})$$
$$c_{i+1} = c_i + 3d_i h_i$$
$$c_n \equiv c_{n-1} + 3d_{n-1}h_{n-1}$$

Interpolare Spline

SPLINE de clasa C2 - polinoame de grad 3;
Deci adunand numarul conditiilor de interpolare (n+1) cu numarul de conditii de racordare (3n-3) obtinem 4n-2 conditii si mai avem nevoie de inca doua. In acest context definim conceptele de SPLINE natural si SPLINE tensionat, astfel:

- Natural: $s_0''(x_0) = 0, \quad s_{n-1}''(x_n) = 0$
- Tensionat: $s_0'(x_0) = f'(x_0), \quad s_{n-1}'(x_n) = f'(x_n)$

Interpolare Spline

SPLINE de clasa C2 - polinoame de grad 3;
Si deci pentru calculul parametrilor(vom transforma totul in functie de ci) vom avea

$$a_i = f(x_i), \qquad i = 0 : n-1$$
$$d_i = \frac{c_{i+1} - c_i}{3h_i}, \qquad i = 0 : n-1$$
$$b_i = \frac{a_i - a_{i-1}}{h_{i-1}} + \frac{c_{i-1} + 2c_i}{3} h_{i-1}, \qquad i = 1 : n-1$$
$$h_{i-1} \cdot c_{i-1} + 2 \cdot (h_{i-1} + h_i) \cdot c_i + h_i \cdot c_{i+1} = \frac{3 \cdot (a_{i+1} - a_i)}{h_i} - \frac{3 \cdot (a_i - a_{i-1})}{h_{i-1}}$$

Interpolare Spline

SPLINE de clasa C2 - pol. de grad 3; NATURAL

- plus conditiile de la spline-uri naturale:

$$s_0''(x_0) = 2c_0 + 6d_0(x_0 - x_0) = 0 \Rightarrow c_0 = 0$$
$$s_{n-1}''(x_n) = 2c_{n-1} + 6d_{n-1}h_{n-1} \equiv 2c_n = 0 \Rightarrow c_n = 0$$

De unde vom obtine un sistem de forma Ax=b:

Interpolare Spline

SPLINE de clasa C2 - pol. de grad 3; NATURAL

$$A = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ h_0 & 2(h_0 + h_1) & h_1 & & \\ 0 & h_1 & 2(h_1 + h_2) & h_2 & 0 \\ & & \ddots & \ddots & \ddots \\ & & h_{n-2} & 2(h_{n-2} + h_{n-1}) & h_{n-1} \\ 0 & & 0 & 0 & 1 \end{bmatrix}, \quad x = \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix}$$

$$b = \begin{bmatrix} 0 \\ \frac{3}{h_1}(a_2 - a_1) - \frac{3}{h_0}(a_1 - a_0) \\ \vdots \\ \frac{3}{h_{n-1}}(a_n - a_{n-1}) - \frac{3}{h_{n-2}}(a_{n-1} - a_{n-2}) \\ 0 \end{bmatrix}$$

Interpolare Spline

SPLINE de clasa C2 - pol. de grad 3; TENSIONAT

- plus conditiile de la spline-uri tensionate:

$$s_0'(x_0) = b_0 + 2c_0(x_0 - x_0) + 3d_0(x_0 - x_0)^2 = b_0 = f'(x_0)$$
$$b_0 = \frac{a_1 - a_0}{h_0} - \frac{h_0}{3}(2c_0 + c_1) = f'(x_0)$$
$$2h_0c_0 + h_0c_1 = 3f'(x_0) - \frac{a_1 - a_0}{h_0}$$
$$s_{n-1}'(x_n) = b_{n-1} + 2c_{n-1}h_{n-1} + 3d_{n-1}h_{n-1}^2 = b_n = f'(x_n)$$
$$b_n = f'(x_n) = b_{n-1} + 2c_{n-1}h_{n-1} + (c_n - c_{n-1})h_{n-1} = \frac{a_n - a_{n-1}}{h_{n-1}} - \frac{h_{n-1}}{3}(2c_{n-1} + c_n) + h_{n-1}c_{n-1} + h_{n-1}c_n,$$
$$h_{n-1}c_{n-1} + 2h_{n-1}c_n = 3f'(x_n) - \frac{3}{h_{n-1}}(a_n - a_{n-1})$$

Interpolare Spline

SPLINE de clasa C2 - pol. de grad 3; TENSIONAT

$$A = \begin{bmatrix} 2h_0 & h_0 & 0 & \cdots & 0 \\ h_0 & 2(h_0+h_1) & h_1 & & \\ 0 & h_1 & 2(h_1+h_2) & h_2 & 0 \\ & & \ddots & \ddots & h_{n-1} \\ 0 & & h_{n-2} & 2(h_{n-2}+h_{n-1}) & h_{n-1} \\ & & 0 & h_{n-1} & 2h_{n-1} \end{bmatrix} \cdot \mathbf{x} = \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix}$$

$$\mathbf{b} = \begin{bmatrix} \frac{3}{h_0}(a_1-a_0) - 3f'(a) \\ \frac{3}{h_1}(a_2-a_1) - \frac{3}{h_0}(a_1-a_0) \\ \vdots \\ \frac{3}{h_{n-1}}(a_n-a_{n-1}) - \frac{3}{h_{n-2}}(a_{n-1}-a_{n-2}) \\ 3f'(b) - \frac{3}{h_{n-1}}(a_n-a_{n-1}) \end{bmatrix}$$

Curbe Hermite

Orice functie Spline ce se formeza din polinoame de grad 3 si respecta conditiile Hermite se numeste curba Hermite.

- $Q(t) = at^3+bt^2+ct+d = \begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix} \cdot \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix}$
- $Q'(t) = 3at^2+2bt+c \quad \begin{bmatrix} 0 & 3t^2 & 2t & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ a \\ b \\ c \end{bmatrix}$

Curbe Hermite

Inpunem restrictiile

$Q(0)=P_1, Q(1)=P_2$ si $Q'(0) = P_1', Q'(1) = P_2'$

Numim geometrie sau vector de control

$$G_H = \begin{bmatrix} P_1 \\ P_2 \\ P_1' \\ P_2' \end{bmatrix}$$

si indetificand vom obtine

Curbe Hermite

$$\begin{aligned} &Q(0)=d \\ &Q(1)=a+b+c+d \\ &Q'(0)=c \\ &Q'(1)=3a+2b \end{aligned} \Rightarrow \begin{aligned} &a=2P_1-2P_2+P_1'+P_2' \\ &b=-3P_1+3P_2-2P_1'-P_2' \\ &c=P_1' \\ &d=P_1 \end{aligned}$$

$$Q(t) = at^3 + bt^2 + ct + d = \begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix} \cdot \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix}$$

adica

Curbe Hermite

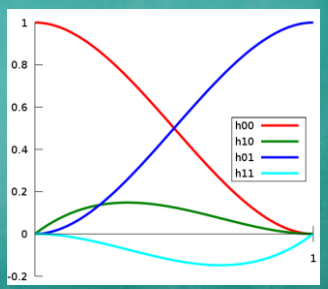
$$Q(t) = \underbrace{\begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix}}_{\text{baza}} \cdot \underbrace{\begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}}_{M_H} \cdot \underbrace{\begin{bmatrix} P_1 \\ P_2 \\ P_1' \\ P_2' \end{bmatrix}}_{\substack{\text{vector} \\ \text{de} \\ \text{control}}} = T^T \cdot M_H \cdot G_H$$

$$Q(t) = \begin{bmatrix} 2t^3 - 3t^2 + 1 & -2t^3 + 3t^2 & t^3 - 2t^2 + t & t^3 - t^2 \end{bmatrix} \cdot \begin{bmatrix} P_1 \\ P_2 \\ P_1' \\ P_2' \end{bmatrix}$$

$$Q(t) = (2t^3-3t^2+1)P_1 + (-2t^3+3t^2)P_2 + (t^3-2t^2+t)P_1' + (t^3-t^2)P_2'$$

Ultima expresie evidentiaza contributia fiecarui punct de control asupra formei curbei, prin intermediul unor ***functii de imbinare*** (blending functions):

Curbe Hermite



Polinoame Bernstein

$$B_i^n(t) = \binom{n}{i} t^i (1-t)^{n-i}, \qquad i = 0 : n$$

Sunt generate din dezvoltaea [t+(1-t)]^n=1 si au urmatoarele proprietati:

- Nenegativitatea $B_i^n > 0, \qquad t \in (0,1]$
- Partiția unității $\sum_{i=0}^{n} B_i^n(t) = 1$

Polinoame Bernstein

- Simetria $B_i^n(t) = B_{n-i}^n(1-t)$
- Maxim unic $t=i/n$ în $[0,1]$
- Formula de recurență $B_i^n(t) = (1-t) \cdot B_i^{n-1}(t) + t \cdot B_{i-1}^{n-1}(t), \qquad i = 0 : n$
- Derivare $\frac{d}{dt} B_i^n(t) = \begin{cases} -nB_i^{n-1} & i = 0 \\ n\left(B_{i-1}^{n-1}(t) - B_i^{n-1}(t)\right) & 0 < i < n \\ nB_{n-1}^{n-1}(t) & i = n \end{cases}$

Curbe Bezier

Se numesc dupa Pierre Étienne Bézier ((1910-1999) inginer francez care folosea aceste curbe pentru modelarea caroseriilor masinilor Renault) dar se pare ca descrierea acestor curbe a fost prima data facuta de Paul de Casteljau ((1930-….) fizician si matematician francez care in 1959, in timp ce lucra la Citroen a produs un algoritm de calcul al unei clase de functii care mai apoi a fost folosita de Pierre Bezier si s-au numit curbe De Casteljau sau curbe Bezier)

Curbe Bezier

Se defineste pentru un set de n+1 puncte P0,P1,…,Pn

$$B(t) = \sum_{i=0}^{n} P_i B_i^n(t), \qquad t \in [0,1]$$

Poligonul ce uneste cele n+1 puncte de control se numeste poligon Bezier si contine in interiorul sau curba Bezier.

Curbe Bezier

Proprietăți ale curbelor Bézier:

- *proprietatea punctelor de capăt* – curba începe în P_0 și se termină în P_n.
- curba Bézier este lineară numai dacă punctele de control sunt coliniare
- curba Bézier este tangentă segmentelor P_0P_1 și $P_{n-1}P_n$.
- o curbă Bézier este conținută complet în înfășurătoarea convexă a punctelor de control.
- O curbă Bézier poate fi separată în alte două curbe Bézier.
- Transformările afine (translația și rotația) aplicată punctelor de control are același efect cu aplicarea transformării afine asupra curbei

Curbe Bezier

Parametric se scrie sub forma

$$Q(t) = P_0(1-t)^3 + 3P_1t(1-t)^2 + 3P_2t^2(1-t) + P_3t^3, \qquad t \in [0,1]$$

$$Q(t) = \begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix} \cdot \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix}$$

Poscript si GIMP folosesc pentru reprezentarea literelor curbe Bezier cubice.

Curbe Bezier

Spline-urile Bezier folosesc in locul derivatelor doua puncte suplimentare de control care exprima derivatele in P0 si P3 astfel P1'=3(P1-P0) si P2'=3(P3-P2).

Trecerea de la Hermite la Bezier se face astfel

$$\begin{bmatrix} P_1 \\ P_2 \\ P_1' \\ P_2' \end{bmatrix}_{G_{Hermite}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ -3 & 3 & 0 & 0 \\ 0 & 0 & -3 & 3 \end{bmatrix} \cdot \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix}_{G_{Bezier}}$$

$$\begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix}_{C} = \underbrace{\begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}}_{M_{Hermite}} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ -3 & 3 & 0 & 0 \\ 0 & 0 & -3 & 3 \end{bmatrix} \cdot \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix}_{G_{Bezier}}$$

Curbe Bezier

Functiile de imbinare pentru spline-ul Bezier sunt polinoamele Berstein Bi^3

$$p(t) = \begin{bmatrix} (1-t)^3 \\ 3t(1-t)^2 \\ 3t^2(1-t) \\ t^3 \end{bmatrix}^T \cdot \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix}$$

Spline Bezier

- Linear $B_1(P_0,P_1,t)=(1-t)P_0+tP_1, \quad t\in[0,1]$
- Patratic $B_2(P_0,P_1,P_2,t)=(1-t)^2P_0+2t(1-t)P_1+t^2P_2, \quad t\in[0,1]$

Curbe Bezier

Functiile de imbinare pentru spline-ul Bezier



Bibliografie Interpolare Poly/Spline

- V.Iorga, B.Jora, Metode Numerice, Ed. Albastră, 2004;
- Richard L. Burden, J. Douglas Faires, Numerical Analysis, 9th edition, Brooks/Cole, Cengage Learning, 2011;
- Chapra, Steven C., Applied numerical methods with MATLAB for engineers and scientists, 3rd ed., McGraw-Hill, 2012;

Metoda Neville. Metode de interpolare cu funcţii spline cubice. Curbe Bézier. Algoritmul De Casteljau

Noțiuni teoretice

Metoda Neville

Se consideră o funcţie $f : [a, b] \to R$ cunoscută într-o mulţime finită de puncte $x_0, x_1, ..., x_n$ (numite suportul interpolării) prin valorile:

$$f(x_0), f(x_1), ..., f(x_n).$$

Metoda Neville este o metodă de interpolare care aproximează comportamentul funcţiei f în afara acestor $n + 1$ puncte. În cele ce urmează, folosim notaţia $P_{ij}(x)$ pentru a reprezenta polinomul de interpolare de grad $j - i$ care trece prin punctele $(x_l, f(x_l)), l = i, i + 1, ..., j$.

Polinomul de interpolare P_{ij} este dat de relaţia de recurență:

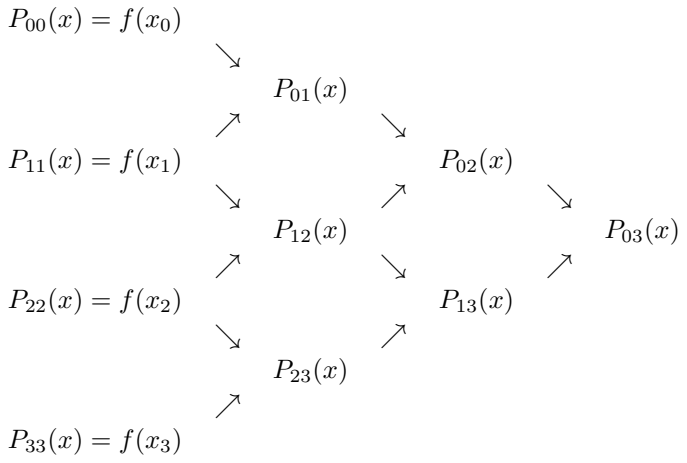
$$P_{ij}(x) = \frac{x - x_j}{x_i - x_j} P_{i,j-1}(x) + \frac{x_i - x}{x_i - x_j} P_{i+1,j}(x), \quad 0 \leq i < j \leq n$$

unde

$$P_{ii}(x) = f(x_i), \quad i = 0 : n.$$

În prima iteraţie, metoda Neville construieşte polinoame de interpolare de grad 0, reprezentatate prin $P_{ii}(x), i = 0 : n$. În următoarea iteraţie, oricare două polinoame de interpolare de grad 0, alăturate, P_{ii} şi $P_{i+1,i+1}, i = 0 : n - 1$, formează un polinom

de interpolare de grad 1 notat cu $P_{i,i+1}(x)$. Procesul continuă până când se obţine polinomul de interpolare $P_{0,n}$ care trece prin toate cele $n+1$ puncte $(x_l, f(x_l)), l = 0, ..., n$. De exemplu, procesul iterativ de obţinere al polinomului de interpolare $P_{03}(x)$, pentru $n = 3$, este arătat în schema următoare:



Metode de interpolare cu funcţii spline cubice

În continuare, considerăm că funcţia $f : [a, b] \to R$ este cunoscută atât într-o mulţime finită de puncte $x_0, x_1, ..., x_n$ prin valorile:

$$f(x_0), f(x_1), ..., f(x_n)$$

cât şi prin derivatele de ordin I ale funcţie în aceste puncte:

$$f^{'}(x_0), f'(x_1), ..., f'(x_n)$$

În cazul interpolării cu funcţii spline cubice, comportamentul funcţiei f se va studia local pe subintervalele:

$$[x_0, x_1], [x_1, x_2], ..., [x_i, x_{i+1}]..., [x_{n-1}, x_n]$$

Interpolare cu funcţii spline în clasa C^1:

Un polinom de gradul 3 se va considera pentru fiecare subinterval $[x_i, x_{i+1}]$ de forma:

$$s_i : [x_i, x_{i+1}] \to R, s_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3, i = 0 : n - 1$$

Prin schimbarea de variabilă $t = \frac{x - x_i}{x_{i+1} - x_i}$ se obţine următoarea formă parametrică pentru o funcţie spline cubică:

$$s_i(t) = a_i + b_i h_i t + c_i h_i t^2 + d_i h_i t^3, i = 0 : n - 1$$

unde am notat $h_i = x_{i+1} - x_i$.

În general, baza de interpolare Bernstein se foloseşte pentru a eficientiza procesul de calculare al coeficienţilor funcţiilor spline cubice:

$$(1 - t)^3, 3t(1 - t)^2, 3t^2(1 - t), t^3$$

Folosind baza de interpolare Bernstein, funcţia spline cubică devine:

$$s_i(t) = a_i^{'}(1 - t)^3 + 3b_i^{'}t(1 - t)^2 + 3c_i^{'}t^2(1 - t) + d_i^{'}t^3, i = 0 : n - 1$$

Pentru a calcula cei $4n$ coeficienţi, se impun următorele condiţii în cazul interpolării cu funcţii spline în clasă C^1:

- $2n + 2$ condiţii de interpolare de tip Hermite:

$$s_i(x_i) = f(x_i), \quad i = 0 : n - 1$$

$$s_{n-1}(x_n) = f(x_n)$$

$$s_i^{'}(x_i) = f^{'}(x_i), \quad i = 0 : n - 1$$

$$s_{n-1}^{'}(x_n) = f^{'}(x_n)$$

- $2n - 2$ condiţii de racordare ce asigură continuitatea atat a funcţiilor spline cubice cat şi a derivatei de ordinul I pentru funcţiile spline cubice:

$$s_i(x_{i+1}) = s_{i+1}(x_{i+1}), \quad i = 0 : n - 2$$

$$s_i^{'}(x_{i+1}) = s_{i+1}^{'}(x_{i+1}), \quad i = 0 : n - 2$$

Considerand pentru funcţiile spline cubice forma parametrică în baza Bernstein, se rezolva sistemul cu cele $4n$ ecuaţii si se obţin următoarele relaţii pentru coeficienţi:

$$a_i^{'} = f(x_i), \quad i = 0 : n-1$$

$$d_i^{'} = f(x_{i+1}), \quad i = 0 : n-1$$

$$b_i^{'} = f(x_i) + \frac{h_i}{3} f^{'}(x_i), \quad i = 0 : n-1$$

$$c_i^{'} = f(x_{i+1}) - \frac{h_i}{3} f^{'}(x_{i+1}), \quad i = 0 : n-1$$

În cele din urmă, forma în variabila x pentru fiecare funcţie spline cubică în clasă C^1 se obţine prin schimbarea de variabilă $t = \frac{x-x_i}{h_i}$.

Interpolare cu funcţii spline în clasă C^2:

Un polinom de gradul 3 se va considera pentru fiecare subinterval $[x_i, x_{i+1}]$ de forma:

$$s_i : [x_i, x_{i+1}] \to R, s_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3, i = 0 : n-1$$

În cazul interpolării cu funcţii spline în clasă C^2, cei $4n$ coeficienţi se determină prin impunerea a:

- $n + 1$ condiţii de interpolare de tip Lagrange:

$$s_i(x_i) = f(x_i), \quad i = 0 : n-1$$

$$s_{n-1}(x_n) = f(x_n)$$

- $3n - 3$ condiţii de continuitate, derivabilitate şi curbură pentru funcţiile spline cubice vecine:

$$s_i(x_{i+1}) = s_{i+1}(x_{i+1}), \quad i = 0 : n-2$$

$$s_i^{'}(x_{i+1}) = s_{i+1}^{'}(x_{i+1}), \quad i = 0 : n-2$$

$$s_i^{''}(x_{i+1}) = s_{i+1}^{''}(x_{i+1}), \quad i = 0 : n-2$$

- următoarelor 2 condiţii pentru funcţiile spline naturale:

$$s_0^{''}(x_0) = 0$$

$$s_{n-1}^{''}(x_n) = 0$$

respectiv următoarelor 2 condiţi pentru funcţiile spline tensionate:

$$s_0{}^{'}(x_0) = f^{'}(x_0)$$

$$s_{n-1}^{'}(x_n) = f^{'}(x_n)$$

Coeficienţii funcţiilor spline cubice în clasă C^2 sunt daţi de relaţiile:

$$a_i = f(x_i), \quad i = 0 : n$$

$$d_i = \frac{c_{i+1} - c_i}{3h_i}, \quad i = 0 : n-1$$

$$b_i = \frac{a_{i+1} - a_i}{h_i} - \frac{h_i}{3}(2c_i + c_{i+1}), \quad i = 0 : n-1$$

iar coeficienţii $c_i, i = 0 : n-1$ se obţin prin rezolvarea unui sistem tridiagonal de forma:

- pentru funcţii spline naturale

$$\begin{bmatrix} 1 & 0 & 0 & & & 0 \\ h_0 & 2(h_0+h_1) & h_1 & & & 0 \\ & \ddots & \ddots & \ddots & & \\ & & h_{n-2} & 2(h_{n-2}+h_{n-1}) & h_{n-1} & \\ 0 & & & 0 & 1 & \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{n-1} \\ c_n \end{bmatrix} =$$

$$\begin{bmatrix} 0 \\ \frac{3(a_2-a_1)}{h_1} - \frac{3(a_1-a_0)}{h_0} \\ \vdots \\ \frac{3(a_n-a_{n-1})}{h_{n-1}} - \frac{3(a_{n-1}-a_{n-2})}{h_{n-2}} \\ 0 \end{bmatrix}$$

- pentru funcţii spline tensionate

$$\begin{bmatrix} 2h_0 & h_0 & 0 & & & 0 \\ h_0 & 2(h_0+h_1) & h_1 & & & 0 \\ & \ddots & \ddots & \ddots & & \\ & & h_{n-2} & 2(h_{n-2}+h_{n-1}) & h_{n-1} & \\ 0 & & & h_{n-1} & 2h_{n-1} & \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{n-1} \\ c_n \end{bmatrix} =$$

$$\begin{bmatrix} \frac{3(a_1-a_0)}{h_0} - 3f^{'}(x_0) \\ \frac{3(a_2-a_1)}{h_1} - \frac{3(a_1-a_0)}{h_0} \\ \vdots \\ \frac{3(a_n-a_{n-1})}{h_{n-1}} - \frac{3(a_{n-1}-a_{n-2})}{h_{n-2}} \\ 3f^{'}(x_n) - \frac{3(a_n-a_{n-1})}{h_{n-1}} \end{bmatrix}$$

Funcţia spline s_n a fost introdusă pentru a ajuta la calcularea funcţiilor spline $s_i, i = 0 : n-1$.

Curbe Bézier

Fiind dată o mulţime de $n+1$ puncte $P_0, P_1, ..., P_n$ (P_0 si P_n se numesc puncte de interpolare iar P_1 ... P_{n-1} se numesc puncte de control) în plan sau în spaţiu, curba Bézier de grad n determinată de aceste puncte are forma parametrică:

$$B(t) = \sum_{i=0}^{n} P_i B_{i,n}(t), \quad t \in [0,1]$$

unde $B_{i,n}(t)$ se numesc polinoame Bernstein de grad n definite prin relaţia:

$$B_{i,n}(t) = \binom{n}{i}(1-t)^{(n-i)}t^i, \quad i = 0 : n$$

Gradul polinomului care aproximează curba Bézier depinde de numărul punctelor de control. Fiecare punct de control determina forma curbei Bézier în mod particular. Dacă modificăm poziţia unui punct de control şi forma curbei Bézier va fi afectată.

Spre deosebire de funcţiile spline, curba Bézier nu trece prin toate punctele P_0,P_1,...,P_n ci doar prin punctele P_0 si P_n. Orice curbă Bézier începe în punctul P_0 şi se termină în punctul P_n, adică:

$$B(0) = P_0, B(1) = P_n$$

Mai mult, curba Bézier este tangentă segmentelor P_0P_1 si $P_{n-1}P_n$. Orice curbă Bézier este conţinută complet de înfăşurătoarea convexă pe care punctele $P_0, P_1, ..., P_n$ o definesc.

În mod uzual, se folosesc curbe Bézier cubice determinate de 4 puncte P_0, P_1, P_2, P_3, având forma parametrică:

$$B(t) = P_0B_{0,3}(t) + P_1B_{1,3}(t) + P_2B_{2,3}(t) + P_3B_{3,3}(t) = P_0(1-t)^3 + 3P_1t(1-t)^2 + 3P_2t^2(1-t) + P_3t^3 = \begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix} \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix}, \quad t \in [0,1]$$

Algoritmul De Casteljau

Într-o abordare directă, calcularea unui punct aflat pe o curbă Bézier se obţine folosind ecuaţia parametrică $B(t) = \sum_{i=0}^{n} P_iB_i^n(t), t \in [0,1]$. Acestă metodă este ineficientă deoarece numerele mici ridicate la puteri mari genereaza erori mari.

Algoritmului De Casteljau este o modalitate eficientă de calculare a unui punct aflat pe o curbă Bézier. Acest algoritm este puţin mai lent insa este numeric stabil şi cu ajutorul său putem obţine detalii despre curba Bézier:

- vectorul tangent într-un punct de pe curba Bézier folosind calculul derivatei;
- divizarea curbei Bézier. Uneori este necesar să separăm o curbă Bézier în alte curbe Bézier.

Algoritmul De Casteljau foloseşte relaţia de recurenţă următoare:

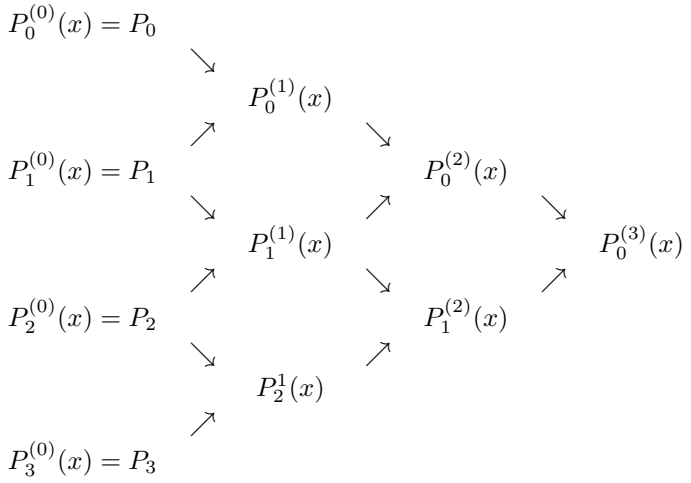
$$P_i^{(0)} = P_i, \quad i = 0 : n$$

$$P_i^{(j)} = P_i^{(j-1)}(1-t_0) + P_{i+1}^{(j-1)}t_0, \quad j = 1 : n, i = 0 : n-j,$$

Fie $P_i^{(0)}$ şi $P_{i+1}^{(0)}$ două puncte succesive şi $P_i^{(1)}$ un punct care împarte segmentul $P_i^{(0)}P_{i+1}^{(0)}$ în raportul $t/(1-t)$. $P_i^{(1)}$ este o combinatie liniara intre punctele $P_i^{(0)}$ şi $P_{i+1}^{(0)}$, adica :

$$P_i^{(1)} = (1-t)P_i^{(0)} + tP_{i+1}^{(0)}$$

Se formează în acest fel poligonul $P_0^{(1)}, P_1^{(1)}, ..., P_{n-1}^{(1)}$. Se aplică relaţia de recurenţă noului poligon obţinându-se poligonul $P_0^{(2)}, P_1^{(2)}, ..., P_{n-2}^{(2)}$. Repetând procesul de n ori, se obţine un singur punct $P_0^{(n)}$. Acest punct obţinut se află pe curba Bézier. Pentru $n = 3$, procesul iterativ de obţinere al punctului $P_0^{(3)}(x)$, este arătat în schema următoare:



Probleme rezolvate

Problema 1

Se consideră funcţia f cunoscută prin valorile:

x	**0.4**	**0.9**	**1.5**	**2.5**
f(x)	**0.9**	**0.5**	**0.2**	**0.1**

Determinaţi valoarea funcţiei în punctul $x = 4$.

Soluţie:

$$P_{00}(4) = f(x_0) = 0.9$$
$$P_{11}(4) = f(x_1) = 0.5$$
$$P_{22}(4) = f(x_2) = 0.2$$
$$P_{33}(4) = f(x_3) = 0.1$$
$$P_{01}(4) = \frac{(4 - x_1)P_{00}(4) + (x_0 - 4)P_{11}(4)}{x_0 - x_1} = \frac{(4 - 0.9)0.9 + (0.4 - 4)0.5}{0.4 - 0.9} = -1.98$$
$$P_{12}(4) = \frac{(4 - x_2)P_{11}(4) + (x_1 - 4)P_{22}(4)}{x_1 - x_2} = \frac{(4 - 1.5)0.5 + (0.9 - 4)0.2}{0.9 - 1.5} = -1.05$$

$$P_{23}(4) = \frac{(4-x_3)P_{22}(4) + (x_2-4)P_{33}(4)}{x_2 - x_3} = \frac{(4-2.5)0.2 + (1.5-4)0.1}{1.5-2.5} = -0.05$$

$$P_{02}(4) = \frac{(4-x_2)P_{01}(4) + (x_0-4)P_{12}(4)}{x_0 - x_2} =$$

$$\frac{(4-1.5)(-1.98) + (0.4-4)(-1.05)}{0.4-1.5} = 1.06$$

$$P_{13}(4) = \frac{(4-x_3)P_{12}(4) + (x_1-4)P_{23}(4)}{x_1 - x_3} =$$

$$\frac{(4-2.5)(-1.05) + (0.9-4)(-0.05)}{0.9-1.5} = 0.89$$

$$P_{03}(4) = \frac{(4-x_3)P_{02}(4) + (x_0-4)P_{13}(4)}{x_0 - x_3} = \frac{(4-2.5)1.06 + (0.4-4)0.88}{0.4-2.5} = 0.76$$

Prin urmare, am obţinut $f(4) \simeq P_{03}(4) = 0.76$

Problema 2

Scrieţi un program OCTAVE care calculează valoarea unei funcţii într-un punct folosind metoda Neville. Programul primeşte ca parametri de intrare: x - suportul interpolării, y - vectorul ordonatelor pentru valorile din suportul interpolării, xi - abscisa în care se calculează valoarea funcţiei. Rezultatul programului este valoarea funcţiei în xi.

Soluţie:

```
function yi = Neville(x, y, xi)
  n = length(x);

  for k = 1 : n-1
    for i = 1 : n-k
      raport = (xi-x(k+i))/(x(i)-x(k+i));
      y(i) = raport*y(i)+(1-raport)*y(i+1);
    endfor
  endfor

  yi = y(1);
endfunction
```

Listing 1: Metoda Neville de interpolare.

Probleme propuse

Problema 1

Scrieţi un program OCTAVE care calculează valoarea unei funcţii într-un punct, ca rezultat al interpolării obţinute folosind funcţii spline în clasă C^1. Programul primeşte ca parametrii de intrare: x - suportul interpolării, y - vectorul ordonatelor pentru valorile din suportul interpolării, dy - vectorul derivatelor pentru valorile din suportul interpolării, xi - abcisa în care se calculează valoarea funcţiei. Rezultatul programului este valoarea funcţiei în xi.

```
function yi = SplineC1(x, y, dy, xi)
```

Listing 2: Interpolare cu funcţii spline în clasă C^1.

Pentru testarea programului, puteţi folosi următoarea secvenţă:

```
x = 0:pi/3:7*pi;
y = sin(x)+cos(3*x);
dy = cos(x)-3*sin(3*x);
xi = 0:pi/15:7*pi;

for i = 1 : length(xi)
  yi(i) = feval('SplineC1', x, y, dy, xi(i));
endfor

plot(x, y, 'g-o', xi, yi, 'r-*');
legend ('data', 'spline C1');
axis([0 23 -2 2.7]);
```

Listing 3: Fişier de testare.

iar rezultatul este:

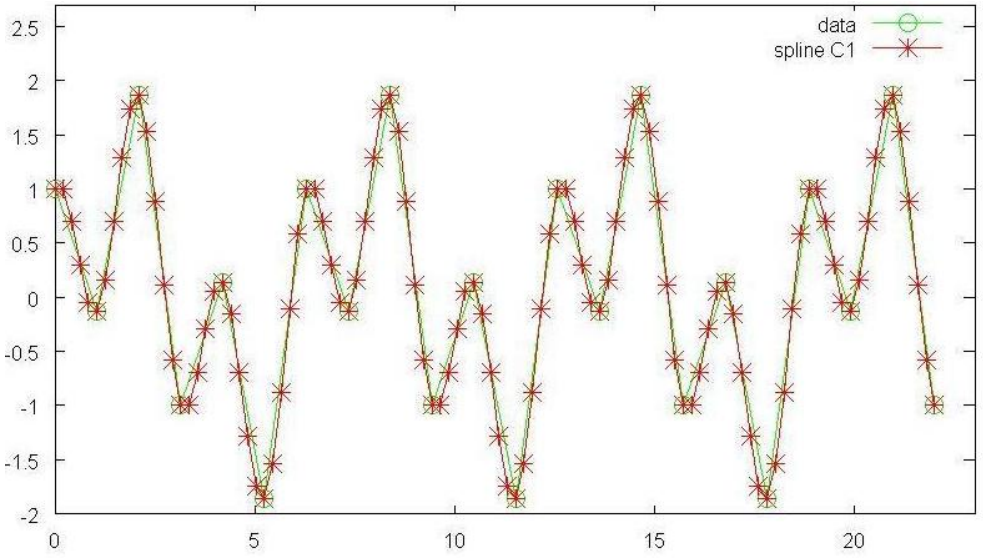


Figure 1: Graficul rezultat pentru fişierul de testare.

Problema 2

Scrieţi un program OCTAVE care calculează valoarea unei funcţii într-un punct, ca rezultat al interpolării obţinute folosind funcţii spline naturale în clasă C^2. Programul primeşte ca parametri de intrare: x - suportul interpolării, y - vectorul ordonatelor pentru valorile din suportul interpolării, xi - abscisa în care se calculează valoarea funcţiei. Rezultatul programului este valoarea funcţiei în xi.

```
function yi = SplineC2natural(x, y, xi)
```

Listing 4: Interpolare cu funcţii spline naturale în clasă C^2.

Pentru testarea programului puteţi folosi următoarea secvenţă:

```
x = 0:pi/3:7*pi;
y = sin(x)+cos(3*x);
xi = 0:pi/15:7*pi;

```

```
5   for i = 1 : length(xi)
6     yi(i) = feval('SplineC2natural', x, y, xi(i));
7   endfor
8
9   plot(x, y, 'g-o', xi, yi, 'r-*');
10  legend ('data', 'spline C2 natural');
11  axis([0 23 -2 2.7]);
```

Listing 5: Fişier de testare.

iar rezultatul este:

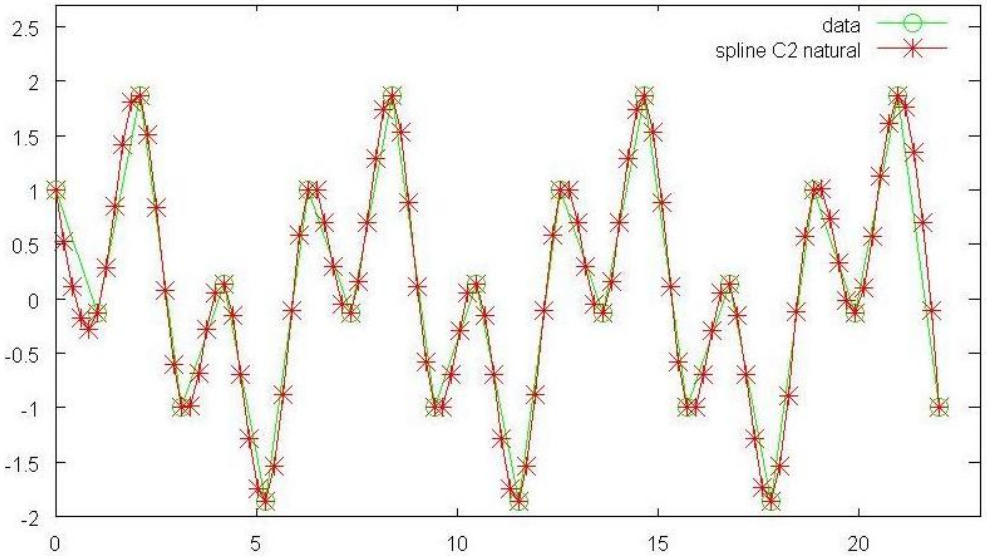


Figure 2: Graficul rezultat pentru fişierul de testare.

Problema 3

Scrieţi un program OCTAVE care calculează valoarea unei funcţii într-un punct ca rezultat al interpolării obţinute folosind funcţii spline tensionate în clasă C^2. Programul primeşte ca parametri de intrare:

x - suportul interpolării,

y - vectorul ordonatelor pentru valorile din suportul interpolării,

$dy1$ - derivata funcţiei în primul punct din suportul interpolării,

dyn - derivata funcţiei în ultimul punct din suportul interpolării,

xi - abscisa în care se calculează valoarea funcţiei.

Rezultatul programului este valoarea funcţiei în xi.

Problema 4

Scrieţi o funcţie OCTAVE care să calculeze un punct aflat pe o curbă Bézier folosind algoritmul De Casteljau. Funcţia primeşte ca parametri de intrare: x - vectorul absciselor punctelor de control, y - vectorul ordonatelor punctelor de control, t - parametru. Rezultatul funcţiei este un punct P aflat pe curba Bézier.

```
function P = Casteljau(x, y, t)

  n = length(x);

  plot(x, y, 'k-o');
  legend ('puncte de control');
  hold on;
  axis([0.5, 4.5, 0.2, 2.1]);

  TODO
endfunction
```

Listing 6: Algoritmului De Casteljau.

Pentru testarea funcţiei anterioare puteţi folosi următoarea secvenţă, care trasează o curbă Bézier folosind algoritmul De Casteljau:

```
  x = [1 0.7 2.7 3.7];
  y = [0.5 2 2 0.5];

  i = 1;
  B_x = zeros();
  B_y = zeros();
  hold off;
```

```
8
9   for t = 0 : 0.05 : 1
10    P = Casteljau(x, y, t);
11    B_x(i) = P(1);
12    B_y(i) = P(2);
13    plot(B_x, B_y, 'b-s');
14    legend ('puncte calculate', 'puncte aflate pe curba Bezier'
      );
15    i++;
16
17    pause(0.5);
18    hold off;
19  endfor
```

Listing 7: Fişier de testare.

iar rezultatul este:

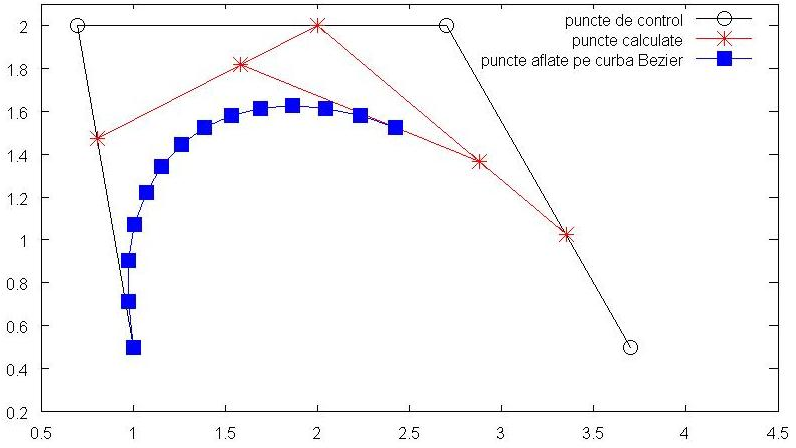


Figure 3: Graficul intermediar rezultat pentru fişierul de testare.

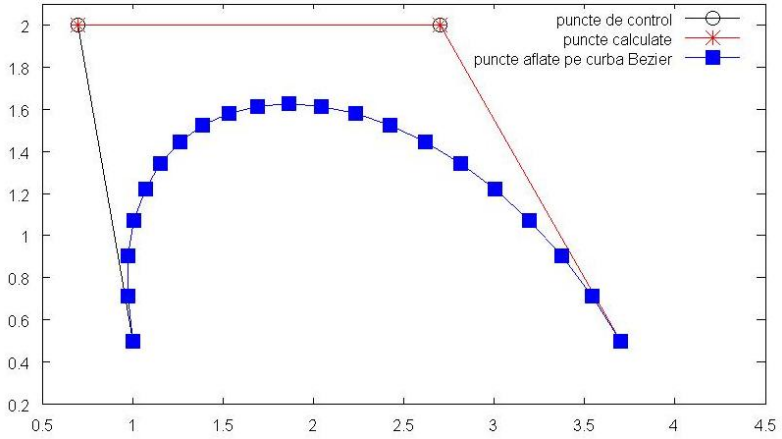


Figure 4: Graficul final rezultat pentru fişierul de testare.

Problema 5

Se consideră funcţia f cunoscută prin valorile:

x	-3	1	2
f(x)	0	3	4
f'(x)	1	2	3

Determinaţi funcţiile spline în clasa C^1 pentru interpolarea lui f.

Problema 6

Se consideră funcţia f cunoscută prin valorile:

x	-2	2	3	5
f(x)	-3	1	3	7

Determinaţi valoarea funcţiei în punctul $x = 1$.

Metode Numerice @ CS-UPB

Echipa MN CS - 2015:

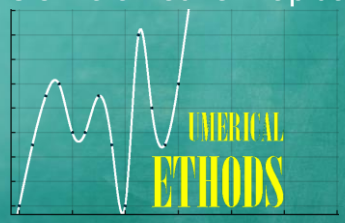
Profesori:

- Pantelimon George Popescu - Seria CA;
- Florin Pop - Seria CB, CC;

Asistenti:

PG Popescu, Clementin Cercel, Sorin Ciolofan, Bogdan Tiganoaia, Alexandru Tifrea, David Iancu, Bogdan Cristian Marchis, Radu Poenaru, Madalina Grosu, Andrei Vlad Postoaca, Radu Constantinescu, Roxana Istrate, Madalina Hristache, Adelina Vidovici;

Aproximare in sensul CMMP
Aproximare cu functii rationale
Transformata Fourier Rapida



Aproximare in sensul CMMP

Pentru o functie lineara, dar din cauza erorilor de masurare linearitatea s-a pierdut. Si deci ne aflam in situatia in care daca am interpola aceste date am fi nevoiti sa suferim consecintele perturbatiilor. Daca am aproxima cumva functia pornind de la aceste date si deci nu am respecta ca aproximatia obtinuta sa treaca prin punctele date, atunci in acest caz am fi avantajati.

Aproximare in sensul CMMP

Ne-ar interesa o linie, a1x+a0, care sa treaca printre punctele date si care sa aproximeze functia, considerata prin puncte cat mai bine. Adica am dori ca eroarea exprimata astfel

$$E \equiv E_2(a_0, a_1) = \sum_{i=1}^{m} \left[y_i - (a_1 x_i + a_0) \right]^2$$

sa fie cat mai mica. Si de aici si titulatura de “in sensul celor mai mici patrate”.

Aproximare in sensul CMMP

Ca acesta eroarea sa fie cat mai mica ar trebui gasiti coeficientii a1 si a0 pentru care E sa fie cat mai mica. Si deci conditia care se pune este ca derivatele partiale dupa a1 si dupa a0 ale lui E sa fie 0.

$$0 = \frac{\partial}{\partial a_0} \sum_{i=1}^{m} \left[y_i - (a_1 x_i - a_0) \right]^2 = 2 \sum_{i=1}^{m} (y_i - a_1 x_i - a_0)(-1)$$

$$0 = \frac{\partial}{\partial a_1} \sum_{i=1}^{m} \left[y_i - (a_1 x_i + a_0) \right]^2 = 2 \sum_{i=1}^{m} (y_i - a_1 x_i - a_0)(-x_i).$$

Aproximare in sensul CMMP

Adica

$$a_0 \cdot m + a_1 \sum_{i=1}^{m} x_i = \sum_{i=1}^{m} y_i \qquad a_0 \sum_{i=1}^{m} x_i + a_1 \sum_{i=1}^{m} x_i^2 = \sum_{i=1}^{m} x_i y_i.$$

de unde rezulta a1 si a0 (SEL 2x2)

$$a_0 = \frac{\sum_{i=1}^{m} x_i^2 \sum_{i=1}^{m} y_i - \sum_{i=1}^{m} x_i y_i \sum_{i=1}^{m} x_i}{m \left(\sum_{i=1}^{m} x_i^2 \right) - \left(\sum_{i=1}^{m} x_i \right)^2}$$

$$a_1 = \frac{m \sum_{i=1}^{m} x_i y_i - \sum_{i=1}^{m} x_i \sum_{i=1}^{m} y_i}{m \left(\sum_{i=1}^{m} x_i^2 \right) - \left(\sum_{i=1}^{m} x_i \right)^2}.$$

Aproximare in sensul CMMP

Mai departe ne-ar interesa o aproximare mai exacta ca cea lineara in sensul cmmp. Si atunci in loc pe un polinom de grad 1(o linie) consideram un polinom de grad mai mare. $P_n(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0$

Adica ar trebui sa gasim coeficientii an,…,a1,a0 pentru care eroarea E sa fie cat mai mica, adica

$$E = \sum_{i=1}^{m}(y_i - P_n(x_i))^2 = \sum_{i=1}^{m} y_i^2 - 2\sum_{i=1}^{m} P_n(x_i)y_i + \sum_{i=1}^{m}(P_n(x_i))^2$$

Aproximare in sensul CMMP

SI daca dezvoltam Pn obtinem E ca fiind

$$= \sum_{i=1}^{m} y_i^2 - 2\sum_{i=1}^{m}\left(\sum_{j=0}^{n} a_j x_i^j\right) y_i + \sum_{i=1}^{m}\left(\sum_{j=0}^{n} a_j x_i^j\right)^2$$
$$= \sum_{i=1}^{m} y_i^2 - 2\sum_{j=0}^{n} a_j \left(\sum_{i=1}^{m} y_i x_i^j\right) + \sum_{j=0}^{n}\sum_{k=0}^{n} a_j a_k \left(\sum_{i=1}^{m} x_i^{j+k}\right).$$

Pentru ca E sa fie minim, ca si in cazul linear derivatele partiale ale lui E dupa aj trebuie sa fie 0.

Aproximare in sensul CMMP

Adica pentru fiecare j=0,…n

$$0 = \frac{\partial E}{\partial a_j} = -2\sum_{i=1}^{m} y_i x_i^j + 2\sum_{k=0}^{n} a_k \sum_{i=1}^{m} x_i^{j+k}.$$

De unde vom avea n+1 ecuatii si n+1 necunoscute (coeficientii aj).

Aproximare in sensul CMMP

Rescriind ecuatiile sub forma de sistem vom avea

$$\begin{aligned}
a_0\sum_{i=1}^{m} x_i^0 + a_1\sum_{i=1}^{m} x_i^1 + a_2\sum_{i=1}^{m} x_i^2 + \cdots + a_n\sum_{i=1}^{m} x_i^n &= \sum_{i=1}^{m} y_i x_i^0,\\
a_0\sum_{i=1}^{m} x_i^1 + a_1\sum_{i=1}^{m} x_i^2 + a_2\sum_{i=1}^{m} x_i^3 + \cdots + a_n\sum_{i=1}^{m} x_i^{n+1} &= \sum_{i=1}^{m} y_i x_i^1,\\
&\vdots\\
a_0\sum_{i=1}^{m} x_i^n + a_1\sum_{i=1}^{m} x_i^{n+1} + a_2\sum_{i=1}^{m} x_i^{n+2} + \cdots + a_n\sum_{i=1}^{m} x_i^{2n} &= \sum_{i=1}^{m} y_i x_i^n.
\end{aligned}$$

Aproximare in sensul CMMP

Un spatiu PH este un dublet (F,u), unde F este un spatiu vectorial cu scalari din corpul R(sau C), iar u este un produs scalar, u:FxF->R si are urmatoarele proprietati:

linearitate	$\langle f_1+f_2,\ f_3\rangle = \langle f_1, f_3\rangle + \langle f_2, f_3\rangle$, $\langle c\cdot f_1,\ f_2\rangle = c\cdot\langle f_1,\ f_2\rangle$,
comutativitate	$\langle f_1,\ f_2\rangle = \langle f_2,\ f_1\rangle$
definire pozitivă	$\langle f,\ f\rangle \geq 0$
nesingularitate	$\langle f,\ f\rangle = 0 \Leftrightarrow f = 0$

Aproximare in sensul CMMP

Fie G din F un subspatiu de dimensiune finita(are un numar finit de elemente linear independente);
Norma unui element f din F se defineste ca fiind

$$\|f\| = \sqrt{\langle f, f\rangle}$$

Cel mai bun aproximant in sensul celor mai mici patrate a unui element f din F in subspatiul G este un element g* din G cu proprietatea

$$\|f - g^*\| = \min_{g\in G}\|f - g\|$$

Aproximare in sensul CMMP

T1: g* din G este cel mai bun aproximant al lui f din F daca $<f-g^*, g>=0, \ \forall g \in G$

T2: Cea mai buna aproximare in sensul cmmp, g*din G, a lui f din F este unica.

Daca u1,u2,...,un este o baza din G atunci orice element g din G se poate scrie sub forma g=sum ckuk, adica si g* se poate scrie sub forma

Aproximare in sensul CMMP

$$g^* = \sum_{k=1}^{n} c_k^* u_k$$

Teorema 1 se transforma in

$$\langle f - g^*, g \rangle = \left\langle f - g^*, \sum_{j=1}^{n} c_j u_j \right\rangle = \sum_{j=1}^{n} c_j \langle f - g^*, u_j \rangle = 0$$

$$\langle f - g^*, u_j \rangle = 0, \quad j = 0 : n$$

$$\langle g^*, u_j \rangle = \langle f, u_j \rangle$$

$$\sum_{k=1}^{n} \langle c_k^* u_k, u_j \rangle = \langle f, u_j \rangle$$

$$\sum_{k=1}^{n} c_k^* \langle u_k, u_j \rangle = \langle f, u_j \rangle, \quad j = 1 : n$$

Aproximare in sensul CMMP

De unde rezulta un sistem de ecuatii

$$\begin{cases} \langle u_1, u_1 \rangle \cdot c_1 + \langle u_2, u_1 \rangle \cdot c_2 + \ldots + \langle u_n, u_1 \rangle \cdot c_n = \langle f, u_1 \rangle \\ \langle u_1, u_2 \rangle \cdot c_1 + \langle u_2, u_2 \rangle \cdot c_2 + \ldots + \langle u_n, u_2 \rangle \cdot c_n = \langle f, u_2 \rangle \\ \ldots \quad \ldots \quad \ldots \quad \ldots \\ \langle u_1, u_n \rangle \cdot c_1 + \langle u_2, u_n \rangle \cdot c_2 + \ldots + \langle u_n, u_n \rangle \cdot c_n = \langle f, u_n \rangle \end{cases}$$

Sistem simetric si rau conditionat, al carui determinant poarta munele de determinat GRAM. Deoarece rezolvarea sistemului este anevoioasa se aleg cazuri particulare, adica diferite baze.

Aproximare in sensul CMMP

De exemplu pentru o baza ortonormata sistemul devine diagonal, iar coeficientii ck*=<f,uk> se numesc coeficienti Fourier(Jean Baptiste Joseph Fourier (1768 - 1830) matematician si fizician francez)

Calitatea aproximarii se evalueaza prin distanta

$$\|f - g^*\|^2 = \langle f - g^*, f - g^* \rangle = \langle f - g^*, f \rangle - \underbrace{\langle f - g^*, g^* \rangle}_{0} = \langle f, f \rangle - \langle g^*, f \rangle$$

$$\|f - g^*\|^2 = \|f\|^2 - \sum_{k=1}^{n} c_k^* \langle f, u_k \rangle$$

Iar daca baza este ortonormata avem forma simplificata

$$\|f - g^*\|^2 = \|f\|^2 - \sum_{k=1}^{n} c_k^{*2}$$

Aproximare Continua CMMP

Se aleg produsul scalar si norma, astfel:

$$\langle f, g \rangle = \int_a^b w(x) \cdot f(x) \cdot g(x) \, dx$$

$$\|f\| = \sqrt{\int_a^b w(x) \cdot f^2(x) \, dx}$$

Aproximarea continua in sensul cmmp g* a lui f pe C([a,b]) este

$$\underbrace{\int_a^b w(x) \cdot [f(x) - g^*(x)]^2 \, dx}_{E^*} = \min_{g \in G} \underbrace{\int_a^b w(x) \cdot [f(x) - g(x)]^2 \, dx}_{E}$$

Aproximare Continua CMMP

Daca avem o baza u={u0,u1,...,un} pentru G, atunci

$g = \sum_{k=0}^{n} c_k u_k$ minimul se obtine pentru

$$\frac{\partial E}{\partial c_i} = 0 \quad \text{cu } i = 0 : n$$

Concret trebuie statisfacuta conditia de ortogonalitate

$$\int_a^b w(x) \cdot [f(x) - g^*(x)] \cdot g(x) \, dx = 0$$

$$\sum_{k=1}^{n} c_k^* \int_a^b w(x) \cdot u_k(x) \cdot u_j(x) \, dx = \int_a^b w(x) \cdot f(x) \cdot u_j(x) \, dx, \quad j = 1 : n$$

Aproximare Continua CMMP

Alegem o baza (|G|=n+1) polinomiala, u0(x)=1,u1(x)=x,…,un(x)=x^n, ceea ce implica ca sistemul normal se transforma in

$$\sum_{k=0}^{n} c_k^* \int_a^b w(x)\cdot x^{k+j}\,dx = \int_a^b w(x)\cdot f(x)\cdot x^j\,dx, \qquad j = 0:n$$

Dar aceasta baza nu este normata si ca sa o normam folosim algoritmul Gram-Schmitd si obtinem v0,v1,…,vn, baza ortonormata.

Aproximare Continua CMMP

$$w_0 = u_0, \qquad v_0 = \frac{w_0}{\|w_0\|_2}$$

$$w_1 = u_1 - \langle u_1, v_0\rangle v_0, \qquad v_1 = \frac{w_1}{\|w_1\|_2}$$

$$w_m = u_m - \sum_{p=0}^{m-1} \langle u_m, v_p\rangle v_p, \qquad v_m = \frac{w_m}{\|w_m\|_2}$$

Aproximare Continua CMMP

Aproximare continua trigonometrica in sensul celor mai mici patrate se refera la alegerea bazei ortogonale ce are 2n+1 componente

$$\frac{1}{\sqrt{2}}\ \sin(x)\ \cos(x)\ \cdots\ \sin(nx)\ \cos(nx)$$

si functia pondere w(x)=1.

Deci relatiile de ortogonalitate vor fi

$$\frac{1}{\pi}\int_0^{2\pi} u_q(x)u_r(x)\,dx = \delta_{qr} = \begin{cases} 0, & q \neq r \\ 1, & q = r \end{cases}$$

Aproximare Continua CMMP

Deci sistemul GRAM va deveni diagonal, iar coeficientii lui g* se vor deduce astfel

$$a_0 = \langle f, u_0\rangle = \frac{1}{\pi\sqrt{2}}\int_0^{2\pi} f(x)\,dx$$

$$b_1 = \frac{1}{\pi}\int_0^{2\pi} f(x)\sin x\,dx \qquad a_1 = \frac{1}{\pi}\int_0^{2\pi} f(x)\cos x\,dx$$

$$b_p = \frac{1}{\pi}\int_0^{2\pi} f(x)\sin px\,dx \qquad a_p = \frac{1}{\pi}\int_0^{2\pi} f(x)\cos px\,dx$$

$$g^* = \frac{a_0}{\sqrt{2}} + \sum_{p=1}^{n}\left(a_p\cos px + b_p\sin px\right)$$

Aproximare Continua CMMP

Aproximare continua Cebasev in sensul celor mai mici patrate se referea la alegerea bazei ca n+1 polinome Cebasev care sunt ortogonale $\frac{1}{\sqrt{2}}, T_1(x), \ldots, T_n(x)$ si a functiei pondere corespunzatoare w(x) = $\frac{1}{\sqrt{1-x^2}}$

Relatiile de ortogonalitate

$$\int_{-1}^{1} \frac{T_q(x)\cdot T_r(x)\,dx}{\sqrt{1-x^2}} = \begin{cases} 0 & q \neq r, \\ \pi & q = r \neq 0, \\ 2\pi & q = r = 0. \end{cases}$$

Aproximare Continua CMMP

Si deci sistemul GRAM devine iara diagonal si deci coeficientii lui g* se deduc din

$$g_n^*(x) = \frac{a_0}{\sqrt{2}} + a_1T_1(x) + \ldots + a_nT_n(x)$$

$$a_0 = \frac{1}{\pi\sqrt{2}}\int_{-1}^{1}\frac{f(x)}{\sqrt{1-x^2}}\,dx$$

$$a_p = \frac{2}{\pi}\int_{-1}^{1}\frac{f(x)\cdot T_p(x)}{\sqrt{1-x^2}}\,dx$$

Aproximare Discreta CMMP

Daca f din F=C([a,b]) este cunoscuta in punctele (x0,f(x0)),...,(xp,f(xp)) se doreste aproximarea optimala in sensul celor mai mici patrate printr-o functie g din G, subspatiu din F, cunoscuta in aceleasi puncte prin (x0,g(x0)),...,(xp,g(xp)).

Consideram ca G este generat de baza u0(x),u1(x),..., un(x) din F, iar produsul scalar si norma sunt alese astfel:

$$\langle f, g\rangle = \sum_{i=0}^{m} w(x_i)\, f(x_i)\, g(x_i)$$

$$\| f \| = \sqrt{\sum_{i=0}^{n} w(x_i)\cdot f^2(x_i)}$$

Aproximare Discreta CMMP

Problema ramane gasirea lui g*(x)=sum ck*uk(x), adica gasirea coeficientilor ck* cu

$$\| f(x) - g^*(x) \| = \min_{g\in G} \| f(x) - g(x) \|$$

Un sistem de functii ortonormate discret satisface conditiile

$$\sum_{i=0}^{n} w(x_i)u_j(x_i)u_k(x_i) = 0, \quad \text{pentru } j \neq k, \quad j, k = 0 : n,$$

$$\sum_{i=0}^{n} w(x_i)u_j^2(x_i) = 1$$

Aproximare Discreta CMMP

Teorema de caracterizare $\sum_{i=0}^{p} w(x_i)\cdot[f(x_i) - g^*(x_i)]\cdot g(x_i) = 0$ conduce la sistemul normal diagonal

$$\sum_{k=0}^{n} c_k^* \sum_{i=0}^{p} w(x_i)\cdot u_k(x_i)\cdot u_j(x_i) = \sum_{i=0}^{p} w(x_i)\cdot f(x_i)\cdot u_j(x_i), \qquad j = 0 : n$$

de unde coeficientii Fourier devin

$$c_k^* = \sum_{i=0}^{p} w(x_i)\, f(x_i)\, u_k(x_i), \qquad k = 0 : n$$

Aproximare Discreta CMMP

Daca se foloseste o baza polinomiala vom avea urmatorul sistem

$$\sum_{k=0}^{n} c_k^* \sum_{i=0}^{n} w(x_i)\, x_i^{k+j} = \sum_{i=0}^{n} w(x_i)\, f(x_i)\, x_i^j, \qquad j = 0 : n$$

Aproximare Discreta CMMP

Aproximare discreta trigonometrica in sensul celor mai mici patrate. Vom alege baza trigonometrica cu 2n+1 elemente si functia pondere w(x)=1

$$\frac{1}{\sqrt{2}}, \sin(x), \cos(x), \cdots \sin nx, \cos nx$$

Suportul de puncte este dat de punctele echidistante din [0,2π], adica

$$x_k = \frac{2\pi\cdot k}{2n+2} = \frac{k\pi}{n+1}, \qquad k = 1 : 2n+2$$

Aproximare Discreta CMMP

Din relatiile de ortogonalitate

$$\sum_{k=1}^{2n+2} u_q(x_k)\cdot u_r(x_k) = \begin{cases} 0, & q \neq r \\ n+1, & q = r' \end{cases} \qquad q, r = 1 : 2n+2$$

Deducem ca sistemul GRAM a devenit diagonal si deci coeficientii lui g* se deduc din

$$a_0 = \frac{1}{n+1}\left\langle f, \frac{1}{\sqrt{2}}\right\rangle = \frac{1}{n+1}\frac{1}{\sqrt{2}}\cdot\sum_{k=1}^{2n+2} f\left(\frac{k\pi}{n+1}\right)$$

$$b_j = \frac{1}{n+1}\langle f, \sin jx\rangle = \frac{1}{n+1}\sum_{k=1}^{2n+2} f\left(\frac{k\pi}{n+1}\right)\sin\frac{k\pi}{n+1}j$$

$$a_j = \frac{1}{n+1}\langle f, \cos jx\rangle = \frac{1}{n+1}\sum_{k=1}^{2n+2} f\left(\frac{k\pi}{n+1}\right)\cos\frac{k\pi}{n+1}j, \qquad j = 1 : n.$$

Aproximare Discreta CMMP

Aproximare discreta Cebasev in sensul celor mai mici patrate. Vom alege baza trigonometrica cu n+1 elemente si functia pondere w(x)=1

$$\frac{1}{\sqrt{2}},\ T_1(x),\ \dots,\ T_n(x)$$

Suportul de puncte este dat de radacinile poliunomului $T_{n+1}(x_k)=0$, adica

$$x_k = \cos\frac{2k+1}{2n+2}\pi, \qquad k = 0 : n$$

Aproximare Discreta CMMP

Facem schimbarea de variabila $x = \cos\theta \Rightarrow \theta_k = \frac{2k+1}{2n+2}\pi$ si avem

$$\sum_{k=0}^{n} T_q(x_k)T_r(x_k) = \sum_{k=0}^{n}\cos q\theta_k \cos r\theta_k = \frac{1}{2}\sum_{k=0}^{n}\left[\cos(q+r)\theta_k + \cos(q-r)\theta_k\right]$$

Iar pentru coeficientii lui g* sistemul GRAM este diagonal si deci

Aproximare Discreta CMMP

$$g^*(x) = \frac{a_0}{\sqrt{2}} + a_1T_1(x) + \cdots + a_nT_n(x)$$

$$\frac{n+1}{2}a_0 = \langle f, u_0\rangle = \frac{1}{\sqrt{2}}\sum_{k=0}^{n} f_k, \qquad a_0 = \frac{\sqrt{2}}{n+1}\sum_{k=0}^{n} f_k$$

$$\frac{n+1}{2}a_j = \langle f, u_j\rangle = \sum_{k=0}^{n} f_kT_j(x_k), \qquad a_j = \frac{2}{n+1}\sum_{k=0}^{n} f_kT_j(x_k)$$

Aproximare cu Functii Rationale

Daca la aproximarea polinomiala apar oscialtii, aici se doreste ca aceste oscialtii sa fie uniforme pe tot intervalul. Se aproximeaza o functie f prin r(x)=p(x)/q(x), unde p este un polinom de grad n si q un polinom de grad m, cu m+n=N.

$$r(x) = \frac{p(x)}{q(x)} = \frac{p_0 + p_1x + \cdots + p_nx^n}{q_0 + q_1x + \cdots + q_mx^m}$$

Ca sa fie definit si in 0, se considera q0≠0 sau mai degraba q0=1, fara a restrange generalitatea(daca nu se imparte si sus si jos cu q0 si obtinem 1).

Aproximare cu Functii Rationale

Aproximare Pade

Si deci avem N+1 parametrii pentru aproximarea cu r(x), adica q1,q2,…,qm si p0,p1,…,pn. Se aleg cei N+1 parametrii impunand conditia ca $f^{(k)}(0) = r^{(k)}(0)$ pentru k=0,1,…,N.

Este defapt o extensie a aproximarii Taylor pentru functii rationale.

Acesta aproximare poarta numele de aproximare Pade(Henri Eugène Padé (1863 -1953), mat. Francez)

Aproximare cu Functii Rationale

Aproximare Pade

Consideram diferenta

$$f(x) - r(x) = f(x) - \frac{p(x)}{q(x)} = \frac{f(x)q(x) - p(x)}{q(x)} = \frac{f(x)\sum_{i=0}^{m} q_ix^i - \sum_{i=0}^{n} p_ix^i}{q(x)}$$

pentru care consideram ca f are o dezvoltare in serie Maclaurin(Colin Maclaurin (1698 -1746), mat. scotian) adica $f(x) = \sum_{i=0}^{\infty} a_ix^i$

Aproximare cu Functii Rationale

Aproximare Pade

Atunci $f(x) - r(x) = \frac{\sum_{i=0}^{\infty} a_i x^i \sum_{i=0}^{m} q_i x^i - \sum_{i=0}^{n} p_i x^i}{q(x)}.$

Pentru a respecta conditia ca $f^{(k)}(0) - r^{(k)}(0) = 0,$ alegem la numarator ceva care sa nu aiba niciun termen de grad mai mic sau egal cu N.

$$(a_0 + a_1 x + \cdots)(1 + q_1 x + \cdots + q_m x^m) - (p_0 + p_1 x + \cdots + p_n x^n),$$

Notam si $p_{n+1} = p_{n+2} = \cdots = p_N = 0$ $q_{m+1} = q_{m+2} = \cdots = q_N = 0.$

Aproximare cu Functii Rationale

Aproximare Pade

Si deci coeficientul lui x^k de la numarator, scris in forma precedenta va fi $\left(\sum_{i=0}^{k} a_i q_{k-i}\right) - p_k.$

Si atunci aproximatia Pade va rezulta din solutionarea sistemului linear de N+1 ecuatii cu N+1 necunoscute.

$$\sum_{i=0}^{k} a_i q_{k-i} = p_k, \quad k = 0, 1, \ldots, N$$

Transformata Fourier Rapida FFT

La aproximare discreta trigonometrica in sensul cmmp coeficientii polinomului g* se obtineau din

$$a_k = \frac{1}{m} \sum_{j=0}^{2m-1} y_j \cos kx_j, \quad k = 0, 1, \ldots, m,$$

$$b_k = \frac{1}{m} \sum_{j=0}^{2m-1} y_j \sin kx_j \quad k = 1, 2, \ldots, m-1.$$

Se doreste obtinea coeficientilor mai eficient(aici avem 4m^2 operatii) prin calculul altor coeficienti.

Transformata Fourier Rapida FFT

In 1965 Cooley(James William Cooley(1926-...), mat. American, IBM) si Tukey(John Wilder Tukey(1915 - 2000) mat. American, Princeton) au publicat un articol in *Mathematics of Computation* unde au aratat ca sunt necesare doar O(m/*log*m) operatii.

An Algorithm for the Machine Calculation of Complex Fourier Series

By James W. Cooley and John W. Tukey

Acest algoritm poarta numele de algoritmul Cooley-Tukey sau FFT.

Gilbert Strang(William Gilbert Strang (1934-...), mat. American, MathWorks prof of MIT) despre FFT: "the most important numerical algorithm of our lifetime"

Exista si alte versiuni de FFT.

Transformata Fourier Rapida FFT

Se doreste calculul coef. ck de forma $c_k = \sum_{j=0}^{2m-1} y_j e^{ik\pi j/m}$ $k = 0, 1, \ldots, 2m-1.$

Daca avem ck calculati, utilizand formula lui Euler $e^{iz} = \cos z + i \sin z.$ se obtin si ak si bk din $a_k + ib_k = \frac{(-1)^k}{m} c_k.$

Consideram m=2^p, atunci pentru orice $k = 0, 1, \ldots, m-1$

$$c_k + c_{m+k} = \sum_{j=0}^{2m-1} y_j e^{ik\pi j/m} + \sum_{j=0}^{2m-1} y_j e^{i(m+k)\pi j/m} = \sum_{j=0}^{2m-1} y_j e^{ik\pi j/m}(1 + e^{\pi ij}).$$

dar $1 + e^{i\pi j} = \begin{cases} 2, & \text{daca j este par} \\ 0, & \text{daca j este impar} \end{cases}$ si deci avem doar m adunari;

Transformata Fourier Rapida FFT

Daca facem j->2j atunci formula se transforma in $c_k + c_{m+k} = 2\sum_{j=0}^{m-1} y_{2j} e^{ik\pi(2j)/m};$ echivalent cu $c_k + c_{m+k} = 2\sum_{j=0}^{m-1} y_{2j} e^{ik\pi j/(m/2)}.$

Si intr-o maniera similara se scrie si diferenta

$$c_k - c_{m+k} = 2e^{ik\pi/m} \sum_{j=0}^{m-1} y_{2j+1} e^{ik\pi j/(m/2)}.$$

Din cele doua se pot deduce ck si cm+k aidca toti 2m coeficienti ck.

Transformata Fourier Rapida FFT

In acest moment de la 4m^2 am ajuns la 2m^2+m operatii.

$$m \cdot m + m(m+1) = 2m^2 + m.$$

Pentru ca m este o putere a lui 2 putem repeta procesul si vom sparge sumele in doua de la j=0 pana la m/2-1.

$$2\left[\frac{m}{2} \cdot \frac{m}{2} + \frac{m}{2} \cdot \left(\frac{m}{2} + 1\right)\right] = m^2 + m.$$

$$(m^2 + m) + m = m^2 + 2m$$





Transformata Fourier Rapida FFT

Daca repetam procesul de r ori atunci numarul de operatii complexe se va reduce la

$$\frac{m^2}{2^{r-2}} + mr.$$

Procesul este complet cand r=p+1, iar numarul de operatii devine

$$\frac{(2^p)^2}{2^{p-1}} + m(p+1) = 2m + pm + m = 3m + m\log_2 m = O(m\log_2 m).$$



Bibliografie Aproximari CMMP

- V.Iorga, B.Jora, Metode Numerice, Ed. Albastră, 2004;
- Richard L. Burden, J. Douglas Faires, Numerical Analysis, 9th edition, Brooks/Cole, Cengage Learning, 2011;





Transformarea Fourier Rapidă (FFT)

Noțiuni teoretice

Transformarea Fourier rapidă

Interpolarea unui set de date alcătuit din $2m$ puncte folosind tehnica de calcul directă necesită efectuarea unui număr mare de operații (aproximativ $(2m)^2$ operații de înmulțire și $(2m)^2$ operații de adunare), motiv pentru care și eroarea obținută în cazul unui număr mare de puncte este foarte mare.

Tehnica directă presupune determinarea polinomului de interpolare trigonometrică pentru cele $2m$ puncte (x_j, y_j), cu

$$x_j = -\pi + (j/m)\pi,$$

pentru oricare $j = 0, 1, ..., 2m-1$ folosind formula următoare:

$$S_m(x) = \frac{a_0 + a_m \cos mx}{2} + \sum_{k=1}^{m-1} (a_k \cos kx + b_k \sin kx)$$

unde:

- $a_k = \frac{1}{m} \sum_{j=0}^{2m-1} y_j \cos kx_j$, pentru oricare $k = 0, 1, ..., m$
- $b_k = \frac{1}{m} \sum_{j=0}^{2m-1} y_j \sin kx_j$, pentru oricare $k = 1, 2, ..., m-1$

Metoda alternativă de a determina forma polinomului de interpolare trigonometrică necesită $O(m \log_2 m)$ operații de înmulțire și $O(m \log_2 m)$ operații de adunare. Astfel, numărul de operații necesare pentru obținerea polinomului de interpolare trigonometrică devine de ordinul miilor, atunci când setul de date este de ordinul miilor, spre deosebire de calculul direct, unde numărul de operații este de ordinul milioanelor.

Metoda poartă numele de transformata Fourier rapidă (Fast Fourier Transform). În locul evaluării directe a constantelor a_k și b_k, transformarea Fourier rapidă modifică valoarea coeficienților complecși c_k în:

$$\frac{1}{m}\sum_{j=0}^{2m-1} c_k e^{ikx}$$

unde $c_k = \sum_{j=0}^{2m-1} y_j e^{ik\pi j/m}$, pentru oricare $k = 0, 1, ..., 2m-1$.

Odată constantele c_k determinate, a_k și b_k pot fi calculate folosind formula lui *Euler*: $e^{iz} = \cos z + i \sin z$.

În plus, pentru orice valoare întreagă a lui n:

$$e^{n\pi i} = \cos n\pi + i \sin n\pi = (-1)^n.$$

astfel încât $a_k + ib_k = \frac{(-1)^k}{m} c_k$, cu precizarea că atât b_0 și b_m sunt egale cu 0 și nu contribuie la rezultatul final.

Presupunând că $m = 2^p$ (m este o putere a lui 2), pentru orice $k = 0, 1, ..., m-1$, vom avea:

$$c_k + c_{k+m} = \sum_{j=0}^{2m-1} y_j e^{ik\pi j/m} + \sum_{j=0}^{2m-1} y_j e^{i(k+m)\pi j/m} = \sum_{j=0}^{2m-1} y_j e^{ik\pi j/m}(1 + e^{\pi ij}).$$

Dar

$$1 + e^{\pi ij} = \begin{cases} 2 & \text{dacă j este par} \\ 0 & \text{dacă j este impar} \end{cases}$$

ceea ce înseamnă că doar m termeni sunt diferiți de 0 și contribuie la rezultatul final.

Dacă înlocuim indicele j cu $2j$ atunci când scriem suma, vom obține:

$$c_k + c_{k+m} = 2\sum_{j=0}^{m-1} y_{2j} e^{ik\pi(2j)/m},$$

echivalent cu relația:

$$c_k + c_{k+m} = 2\sum_{j=0}^{m-1} y_{2j} e^{ik\pi j/(m/2)}.$$

În mod similar,

$$c_k - c_{k+m} = 2e^{ik\pi/m} \sum_{j=0}^{m-1} y_{2j+1} e^{ik\pi j/(m/2)}$$

Atât c_k și c_{k+m} pot fi determinate din cele două relații. Procesul va lua sfârșit după $p+1$ iterații.

Probleme rezolvate

Problema 1

Aplicați transformarea Fourier rapidă asupra unui set de $8 = 2^3$ puncte (x_j, y_j), unde $x_j = -\pi + j\pi/4$, cu $j = 0, 1, ..., 7$. Comparați numărul de operații folosite în cadrul tehnicii de calcul directe a polinomului de interpolare trigonometrică cu numărul de operații necesare în cadrul transformării Fourier rapide.

Soluție:

În acest caz, $2m = 8$, deci $m = 4 = 2^2$ și $p = 2$.

$S_4(x) = \frac{a_0 + a_4 \cos 4x}{2} + \sum_{k=1}^{3}(a_k \cos kx + b_k \sin kx)$, unde

$a_k = \frac{1}{4}\sum_{j=0}^{7} y_j \cos kx_j$ și $b_k = \frac{1}{4}\sum_{j=0}^{7} y_j \sin kx_j$, pentru oricare $k = 0, 1, 2, 3, 4$.

Definim transformarea Fourier ca fiind:

$\frac{1}{4}\sum_{j=0}^{7} c_k e^{ikx}$, unde $c_k = \sum_{j=0}^{7} y_j e^{ik\pi j/4}$, pentru $k = 0, 1, ..., 7$.

Pentru a determina constantele a_k și b_k vom folosi relația $a_k + ib_k = \frac{1}{4}e^{-ik\pi}c_k$.

Folosind calculul direct, obținem constantele c_k:

$c_0 = y_0 + y_1 + y_2 + y_3 + y_4 + y_5 + y_6 + y_7$

$c_1 = y_0 + (\frac{i+1}{\sqrt{2}})y_1 + iy_2 + (\frac{i-1}{\sqrt{2}})y_3 - y_4 - (\frac{i+1}{\sqrt{2}})y_5 - iy_6 - (\frac{i-1}{\sqrt{2}})y_7$

$c_2 = y_0 + iy_1 - y_2 - iy_3 + y_4 + iy_5 - y_6 - iy_7$

$c_3 = y_0 + (\frac{i-1}{\sqrt{2}})y_1 - iy_2 + (\frac{i+1}{\sqrt{2}})y_3 - y_4 - (\frac{i-1}{\sqrt{2}})y_5 + iy_6 - (\frac{i+1}{\sqrt{2}})y_7$

$c_4 = y_0 - y_1 + y_2 - y_3 + y_4 - y_5 + y_6 - y_7$

$c_5 = y_0 - (\frac{i+1}{\sqrt{2}})y_1 + iy_2 - (\frac{i-1}{\sqrt{2}})y_3 - y_4 + (\frac{i+1}{\sqrt{2}})y_5 - iy_6 + (\frac{i-1}{\sqrt{2}})y_7$

$c_6 = y_0 - iy_1 - y_2 + iy_3 + y_4 - iy_5 - y_6 + iy_7$

$c_7 = y_0 - (\frac{i-1}{\sqrt{2}})y_1 - iy_2 - (\frac{i+1}{\sqrt{2}})y_3 - y_4 + (\frac{i-1}{\sqrt{2}})y_5 + iy_6 + (\frac{i+1}{\sqrt{2}})y_7$

Pentru că, în acest caz, setul de date este unul mic, mulți dintre coeficienții termenilor y_i sunt egali cu 1 sau cu -1. Pentru a putea compara număr de operații necesare, vom lua în considerare și înmulțirile cu 1 și cu -1. Astfel, sunt necesare 64 de înmultiri/împărțiri și 56 operații de adunare/scădere pentru obținerea constantelor $c_0, c_1, ..., c_7$ folosind calculul direct.

Pentru a determina primul pas din transformarea Fourier rapidă, vom defini:

$d_0 = \frac{c_0+c_4}{2} = y_0 + y_2 + y_4 + y_6$

$d_1 = \frac{c_0-c_4}{2} = y_1 + y_3 + y_5 + y_7$

$d_2 = \frac{c_1+c_5}{2} = y_0 + iy_2 - y_4 - iy_6$

$d_3 = \frac{c_1-c_5}{2} = (\frac{i+1}{\sqrt{2}})(y_1 + iy_3 - y_5 - iy_7)$

$d_4 = \frac{c_2+c_6}{2} = y_0 - y_2 + y_4 - y_6$

$d_5 = \frac{c_2-c_6}{2} = i(y_1 - y_3 + y_5 - y_7)$

$d_6 = \frac{c_3+c_7}{2} = y_0 - iy_2 - iy_4 + iy_6$

$d_7 = \frac{c_3-c_7}{2} = (\frac{i-1}{\sqrt{2}})(y_1 - iy_3 - y_5 + iy_7)$

Pentru cel de-al doilea pas din transformarea Fourier rapidă, vom defini:

$e_0 = \frac{d_0+d_4}{2} = y_0 + y_4$

$e_1 = \frac{d_0-d_4}{2} = y_2 + y_6$

$e_2 = \frac{id_1+d_5}{2} = i(y_1 + y_5)$

$e_3 = \frac{id_1-d_5}{2} = i(y_3 + y_7)$ $e_4 = \frac{d_2+d_6}{2} = y_0 - y_4$

$e_5 = \frac{d_2-d_6}{2} = i(y_2 - y_6)$

$e_6 = \frac{id_3+d_7}{2} = (\frac{i-1}{\sqrt{2}})(y_1 - y_5)$

$e_7 = \frac{id_3-d_7}{2} = i(\frac{i-1}{\sqrt{2}})(y_3 - y_3)$

Cel de-al treilea pas $(3 = p + 1)$ al transformării presupune definirea:

$f_0 = \frac{e_0+e_4}{2} = y_0$

$f_1 = \frac{e_0-e_4}{2} = y_4$

$f_2 = \frac{ie_1+e_5}{2} = iy_2$

$f_3 = \frac{ie_1-e_5}{2} = iy_6$

$f_4 = \frac{(\frac{i+1}{\sqrt{2}})e_2+e_6}{2} = (\frac{i-1}{\sqrt{2}})y_1$

$f_5 = \frac{(\frac{i+1}{\sqrt{2}})e_2-e_6}{2} = (\frac{i-1}{\sqrt{2}})y_5$

$f_6 = \frac{(\frac{i-1}{\sqrt{2}})e_3+e_7}{2} = (\frac{-i-1}{\sqrt{2}})y_3$

$f_7 = \frac{(\frac{i-1}{\sqrt{2}})e_3-e_7}{2} = (\frac{-i-1}{\sqrt{2}})y_7$

Valorile $c_0, c_1, ..., c_7,\ d_0, d_1, ..., d_7,\ e_0, e_1, ..., e_7,\ f_0, f_1, ..., f_7$ sunt independente de setul de puncte şi depind doar de valoarea lui $m = 4$. Pentru fiecare valoare m, există un set unic de constante c_k, d_k, e_k, f_k, cu $k = 0, 1, ...7$. Calculele anterioare nu sunt neapărat necesare în cadrul unei aplicații.

În cele ce urmează vom prezenta calculele necesare pentru determinarea polinomului de interpolare trigonometrică folosind transformata Fourier rapidă:

Determinarea valorilor f_k:

$f_0 = y_0; f_1 = y_4; f_2 = iy_2; f_3 = iy_6;$

$f_4 = (\frac{i-1}{\sqrt{2}})y_1;\ f_5 = (\frac{i-1}{\sqrt{2}})y_5;\ f_6 = (\frac{-i-1}{\sqrt{2}})y_3;\ f_7 = (\frac{-i-1}{\sqrt{2}})y_7;$

Determinarea valorilor e_k:

$e_0 = f_0 + f_1; e_1 = -i(f_2 + f_3); e_2 = (\frac{i-1}{\sqrt{2}})(f_4 + f_5); e_3 = -(\frac{i+1}{\sqrt{2}})(f_6 + f_7);$

$e_4 = f_0 - f_1;\ e_5 = f_2 - f_3;\ e_6 = f_4 - f_5;\ e_7 = f_6 - f_7;$

Determinarea valorilor d_k:

$d_0 = e_0 + e_1; d_1 = -i(e_2 + e_3); d_2 = e_4 + e_5; d_3 = -i(e_6 + e_7);$

$d_4 = e_0 - e_1;\ d_5 = e_2 - e_3;\ d_6 = e_4 - e_5;\ d_7 = e_6 - e_7;$

Determinarea valorilor c_k:

$c_0 = d_0 + d_1; c_1 = d_2 + d_3; c_2 = d_4 + d_5; c_3 = d_6 + d_7;$

$c_4 = d_0 - d_1;\ c_5 = d_2 - d_3;\ c_6 = d_4 - d_5;\ c_7 = d_6 - d_7;$

Analizând numărul de operații necesare în acest caz, vom obține 24 de înmulțiri / împărțiri și 24 de operații de adunare / scădere.

Probleme propuse

Problema 1

Implementați în OCTAVE algoritmul de determinare al polinomului de interpolare trigonometrică pentru $2m$ puncte (x_j, y_j), cu $x_j = -\pi + (j/m)\pi$, pentru oricare $j = 0, 1, ..., 2m-1$, folosind transformarea Fourier rapidă. Datele de intrare sunt: m putere a lui 2, unde $2m$ este numărul de puncte din setul de date; valorile y_0, y_1, ..., y_{2m-1}. Datele de ieșire sunt: valorile complexe c_0, c_1, ..., c_{2m-1}.

Problema 2

Determinați polinomul de interpolare trigonometrică de grad 5, pe intervalul $[0, 3]$, pentru setul de date $(j/5, f(j/5))$ cu $j = 0, 1, ..., 9$, unde:

$$f(x) = 4x^3 - x^2 + \tan(x - 1).$$

Metode Numerice @ CS-UPB

Echipa MN CS - 2015:

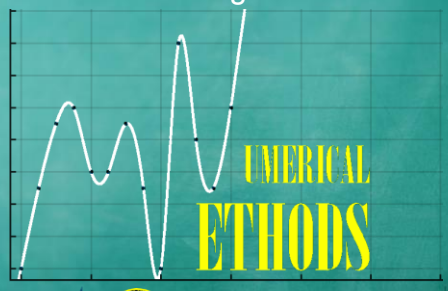
Profesori:

- Pantelimon George Popescu - Seria CA;
- Florin Pop - Seria CB, CC;

Asistenti:

PG Popescu, Clementin Cercel, Sorin Ciolofan, Bogdan Tiganoaia, Alexandru Tifrea, David Iancu, Bogdan Cristian Marchis, Radu Poenaru, Madalina Grosu, Andrei Vlad Postoaca, Radu Constantinescu, Roxana Istrate, Madalina Hristache, Adelina Vidovici;

Derivare si Integrare numerica



Derivare numerica

Derivarea unei functii f intr-un punct x0 se defineste astfel

$$f'(x_0) = \lim_{h \to 0} \frac{f(x_0 + h) - f(x_0)}{h}.$$

Si deci o aproximare a derivatei f' este data de

$$\frac{f(x_0 + h) - f(x_0)}{h}$$

pentru h cat mai mic.

Derivare numerica

Putem afirma precedenta deoarece considerand doua puncte x0 si x1=x0+h si construind polinomul Lagrange pentru punctele x0,x1 P01, putem rescrie f astfel

$$\begin{aligned} f(x) &= P_{0,1}(x) + \frac{(x - x_0)(x - x_1)}{2!} f''(\xi(x)) \\ &= \frac{f(x_0)(x - x_0 - h)}{-h} + \frac{f(x_0 + h)(x - x_0)}{h} + \frac{(x - x_0)(x - x_0 - h)}{2} f''(\xi(x)), \end{aligned}$$

cu epsilon(x) intre x0 si x1.

Derivare numerica

Si prin derivare, parte cu parte vom obtine

$$\begin{aligned} f'(x) &= \frac{f(x_0 + h) - f(x_0)}{h} + D_x \left[\frac{(x - x_0)(x - x_0 - h)}{2} f''(\xi(x)) \right] \\ &= \frac{f(x_0 + h) - f(x_0)}{h} + \frac{2(x - x_0) - h}{2} f''(\xi(x)) \\ &\quad + \frac{(x - x_0)(x - x_0 - h)}{2} D_x(f''(\xi(x))). \end{aligned}$$

Daca renuntam la termenii ce contin epsilon(x) avem

$$f'(x) \approx \frac{f(x_0 + h) - f(x_0)}{h}.$$

Derivare numerica

Mai observam ca daca x este x0, unul dintre termeni se reduce si vom avea

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0)}{h} - \frac{h}{2} f''(\xi).$$

Formula se mai numeste derivare inainte pentru h>0 si derivare inapoi pentru h<0.

Derivare numerica

Pentru a obtine o formula mai generala pentru derivare vom considera n+1 puncte distincte {x0,x1,...,xn} si considerand formula de la interpolare cu polinom Lagrange, pentru f definita pe I, vom avea

$$f(x) = \sum_{k=0}^{n} f(x_k)L_k(x) + \frac{(x-x_0)\cdots(x-x_n)}{(n+1)!} f^{(n+1)}(\xi(x)),$$

cu epsilon(x) in I.

Derivare numerica

Daca derivam aceasta expresie vom obtine

$$f'(x) = \sum_{k=0}^{n} f(x_k)L_k'(x) + D_x\left[\frac{(x-x_0)\cdots(x-x_n)}{(n+1!)}\right] f^{(n+1)}(\xi(x)) + \frac{(x-x_0)\cdots(x-x_n)}{(n+1)!} D_x[f^{(n+1)}(\xi(x))].$$

Si observam ca ultimul termen se reduce pentru x=xj, oricare ar fi j=0,1...,n, adica

$$f'(x_j) = \sum_{\substack{k=0 \\ k\neq j}}^{n} f(x_k)L_k'(x_j) + \frac{f^{(n+1)}(\xi(x_j))}{(n+1)!} \prod_{\substack{k=0 \\ k\neq j}}^{n}(x_j - x_k),$$

formula numita n+1 puncte.

Derivare numerica

Particularizand formula n+1 puncte obtinem formula 3 puncte, unde avem x0, x1=x0+h, x2=x0+2h

$$f'(x_0) = \frac{1}{h}\left[-\frac{3}{2}f(x_0) + 2f(x_0+h) - \frac{1}{2}f(x_0+2h)\right] + \frac{h^2}{3}f^{(3)}(\xi_0),$$

$$f'(x_0+h) = \frac{1}{h}\left[-\frac{1}{2}f(x_0) + \frac{1}{2}f(x_0+2h)\right] - \frac{h^2}{6}f^{(3)}(\xi_1),$$

$$f'(x_0+2h) = \frac{1}{h}\left[\frac{1}{2}f(x_0) - 2f(x_0+h) + \frac{3}{2}f(x_0+2h)\right] + \frac{h^2}{3}f^{(3)}(\xi_2).$$

Derivare numerica

Daca pe prima formula o lasam la fel, in formula din mijloc inlocuim x0+h cu x0 iar in ultima inlocuim pe x0+2h cu x0 obtinem

$$f'(x_0) = \frac{1}{2h}[-3f(x_0) + 4f(x_0+h) - f(x_0+2h)] + \frac{h^2}{3}f^{(3)}(\xi_0),$$

$$f'(x_0) = \frac{1}{2h}[-f(x_0-h) + f(x_0+h)] - \frac{h^2}{6}f^{(3)}(\xi_1),$$

$$f'(x_0) = \frac{1}{2h}[f(x_0-2h) - 4f(x_0-h) + 3f(x_0)] + \frac{h^2}{3}f^{(3)}(\xi_2).$$

Derivare numerica

De unde observand ca daca inlocuim h cu -h defapt avem doar doua formule si adica

Formula 3 puncte punct final:

$$f'(x_0) = \frac{1}{2h}[-3f(x_0) + 4f(x_0+h) - f(x_0+2h)] + \frac{h^2}{3}f^{(3)}(\xi_0),$$

Formula 3 puncte punct de mijloc:

$$f'(x_0) = \frac{1}{2h}[f(x_0+h) - f(x_0-h)] - \frac{h^2}{6}f^{(3)}(\xi_1),$$

Pentru formule de aproximare a derivatelor superioare se procedeaza absolut similar.

Integrare numerica

Metoda de baza pentru evaluarea unei integrale definite se numeste quadratura numerica si consta in aproximarea integralei deinite printr-o suma

$$\int_a^b f(x)\,dx \sim \sum_{i=0}^{n} a_i f(x_i)$$

Tinand seama de interpolarea unei functii prin polinom Lagrange, vom exprima integrala definita, cu tot cu eroarea astfel:

Integrare numerica

$$\int_a^b f(x)\,dx = \int_a^b \sum_{i=0}^{n} f(x_i)L_i(x)\,dx + \int_a^b \prod_{i=0}^{n}(x-x_i)\frac{f^{(n+1)}(\xi(x))}{(n+1)!}\,dx$$
$$= \sum_{i=0}^{n} a_i f(x_i) + \frac{1}{(n+1)!}\int_a^b \prod_{i=0}^{n}(x-x_i) f^{(n+1)}(\xi(x))\,dx,$$

unde epsilon(x) este in [a,b] pentru fiecare x, iar

$$a_i = \int_a^b L_i(x)\,dx,$$ pentru i=0,1,...n;



Integrare numerica

Deci in modul general formula quadraturii este

$$\int_a^b f(x)\,dx \approx \sum_{i=0}^{n} a_i f(x_i),$$

cu eroarea,

$$E(f) = \frac{1}{(n+1)!}\int_a^b \prod_{i=0}^{n}(x-x_i) f^{(n+1)}(\xi(x))\,dx.$$



Integrare numerica

Vom prezenta in continuare formulele particularizate pentru polinomul Lagrange de ordin 1 si 2, adica regula trapezelor si regula Simpson.

Regula trapezelor, comsideram x0=a, x1=b, h=b-a, iar polinomul Lagrange de ordin 1 va fi

$$P_1(x) = \frac{(x-x_1)}{(x_0-x_1)}f(x_0) + \frac{(x-x_0)}{(x_1-x_0)}f(x_1).$$

De unde aproximarea integralei definite va fi



Integrare numerica

$$\int_a^b f(x)\,dx = \int_{x_0}^{x_1}\left[\frac{(x-x_1)}{(x_0-x_1)}f(x_0) + \frac{(x-x_0)}{(x_1-x_0)}f(x_1)\right]dx$$
$$+\frac{1}{2}\int_{x_0}^{x_1} f''(\xi(x))(x-x_0)(x-x_1)\,dx.$$

Dar pentru ca (x-x0)(x-x1) nu isi schimba semnul in [x0,x1], atunci putem aplica Teorema Mediilor de la integrale si vom avea



Integrare numerica

$$\int_{x_0}^{x_1} f''(\xi(x))(x-x_0)(x-x_1)\,dx = f''(\xi)\int_{x_0}^{x_1}(x-x_0)(x-x_1)\,dx$$
$$= f''(\xi)\left[\frac{x^3}{3} - \frac{(x_1+x_0)}{2}x^2 + x_0x_1x\right]_{x_0}^{x_1}$$
$$= -\frac{h^3}{6}f''(\xi).$$

Iar formula se va transforma in

$$\int_a^b f(x)\,dx = \left[\frac{(x-x_1)^2}{2(x_0-x_1)}f(x_0) + \frac{(x-x_0)^2}{2(x_1-x_0)}f(x_1)\right]_{x_0}^{x_1} - \frac{h^3}{12}f''(\xi)$$
$$= \frac{(x_1-x_0)}{2}[f(x_0)+f(x_1)] - \frac{h^3}{12}f''(\xi).$$

Iar pentru ca b-a=x1-x0=h, vom avea



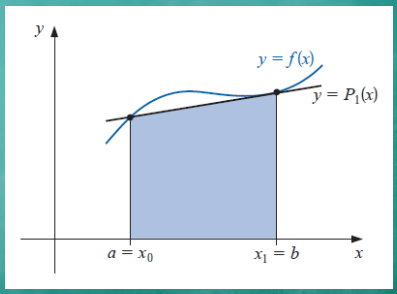
Integrare numerica

$$\int_a^b f(x)\,dx = \frac{h}{2}[f(x_0)+f(x_1)] - \frac{h^3}{12}f''(\xi).$$

Adica pentru o functie cu valori pozitive aproximam integrala definita cu aria trapezului format si de aici si titulatura de regula trapezului.



Integrare numerica



Integrare numerica

Regula lui Simpson. Consideram x0=a,x2=b si x1=x0+h, unde h=(b-a)/2. Expresia integralei definite folosind polinomul Lagrange de ordin 2 va fi

$$\int_a^b f(x)\,dx = \int_{x_0}^{x_2} \left[\frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)} f(x_0) + \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)} f(x_1) + \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)} f(x_2) \right] dx + \int_{x_0}^{x_2} \frac{(x-x_0)(x-x_1)(x-x_2)}{6} f^{(3)}(\xi(x))\,dx.$$

Integrare numerica

Daca consideram dezvoltarea in serie Taylor cu polinom de ordin 3 in jurul lui x1, atunci pentru orice x din [x0,x2], exista epsilon(x) din (x0,x2) astfel incat:

$$f(x) = f(x_1) + f'(x_1)(x-x_1) + \frac{f''(x_1)}{2}(x-x_1)^2 + \frac{f'''(x_1)}{6}(x-x_1)^3 + \frac{f^{(4)}(\xi(x))}{24}(x-x_1)^4$$

Si integrand avem

$$\int_{x_0}^{x_2} f(x)\,dx = \left[f(x_1)(x-x_1) + \frac{f'(x_1)}{2}(x-x_1)^2 + \frac{f''(x_1)}{6}(x-x_1)^3 + \frac{f'''(x_1)}{24}(x-x_1)^4 \right]_{x_0}^{x_2} + \frac{1}{24}\int_{x_0}^{x_2} f^{(4)}(\xi(x))(x-x_1)^4\,dx.$$

Integrare numerica

Si cum (x-x1)^4 este mereu pozitiv putem aplica iarasi teorema mediilor de la integrale si vom obtine

$$\frac{1}{24}\int_{x_0}^{x_2} f^{(4)}(\xi(x))(x-x_1)^4\,dx = \frac{f^{(4)}(\xi_1)}{24}\int_{x_0}^{x_2}(x-x_1)^4\,dx = \frac{f^{(4)}(\xi_1)}{120}(x-x_1)^5\Big]_{x_0}^{x_2},$$

pentru un epsilon1 din (x0,x2);

Dar tinand seama de $h = x_2 - x_1 = x_1 - x_0$ avem

$$\int_{x_0}^{x_2} f(x)\,dx = 2hf(x_1) + \frac{h^3}{3}f''(x_1) + \frac{f^{(4)}(\xi_1)}{60}h^5.$$

Integrare numerica

Si daca il aproximam pe f''(x1) prin (formula de derivare de doua ori in punct de mijloc)

$$\frac{1}{h^2}[f(x_0) - 2f(x_1) + f(x_2)] - \frac{h^2}{12}f^{(4)}(\xi_2)$$

vom avea

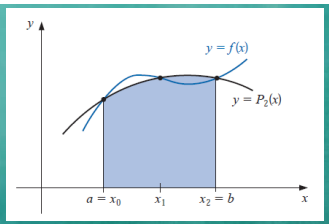
$$\int_{x_0}^{x_2} f(x)\,dx = 2hf(x_1) + \frac{h^3}{3}\left\{\frac{1}{h^2}[f(x_0) - 2f(x_1) + f(x_2)] - \frac{h^2}{12}f^{(4)}(\xi_2)\right\} + \frac{f^{(4)}(\xi_1)}{60}h^5$$

$$= \frac{h}{3}[f(x_0) + 4f(x_1) + f(x_2)] - \frac{h^5}{12}\left[\frac{1}{3}f^{(4)}(\xi_2) - \frac{1}{5}f^{(4)}(\xi_1)\right].$$

Si daca consideram epsilon1=epsilon2=epsilon(exista o teorema care ne permite), vom obtine

Integrare numerica

$$\int_{x_0}^{x_2} f(x)\,dx = \frac{h}{3}[f(x_0) + 4f(x_1) + f(x_2)] - \frac{h^5}{90}f^{(4)}(\xi).$$



Integrare numerica

Formulele trapezelor si Simpson fac parte din clasa mai generala Newton-Cotes

Exista doua tipuri de formule deschise si inchise. Pentru inchise, setul de puncte xi contine si capetele intervalului a si b, iar pentru deschise setul de puncte nu contine capetele.

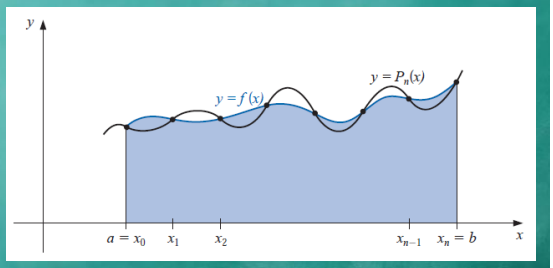
Integrare numerica

Formulele Newton-Cotes inchise, n+1 puncte xi=x0+ih, i=0,1,...,n, x0=a,xn=b, h=(b-a)/n

$$\int_a^b f(x)\,dx \approx \sum_{i=0}^{n} a_i f(x_i),$$

$$a_i = \int_{x_0}^{x_n} L_i(x)\,dx = \int_{x_0}^{x_n} \prod_{\substack{j=0 \\ j\neq i}}^{n} \frac{(x-x_j)}{(x_i-x_j)}\,dx.$$

Integrare numerica



Integrare numerica

Pentru n=1, fomula trapezelor

$$\int_{x_0}^{x_1} f(x)\,dx = \frac{h}{2}[f(x_0)+f(x_1)] - \frac{h^3}{12}f''(\xi), \qquad x_0 < \xi < x_1.$$

Pentru n=2, formula Simpson

$$\int_{x_0}^{x_2} f(x)\,dx = \frac{h}{3}[f(x_0)+4f(x_1)+f(x_2)] - \frac{h^5}{90}f^{(4)}(\xi), \qquad x_0 < \xi < x_2.$$

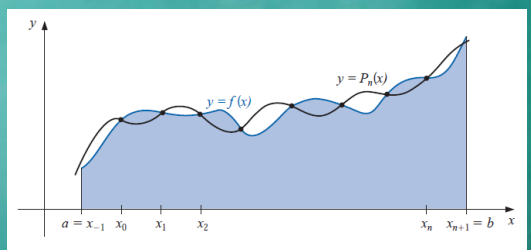
Integrare numerica

Formulele Newton-Cotes deschise, punctele xi=x0+ih, i=0,1,...,n+1, h=(b-a)/n+2, x0=a+h, xn=b-h si notam cu x_-1=a si x_n+1=b, atunci

$$\int_a^b f(x)\,dx = \int_{x_{-1}}^{x_{n+1}} f(x)\,dx \approx \sum_{i=0}^{n} a_i f(x_i),$$

$$a_i = \int_a^b L_i(x)\,dx.$$

Integrare numerica



Integrare numerica

Pentru n=0, avem punctul de mijloc

$$\int_{x_{-1}}^{x_1} f(x)\,dx = 2hf(x_0) + \frac{h^3}{3}f''(\xi), \qquad x_{-1} < \xi < x_1.$$

Pentru n=1, avem

$$\int_{x_{-1}}^{x_2} f(x)\,dx = \frac{3h}{2}[f(x_0) + f(x_1)] + \frac{3h^3}{4}f''(\xi), \qquad x_{-1} < \xi < x_2.$$

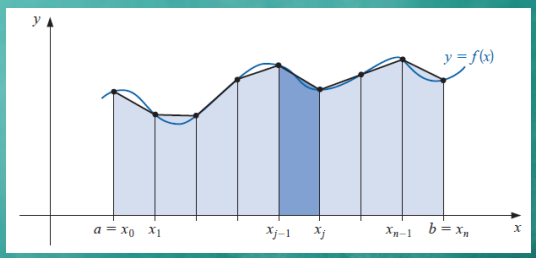
Pentru n=2, avem $\int_{x_{-1}}^{x_3} f(x)\,dx = \frac{4h}{3}[2f(x_0) - f(x_1) + 2f(x_2)] + \frac{14h^5}{45}f^{(4)}(\xi), \quad x_{-1} < \xi < x_3.$

Integrare numerica

Formula compusa de integrare numerica, impartim intervalul in n subintervale si aplicam formula trapezelor.

$$\int_a^b f(x)\,dx = \frac{h}{2}\left[f(a) + 2\sum_{j=1}^{n-1} f(x_j) + f(b)\right] - \frac{b-a}{12}h^2 f''(\mu).$$

Integrare numerica

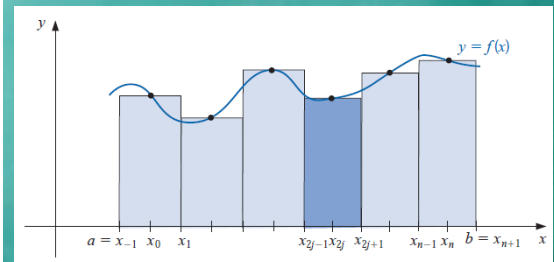


Integrare numerica

Formula compusa de integrare numerica, impartim intervalul in n subintervale si aplicam formula trapezelor cu punct de mijloc.

$$\int_a^b f(x)\,dx = 2h\sum_{j=0}^{n/2} f(x_{2j}) + \frac{b-a}{6}h^2 f''(\mu).$$

Integrare numerica



Integrare numerica

Formula compusa de integrare numerica, alegem numarul n ca fiind par, adica 2n si apoi impartim intervalul in n subintervale si aplicam formula Simpson pe o pereche de subintervale consecutive.

$$\int_a^b f(x)\,dx = \sum_{j=1}^{n/2}\int_{x_{2j-2}}^{x_{2j}} f(x)\,dx$$

$$= \sum_{j=1}^{n/2}\left\{\frac{h}{3}[f(x_{2j-2}) + 4f(x_{2j-1}) + f(x_{2j})] - \frac{h^5}{90}f^{(4)}(\xi_j)\right\}$$

Pentru $x_{2j-2} < \xi_j < x_{2j}$

Integrare numerica

Tinand seama de faptul ca in suma sunt termeni de genul f(x2j), care apar in termenul corespunzator intervalului $[x_{2j-2}, x_{2j}]$ dar si in termenul corespunzator intervalului $[x_{2j}, x_{2j+2}]$ putem reduce suma astfel

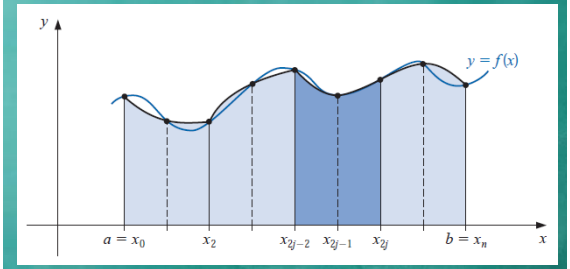
$$\int_a^b f(x)\,dx = \frac{h}{3}\left[f(x_0) + 2\sum_{j=1}^{(n/2)-1} f(x_{2j}) + 4\sum_{j=1}^{n/2} f(x_{2j-1}) + f(x_n)\right] - \frac{h^5}{90}\sum_{j=1}^{n/2} f^{(4)}(\xi_j).$$

Integrare numerica

Exista un rezultat care margineste eroarea si prezinta forma aproximativa a integrarii cu formula compusa Simpson, astfel

$$\int_a^b f(x)\,dx = \frac{h}{3}\left[f(a) + 2\sum_{j=1}^{(n/2)-1} f(x_{2j}) + 4\sum_{j=1}^{n/2} f(x_{2j-1}) + f(b)\right] - \frac{b-a}{180}h^4 f^{(4)}(\mu).$$

Integrare numerica



Bibliografie Derivare si Integrare

- V.Iorga, B.Jora, Metode Numerice, Ed. Albastră, 2004;
- Richard L. Burden, J. Douglas Faires, Numerical Analysis, 9th edition, Brooks/Cole, Cengage Learning, 2011;

Derivare numerică. Metode de integrare Newton-Cotes

Noţiuni teoretice

Ne propunem să calculăm, în mod aproximativ, valorile $I[f] = \int_a^b f(x)dx$ şi $D[f] = f^{(p)}(x_0)$, în condiţiile în care:

- funcţia f este continuă pe intervalul [a, b];
- primitiva F nu este cunoscută;
- funcţia f este cunoscută numai prin valorile $f(x_i)$, într-un număr restrâns de puncte x_i, $i = 0 : N$.

Derivare numerică

Întrucât derivata funcţiei f în punctul x_0 este definită ca

$$f'(x_0) = \lim_{h \to 0} \frac{f(x_0 + h) - f(x_0)}{h},$$

un mod evident de a aproxima $f'(x_0)$ este calculul

$$\frac{f(x_0 + h) - f(x_0)}{h},$$

pentru valori mici ale lui h.

Vom analiza eroarea obţinută prin această metodă, cu ajutorul polinomului de interpolare Lagrange de grad 1, cunoscând valorile funcţiei în punctele x_0 şi $x_1 = x_0 + h$. Considerăm $f \in C^2[a, b]$ şi $x_0, x_1 \in [a, b]$.

$$f(x) = P_1(x) + \frac{(x-x_0)(x-x_1)}{2!} f''(\xi(x))$$

$$f(x) = \frac{f(x_0)(x-x_0-h)}{-h} + \frac{f(x_0+h)(x-x_0)}{h} + \frac{(x-x_0)(x-x_1)}{2} f''(\xi(x)),$$

$$\xi(x) \in [a,b]$$

Derivând, se ajunge la

$$f'(x) = \frac{f(x_0+h)-f(x_0)}{h} + D_x \left[\frac{(x-x_0)(x-x_0-h)}{2} f''(\xi(x)) \right]$$

$$= \frac{f(x_0+h)-f(x_0)}{h} + \frac{2(x-x_0)-h}{2} f''(\xi(x)) + \frac{(x-x_0)(x-x_0-h)}{2} D_x(f''(\xi(x))).$$

Astfel,

$$f'(x) \approx \frac{f(x_0+h)-f(x_0)}{h},$$

cu eroarea

$$\frac{2(x-x_0)-h}{2} f''(\xi(x)) + \frac{(x-x_0)(x-x_0-h)}{2} D_x(f''(\xi(x))).$$

Pentru $x = x_0$, formula se simplifică astfel (two-point formula):

$$f'(x_0) = \frac{f(x_0+h)-f(x_0)}{h} - \frac{h}{2} f''(\xi), \xi \in [x_0, x_0+h].$$

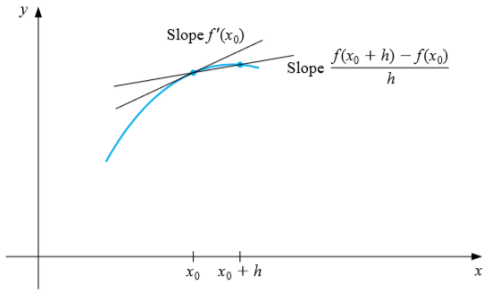


Figure 1: Interpretare geometrică pentru formula de derivare two-point.

Aplicând o tehnică similară, dar folosind polinomul Lagrange de grad 2, cunoscând valorile funcţiei în punctele x_0, x_0+h, x_0+2h şi considerând că există f''' pe un interval ce conţine aceste abscise, obţinem (three-point endpoint formula):

$$f'(x_0) = \frac{1}{2h}[-3f(x_0) + 4f(x_0+h) - f(x_0+2h)] + \frac{h^2}{3}f'''(\xi), \xi \in [x_0, x_0+2h].$$

Această formulă este utilă pentru aproximarea derivatei la capătul unui interval (ex: x_0), situaţie ce apare, spre exemplu, la interpolările cu spline-uri cubice tensionate.

Pentru aproximarea derivatei unei funcţii într-un punct interior unui interval, este recomandată folosirea următoarei formule (obţinută cu ajutorul polinomului de interpolare Lagrange de grad 2, în punctele $x_0 - h$, x_0, $x_0 + h$ (three-point midpoint formula):

$$f'(x_0) = \frac{1}{2h}[f(x_0+h) - f(x_0-h)] - \frac{h^2}{6}f'''(\xi), \xi \in [x_0-h, x_0+h].$$

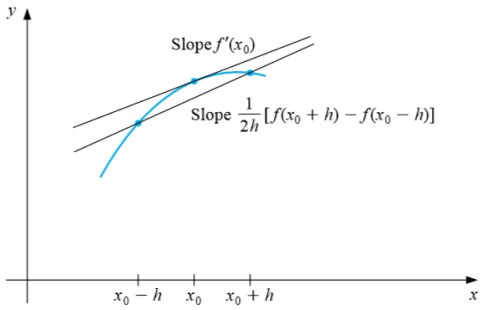


Figure 2: Interpretare geometrică pentru formula de derivare three-point midpoint.

Observaţie

În tehnicile de derivare numerică, reducerea pasului h duce la reducerea erorii teoretice, însă cu costul creşterii erorilor de rotunjire. Pentru a ilustra acest comportament, vom examina formula:

$$f'(x_0) = \frac{1}{2h}[f(x_0+h) - f(x_0-h)] - \frac{h^2}{6}f'''(\xi), \xi \in [x_0-h, x_0+h].$$

Dacă în evaluările $f(x_0+h)$ şi $f(x_0-h)$ apar erorile ϵ_+ şi ϵ_-, notând cu $\tilde{f}(x_0+h)$ şi $\tilde{f}(x_0-h)$ valorile calculate efectiv, avem relaţiile:

$$f(x_0+h) = \tilde{f}(x_0+h) + \epsilon_+$$
$$f(x_0-h) = \tilde{f}(x_0-h) + \epsilon_-$$

Astfel, presupunând că erorile ϵ_+ şi ϵ_- sunt mărginite de $\epsilon > 0$ şi că f''' este marginită de $M > 0$, eroarea totală a aproximării devine:

$$\left| f'(x_0) - \frac{\tilde{f}(x_0+h) - \tilde{f}(x_0-h)}{2h} \right| \leq \frac{\epsilon}{h} + \frac{h^2}{6}M.$$

Pentru a reduce termenul $\frac{h^2}{6}M$ este necesară reducerea lui h, care duce însă la creşterea termenului $\frac{\epsilon}{h}$ (specific erorii de rotunjire), acest termen ajungând să domine calculele.

Integrare numerică

Definim o metodă aproximativă de integrare astfel:

$$I_N[f] = \sum_{i=0}^{N} A_i f(x_i).$$

O astfel de metodă este convergentă dacă:

$$\lim_{N\to\infty} |I[f] - I_N[f]| = 0.$$

Metode Newton-Cotes

Pentru o formulă de integrare aproximativă putem scrie

$$\int_a^b f(x)w(x)dx = \sum_{i=0}^{N} A_i f(x_i) + R_N.$$

În metodele de tip Newton-Cotes, abscisele x_i se aleg echidistante în intervalul $[a, b]$

$$x_i = a + i\frac{(b-a)}{N}, i = 0 : N.$$

Coeficienţii A_i se determină impunând ca formula aproximativă să fie exactă ($R_N = 0$) dacă f aparţine unei anumite clase de funcţii (de exemplu, polinoame de grad $\leq N$). Astfel, vom aproxima funcţia prin polinomul ei de interpolare Lagrange

$$P_N(x) = \sum_{i=0}^{N} f(x_i) l_i(x), \quad l_i(x) = \prod_{j=0, j\neq i}^{N} \frac{(x - x_j)}{(x_i - x_j)}.$$

În acest fel, obţinem $A_i = \int_a^b l_i(x) w(x) dx$.

Deoarece eroarea polinomului de interpolare respectă

$$|f(x) - P_N(x)| \leq \frac{\left|f^{(N+1)}(\xi)\right|}{(N+1)!} \left|(x - x_0)...(x - x_N)\right|,$$

cu $\xi \in [a, b]$, prin integrare obţinem expresia erorii în metodele Newton-Cotes:

$$R_N \leq \frac{\left|f^{(N+1)}(\xi)\right|}{(N+1)!} \int_a^b \left|(x - x_0)...(x - x_N)\right| w(x) dx.$$

Datorită instabilităţii interpolării polinomiale se folosesc polinoame de interpolare de grad mic.

Astfel, pentru $N = 1$ se obţine *formula trapezelor*:

$$\int_a^b f(x) dx = \frac{h}{2} \left[f(a) + f(b)\right] - \frac{h^3 f''(\xi)}{12}, \quad h = b - a.$$

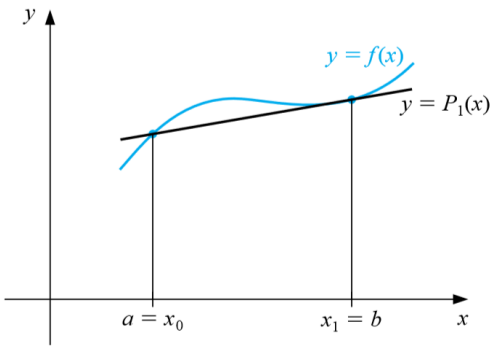


Figure 3: Interpretare geometrică pentru formula trapezelor.

Pentru $N = 2$, se obţine *formula Simpson* (Figure 2):

$$\int_a^b f(x)dx = \frac{h}{3}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right] - \frac{h^5 f^{(4)}(\xi)}{90}, \quad h = \frac{b-a}{2}$$

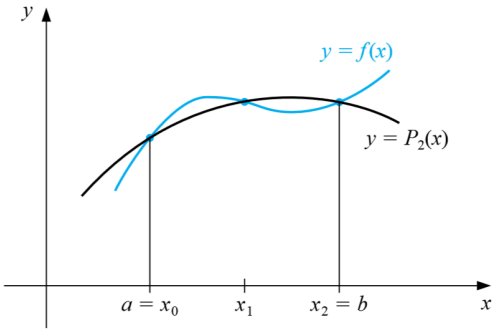


Figure 4: Interpretare geometrică pentru formula Simpson.

Aceste formule folosesc puţine puncte, ceea ce ne determină să aproximăm inte-

grala ca o sumă de integrale calculate pe intervale mai mici

$$\int_{x_0}^{x_N} f(x)dx = \int_{x_0}^{x_1} f(x)dx + ... + \int_{x_{N-1}}^{x_N} f(x)dx.$$

Astfel, obţinem:

- *formula compusă a trapezelor*

$$\int_a^b f(x)dx \approx \frac{h}{2}\left[f(a) + f(b) + 2\sum_{i=1}^{N-1} f(a+ih)\right], \quad h = \frac{(b-a)}{N}$$

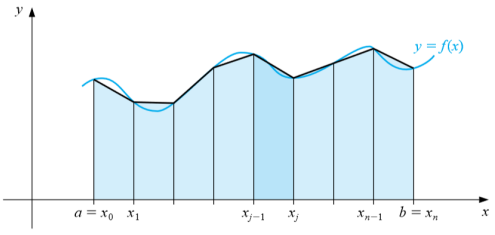


Figure 5: Interpretare geometrică pentru formula compusă a trapezelor.

- *formula compusă Simpson*

$$\int_a^b f(x)dx \approx \frac{h}{3}\left[f(a) + f(b) + 4\sum_{i=1}^{N/2} f(x_{2i-1}) + 2\sum_{i=1}^{N/2-1} f(x_{2i})\right],$$

$$h = \frac{(b-a)}{N}, x_i = a + ih$$

Probleme propuse

Problema 1

Folosind formula de derivare two-point, aproximaţi derivata funcţiei $f(x) = sin(x)$ în punctele $x_0 = 0.2$, $x_1 = 0.5$, $x_2 = 0.9$.

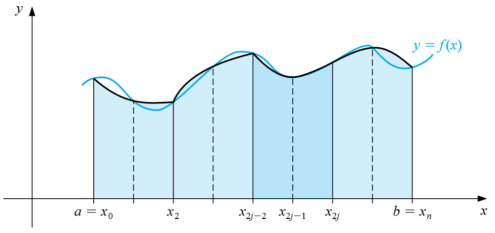


Figure 6: Interpretare geometrică pentru formula compusă Simpson.

Problema 2

a) Scrieţi două funcţii OCTAVE care calculează derivata unei funcţii într-un punct x_0 dat, folosind formulele three-point midpoint, respectiv three-point endpoint. fx este un vector ce conţine valorile funcţiei în punctele necesare fiecărei metode.

b) Folosiţi aceste funcţii pentru a completa tabelul ($f(x) = e^{3x}$):

x	f(x)	f'(x)
2.3	992.27	
2.5	1808.04	
2.7	3294.47	

Problema 3

a) Folosind formula compusă Simpson pentru N = 4, aproximaţi integrala

$$\int_3^5 xlog(x)dx.$$

b) Folosind formula compusă a trapezelor pentru N = 4, aproximaţi integrala

$$\int_1^3 e^{3x}dx.$$

Problema 4

a)Aproximaţi integrala $I = \int_0^{\frac{\pi}{2}} \frac{sin(x)}{x} dx$ prin metoda Simpson folosind 3 puncte.

b) Se dă $f : [a, b] \times [c, d] \to \mathbb{R}$. Calculaţi $I = \int_a^b \int_c^d f(x, y) dx dy$.

Problema 5

Calculaţi aproximativ integrala

$$\int_0^1 ln(1 + x^2) dx,$$

folosind formula compusă a trapezelor pentru $N = 4$.

Problema 6

Pentru formula de integrare de tip Newton-Cotes, $\int_a^b f(x)w(x)dx \approx \sum_{i=0}^{N} A_i f(x_i)$, să se arate că $a_i = \int_a^b w(x)l_i(x)dx$, în care l_i reprezintă multiplicatorii din formula de interpolare Lagrange.

Metode Numerice @ CS-UPB

Echipa MN CS - 2015:

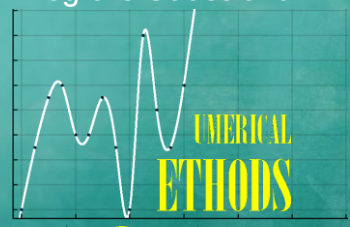
Profesori:

- Pantelimon George Popescu - Seria CA;
- Florin Pop - Seria CB, CC;

Asistenti:

PG Popescu, Clementin Cercel, Sorin Ciolofan, Bogdan Tiganoaia, Alexandru Tifrea, David Iancu, Bogdan Cristian Marchis, Radu Poenaru, Madalina Grosu, Andrei Vlad Postoaca, Radu Constantinescu, Roxana Istrate, Madalina Hristache, Adelina Vidovici;

Extrapolare Richardson
Integrare Romberg
Integrare Gaussiana



Extrapolare Richardson

In 1927 L. F. Richardson(Lewis Fry Richardson (1881 -1953), mat., fizician si meteorolog englez) si J. A. Gaunt au scris un paper despre aceasta extrapolare.

Se doreste aproximarea unei valori M cu o formula N1(h) ce depinde de un parametru, de obicei pasul considerat. Eroarea se exprima astfel

$$M - N_1(h) = K_1 h + K_2 h^2 + K_3 h^3 + \cdots,$$

pentru o colectie de constante k1,k2,...

Extrapolare Richardson

Eroarea fiind O(h), si daca consideram ca nu sunt variatii de ordin de marime intre constantele ki, atunci

$$M - N_1(0.1) \approx 0.1K_1, \quad M - N_1(0.01) \approx 0.01K_1, M - N_1(h) \approx K_1 h.$$

Extrapolarea se refara la combinarea acestor aproximari, de tipul O(h), pentru a produce formule de aproximare cu eroare de ordin mai mare, O(h^2), etc. Am vrea ceva de genul

$$M - N_2(h) = \hat{K}_2 h^2 + \hat{K}_3 h^3 + \cdots,$$

Extrapolare Richardson

$$M - N_2(0.1) \approx 0.01\hat{K}_2, \quad M - N_2(0.01) \approx 0.0001\hat{K}_2,$$

Daca K1 si K^2 au acelasi ordin atunci evident ca N2(h) aproximeaza mai bine ca N1(h).

Si apoi din formulele pentru N2(h) se doreste o aproximare N3(h) care sa exprime eroarea in O(h^3) sasmd.

Extrapolare Richardson

Obtinerea formulelor se face astfel, pornim de la

$$M = N_1(h) + K_1 h + K_2 h^2 + K_3 h^3 + \cdots.$$

Care este valabila pt orice h pozitiv si deci si pentru h/2, adica

$$M = N_1\left(\frac{h}{2}\right) + K_1\frac{h}{2} + K_2\frac{h^2}{4} + K_3\frac{h^3}{8} + \cdots.$$

Si scazand prima din dublul celei de-a doua avem

$$M = N_1\left(\frac{h}{2}\right) + \left[N_1\left(\frac{h}{2}\right) - N_1(h)\right] + K_2\left(\frac{h^2}{2} - h^2\right) + K_3\left(\frac{h^3}{4} - h^3\right) + \cdots.$$

Extrapolare Richardson

Definim atunci N2(h) ca fiind

$$N_2(h) = N_1\left(\frac{h}{2}\right) + \left[N_1\left(\frac{h}{2}\right) - N_1(h)\right].$$

Iar noua formula O(h^2) va fi

$$M = N_2(h) - \frac{K_2}{2}h^2 - \frac{3K_3}{4}h^3 - \cdots.$$

Se repeta procesul si astfel se obtin aproximari mai bune de ordin O(h^2), O(h^4), O(h^6)…

Integrare Romberg

Integrare Romberg(Werner Romberg (1909-2003)) a obtinut rezultatul in 1955.

Daca in metoda compusa a trapezelor eroarea era exprimata astfel (h=(b-a)/2, xi=a+ih)

$$\int_a^b f(x)\,dx = \frac{h}{2}\left[f(a) + 2\sum_{j=1}^{n-1} f(x_j) + f(b)\right] - \frac{(b-a)f''(\mu)}{12}h^2.$$

Integrare Romberg

S-a demonstrat ca eroarea se mai poate exprima si astfel

$$\int_a^b f(x)\,dx = \frac{h}{2}\left[f(a) + 2\sum_{j=1}^{n-1} f(x_j) + f(b)\right] + K_1h^2 + K_2h^4 + K_3h^6 + \cdots$$

unde Ki sunt constante si depind numai de $f^{(2i-1)}(a), f^{(2i-1)}(b)$.

Atunci tinand seama de extrapolarea Richardson putem imbunatati formulele de aproximare prin alegerea lui h=h/2, adica in O(h^4), O(h^6), etc…

Integrare Romberg

Notam aproximarile integralei pe [a,b] cu R11,R21,R31, etc. pentru 1 punct, 2 puncte, 4 puncte, 8 puncte … considerate in suportul interpolarii. Aplicam extrapolare si obtinem O(h^4), adica R22,R32,R42, etc., astfel

$$R_{k,2} = R_{k,1} + \frac{1}{3}(R_{k,1} - R_{k-1,1}), \qquad k = 2,3,\ldots$$

Aplicam iara extrapolare si obtinem O(h^6), adica R33,R43,R53, etc., astfel

$$R_{k,3} = R_{k,2} + \frac{1}{15}(R_{k,2} - R_{k-1,2}), \qquad k = 3,4,\ldots.$$

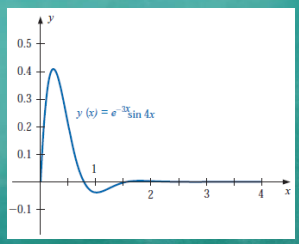
Integrare Romberg

In general, aplicam extrapolare si obtinem O(h^(2j)), astfel

$$R_{k,j} = R_{k,j-1} + \frac{1}{4^{j-1}-1}(R_{k,j-1} - R_{k-1,j-1}), \qquad k = j, j+1,\ldots$$

Quadraturi Adaptive

Sa consideram urmatoarea functie; discutie;

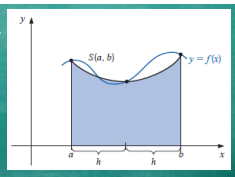


Quadraturi Adaptive

Prezentam o metoda bazata pe Simpson. Sa spunem ca vream sa aproximam integrala de la a la b din f(x)dx cu Simpson, pas h=(b-a)/2 si vom avea

$$\int_a^b f(x)\,dx = S(a,b) - \frac{h^5}{90} f^{(4)}(\xi), \quad S(a,b) = \frac{h}{3}[f(a) + 4f(a+h) + f(b)].$$

Cu epsilon intre a si b.



Quadraturi Adaptive

In continuare vream sa scapam de termenul f^(4) si pentru aceasta vom folosi Simpson cu pasul (b-a)4=h/2, adica

$$\int_a^b f(x)\,dx = \frac{h}{6}\left[f(a) + 4f\left(a+\frac{h}{2}\right) + 2f(a+h) + 4f\left(a+\frac{3h}{2}\right) + f(b)\right] - \left(\frac{h}{2}\right)^4 \frac{(b-a)}{180} f^{(4)}(\xi),$$

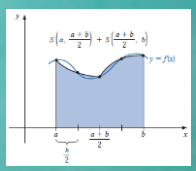
de unde daca notam cu

$$S\left(a, \frac{a+b}{2}\right) = \frac{h}{6}\left[f(a) + 4f\left(a+\frac{h}{2}\right) + f(a+h)\right] \quad S\left(\frac{a+b}{2}, b\right) = \frac{h}{6}\left[f(a+h) + 4f\left(a+\frac{3h}{2}\right) + f(b)\right]$$

avem

$$\int_a^b f(x)\,dx = S\left(a, \frac{a+b}{2}\right) + S\left(\frac{a+b}{2}, b\right) - \frac{1}{16}\left(\frac{h^5}{90}\right) f^{(4)}(\tilde{\xi}).$$

Quadraturi Adaptive



Daca consideram f^(4)(epsilon)~f^(4)(epsilon)

$$S\left(a, \frac{a+b}{2}\right) + S\left(\frac{a+b}{2}, b\right) - \frac{1}{16}\left(\frac{h^5}{90}\right) f^{(4)}(\xi) \approx S(a,b) - \frac{h^5}{90} f^{(4)}(\xi),$$

adica

$$\frac{h^5}{90} f^{(4)}(\xi) \approx \frac{16}{15}\left[S(a,b) - S\left(a, \frac{a+b}{2}\right) - S\left(\frac{a+b}{2}, b\right)\right].$$

Quadraturi Adaptive

De unde rezulta

$$\left|\int_a^b f(x)\,dx - S\left(a, \frac{a+b}{2}\right) - S\left(\frac{a+b}{2}, b\right)\right| \approx \frac{1}{16}\left(\frac{h^5}{90}\right) f^{(4)}(\xi)$$
$$\approx \frac{1}{15}\left|S(a,b) - S\left(a, \frac{a+b}{2}\right) - S\left(\frac{a+b}{2}, b\right)\right|.$$

Ceea ce inseamna ca am aproximat de 15 ori mai bine cu S(a,(a+b)/2)+S((a+b)/2,b) decat cu S(a,b).

Quadraturi Adaptive

Deci daca

$$\left|S(a,b) - S\left(a, \frac{a+b}{2}\right) - S\left(\frac{a+b}{2}, b\right)\right| < 15\varepsilon,$$

ne asteptam ca

$$\left|\int_a^b f(x)\,dx - S\left(a, \frac{a+b}{2}\right) - S\left(\frac{a+b}{2}, b\right)\right| < \varepsilon,$$

Adica faptul ca $S\left(a, \frac{a+b}{2}\right) + S\left(\frac{a+b}{2}, b\right)$ aproximeaza suficient de bine $\int_a^b f(x)\,dx$.

Quadraturi Adaptive

Daca in $\left|S(a,b) - S\left(a, \frac{a+b}{2}\right) - S\left(\frac{a+b}{2}, b\right)\right| < 15\varepsilon,$

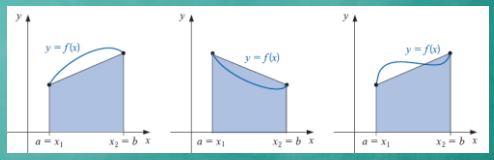
Aproximarire difera cu mai mult de 15e atunci aplicam regula lui Simpson pe subinterv. [a,(a+b)/2] si [(a+b)/2,b]. Apoi folosim tehnica de estimare a erorii ca sa vedem daca eroare pe fiecare subinterfal este mai mica de e/2, si daca da atunci sumam cele doua aproximari si vom obtine o aproximare a integralei in toleranta e. Daca pe un subinterval depasim e/2 atunci impartim subintervalul si reaplicam procedura.

Integrare numerica

Integrare Gaussiana. Daca la Newton-Cotes formulele de integrare erau exacte pentru polinoame de grad cel mult n, deoarece eroarea se exprima ca o derivata de ordin n+1, iar pentru polinoame de grad cel mult n derivata de ordin n+1 = 0 si deci eroare nu exista, adica metodele erau exacte. Aici ne propunem sa extindem gradul de valabilitate al formulelor pentru polinoame de grad mai mare.

Integrare numerica

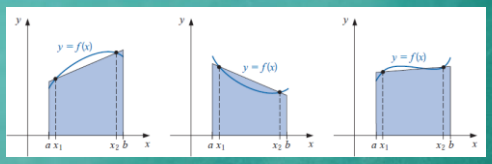
La regula trapezului estimam integralele astfel



Si nu este cea mai buna solutie de obicei.

Integrare numerica

Mai bine ar fi sa estimam integralele astfel



Adica daca la NC, unde punctele erau echidistante, la integrarea Gaussiana punctele xi sunt alese optim.

Integrare numerica

Punctele xi si coeficientii ci sunt alesi astfel incat sa se minimizeze eroarea din formula generala de quadratura

$$\int_a^b f(x)\,dx \approx \sum_{i=1}^{n} c_i f(x_i).$$

Cu alte cuvinte avem de ales 2n necunoscute ci si xi, cu singura restrictie ca xi sa fie din [a,b], intervalul de integrare. Gradul maxim de valabilitate pentru astfel de formule este 2n-1, adica pentru polinoame de grad cel mult 2n-1 formulele de integrare gaussiene sunt exacte.

Integrare numerica

In legatura cu integrarea Gaussiana, prezentam in continuare un rezultat interesant bazat pe polinoamele Legendre monice

$$P_0(x) = 1, \quad P_1(x) = x, \quad P_2(x) = x^2 - \frac{1}{3},$$
$$P_3(x) = x^3 - \frac{3}{5}x, \qquad P_4(x) = x^4 - \frac{6}{7}x^2 + \frac{3}{35}.$$

Daca xi sunt radacinile polinomului monic Legendre de grad n, Pn(x), iar pentru i=1,2,...,n

$$c_i = \int_{-1}^{1} \prod_{\substack{j=1 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}\,dx.$$

Atunci pentru orice polinom P(x) de grad mai mic ca 2n avem

$$\int_{-1}^{1} P(x)\,dx = \sum_{i=1}^{n} c_i P(x_i).$$

Integrare numerica

Daca la integrarea Gaussiana avem un interval [a,b], atunci trecem in [-1,1] prin transformarea lineara

$$t = \frac{2x - a - b}{b - a} \Longleftrightarrow x = \frac{1}{2}[(b - a)t + a + b].$$

si aplicam integrare gaussiana astfel

$$\int_a^b f(x)\,dx = \int_{-1}^{1} f\left(\frac{(b-a)t + (b+a)}{2}\right)\frac{(b-a)}{2}\,dt.$$

Bibliografie Extrapolare Richardson, Integrare Romberg si Gaussiana

- Richard L. Burden, J. Douglas Faires, Numerical Analysis, 9th edition, Brooks/Cole, Cengage Learning, 2011;



Metoda Romberg. Cuadraturi Gaussiene

Noţiuni preliminare

Ne propunem să calculăm în mod aproximativ valorile $I[f] = \int_a^b f(x)\,dx$ și $D[f] = f^{(p)}(x_0)$, în condiţiile:

- funcţia f este continuă pe $[a, b]$;
- primitiva F nu este cunoscută;
- funcţia f este cunoscută numai prin valorile $f(x_i)$ pe care le ia într-un număr restrâns de puncte $x_i,\ i = 0 : N$.

Definim o metodă aproximativă de integrare astfel:

$$I_N[f] = \sum_{i=0}^{N} A_{iN} \cdot f(x_{iN})$$

Metoda aproximativă de aproximare este convergentă dacă

$$\lim_{N\to\infty} |I[f] - I_N[f]| = 0.$$

Cuadraturi Gaussiene

Metodele de tip Newton-Cotes au gradul de valabilitate N (sunt exacte pentru polinoame până la gradul N inclusiv). Dacă în formula aproximativă de integrare:

$$\int_a^b f(x)\,w(x)\,dx \approx \sum_{i=0}^{N} A_{iN} f(x_{iN})$$

se aleg nodurile x_{iN} ca rădăcini ale unui polinom ortogonal, definit în mod unic în raport cu a, b şi funcţia pondere w(x), gradul de valabilitate al formulei devine $2N+1$.

Coeficienţii A_{iN} se vor determina impunând ca formula să aibă grad de valabilitate N (să fie exactă pentru funcţiile $1, x, \cdots, x^N$). Acest fapt conduce la rezolvarea unui sistem de ecuaţii liniare.

Pentru polinoamele ortogonale uzuale, se obţin următoarele formule de integrare:

- Cebâşev ordin 1

$$\int_{-1}^{1} \frac{f(x)}{\sqrt{1-x^2}} dx = \frac{\pi}{N} \sum_{i=0}^{N-1} f\left(cos \frac{(2i+1)\pi}{2N} \right)$$

- Legendre

$$\int_a^b f(x)\, dx = \sum_{i=0}^{N} A_{iN} f(x_{iN}), \; A_{iN} = \frac{2\left(1-x_{iN}^2\right)}{N^2 L_{N-1}^2(x_{iN})}$$

- Laguerre

$$\int_0^{\infty} e^{-x} f(x)\, dx = \sum_{i=0}^{N} A_{iN} f(x_{iN}), \; A_{iN} = \frac{x_{iN}}{(N+1)^2 G_{N+1}^2(x_{iN})}$$

- Hermite

$$\int_0^{\infty} e^{-x^2} f(x)\, dx = \sum_{i=0}^{N} A_{iN} f(x_{iN}), \; A_{iN} = \frac{2^{N+1} N! \sqrt{\pi}}{H_{N+1}^2(x_{iN})}$$

Metoda Romberg

Se formează matricea:

$$I = \begin{bmatrix} I_{11} & & & \\ I_{21} & I_{22} & & \\ I_{31} & I_{32} & I_{33} & \\ \vdots & \vdots & \ddots & \\ I_{N1} & I_{N2} & \cdots & I_{NN} \end{bmatrix}$$

în care prima coloană $I_{11}, I_{21}, \cdots, I_{N1}$ reprezintă estimările integralelor calculate cu formula compusă Simpson, considerând $2^0, 2^1, \cdots, 2^{N-1}$ intervale. I_{N1} poate fi obţinut prin recurenţă din $I_{N-1,1}$ cu formula:

$$I_{N,1} = \frac{1}{2}\left[I_{N-1,1} + \frac{b-a}{2^{N-1}} \sum_{i=1,\Delta i=2}^{2^N-1} f\left(a + \frac{b-a}{2^N} i\right)\right]$$

Elementele din coloana j se calculează cu relaţia de recurenţă:

$$I_{k,j} = \frac{4^{j-1} I_{k,j-1} - I_{k-1,j-1}}{4^{j-1} - 1}, j = 2 : n, k = j : n$$

Fiecare coloană converge către I, cu atât mai rapid cu cât este situată mai la dreapta. Pentru o coloană j, calculul iterativ este oprit în momentul în care $|I_{k,j} - I_{k-1,j}| < \varepsilon \cdot |I_{k,j}|$.

Probleme rezolvate

Problema 1

Calculaţi aproximativ integrala $\int_0^1 \frac{x^4}{\sqrt{x(1-x)}}$.

Soluţie:

Ca prim pas, facem schimbarea de variabilă $t = 2x - 1 \Rightarrow x = \frac{t+1}{2}$.

$x = 0 \Rightarrow t = -1$

$x = 1 \Rightarrow t = 1$

Atunci integrala devine:

$$\int_{-1}^{1} \frac{\left(\frac{t+1}{2}\right)^4}{2\sqrt{\left(\frac{t+1}{2}\right)\left(1 - \frac{t+1}{2}\right)}} dt = \int_{-1}^{1} \frac{\left(\frac{t+1}{2}\right)^4}{\sqrt{(1+t)(1-t)}} dt = \int_{-1}^{1} \frac{\left(\frac{t+1}{2}\right)^4}{\sqrt{1-t^2}} dt.$$

Identificăm

$$f(t) = \left(\frac{t+1}{2}\right)^4, \quad w(t) = \sqrt{1-t^2}, \quad a = -1, \quad b = 1.$$

Din formula de integrare Cebâşev, avem:

$$\int_{-1}^{1} \frac{f(t)}{\sqrt{1-t^2}} dt = \frac{\pi}{N} \sum_{i=0}^{N-1} f\left(cos(x_i)\right), \quad x_i = \frac{(2i+1)\,\pi}{2N}.$$

Deoarece f este un polinom de grad 4, vrem ca formula utilizată pentru aproximare să aibă grad de valabilitate ≥ 4, folosind cât mai puţine puncte.

Alegem $N = 3$, iar gradul de valabilitate va fi 5.

$$x_0 = cos\left(\frac{\pi}{6}\right) = \frac{\sqrt{3}}{2}, \quad x_1 = cos\left(\frac{\pi}{2}\right) = 0, \quad x_2 = cos\left(\frac{5\pi}{6}\right) = -\frac{\sqrt{3}}{2}.$$

În final, formula de integrare devine:

$$\int_0^1 \frac{x^4}{\sqrt{x(1-x)}} dx = \frac{\pi}{3}\left[\left(\frac{\frac{\sqrt{3}}{2}+1}{2}\right)^4 + \left(\frac{1}{2}\right)^4 + \left(\frac{-\frac{\sqrt{3}}{2}+1}{2}\right)^4\right].$$

Problema 2

Se consideră integrala

$$I = \int_{-1}^{1} f(x)\, dx.$$

Determinaţi formula de integrare

$$I \approx a_1 f(x_1) + a_2 f(x_2) + a_3 f(x_3).$$

Precizaţi gradul de valabilitate al formulei.

Soluţie:

În formula de integrare gaussiană:

$$\int_a^b f(x)\, w(x)\, dx \approx \sum_{i=0}^{N} A_i f(x_i),$$

nodurile x_i sunt determinate din condiţia de ortogonalitate a polinomului $\pi(x) = \prod_{i=0}^{N}(x - x_i)$ cu un polinom oarecare de grad mai mic decât cel al lui π, în particular x^k. Avem astfel:

$\int_a^b \pi(x) \cdot w(x) \cdot x^k dx = 0, \quad k = 0 : n-1.$

În cazul nostru,

$\pi(x) = (x - x_1) \cdot (x - x_2) \cdot (x - x_3) = x^3 + ax^2 + bx + c, \quad w(x) = 1$

$\int_{-1}^{1} \left(x^3 + ax^2 + bx + c\right) x^k dx = 0, \quad k = 0 : 2$, care conduce la sistemul de ecuaţii liniare:

$$\begin{cases} \frac{2}{3}a + 2c = 0 \\ \frac{2}{3}b = -\frac{2}{5} \\ \frac{2}{5}a + \frac{2}{3}c = 0 \end{cases}, \text{ cu soluţia } a = c = 0, b = -\frac{3}{5}.$$

$\pi(x) = x^3 - \frac{3}{5}x$, cu rădăcinile $x_1 = -\sqrt{\frac{3}{5}}, \quad x_2 = 0, \quad x_3 = \sqrt{\frac{3}{5}}$.

Pentru determinarea coeficienţilor a_1, a_2, a_3 impunem condiţia ca formula de integrare să fie exactă pentru funcţiile 1, x, x^2. Astfel:

$f(x) = 1 : \qquad \int_{-1}^{1} dx = a_1 + a_2 + a_3 = 2$

$f(x) = x : \qquad \int_{-1}^{1} x dx = -\sqrt{\frac{3}{5}}a_1 + \sqrt{\frac{3}{5}}a_3 = 0$

$f(x) = x^2 : \qquad \int_{-1}^{1} x^2 dx = \frac{3}{5}a_1 + \frac{3}{5}a_3 = \frac{2}{3}$

De aici, găsim că $a_1 = a_3 = \frac{5}{9}, \quad a_2 = \frac{8}{9}$.

În final, formula de integrare devine:

$\int_{-1}^{1} f(x)\, dx = \frac{5}{9} f\left(-\sqrt{\frac{3}{5}}\right) + \frac{8}{9} f(0) + \frac{5}{9} f\left(\sqrt{\frac{3}{5}}\right)$, iar gradul ei de valabilitate este 5.

Probleme propuse

Problema 1

Implementaţi în OCTAVE metoda Romberg şi aproximaţi cu ajutorul ei integrala $\int_3^5 xlog(x)dx$.

Problema 2

Se consideră formula de integrare:

$$\int_{-1}^{1} f(x)dx \approx a_1 f(x_1) + a_2 f(x_2) + a_3 f(x_3) + b_1 f(-1) + b_2 f(1).$$

Determinaţi x_i, a_i, $i = 1:3$ şi b_j, $j = 1:2$ astfel încât formula să aibă grad maxim de valabilitate.

Indicaţie: Nodurile x_1, x_2, x_3 se determină din condiţiile de ortogonalitate

$$\int_{-1}^{1} \pi(x)\rho(x)x^k dx = 0, \qquad k = 0:2,$$

în care $\pi(x) = \prod_{i=1}^{3}(x - x_i)$, $\rho(x) = (x-1)(x-2)$.

Problema 3

Se consideră formula de integrare:

$$\int_0^a f(x)dx \approx a_1 f(x_1) + a_2 f(x_2) + b_1 f(0) + b_2 f(a).$$

Determinaţi x_i, a_i, b_i, $i = 1:2$ astfel încât formula să aibă grad de valabilitate maxim.

Indicaţie: Nodurile x_1 şi x_2 se determină din condiţiile de ortogonalitate

$$\int_{-1}^{1}(x-x_1)(x-x_2)x(x-a)x^k dx = 0, \qquad k = 0:1.$$

Coeficienţii se determină impunând formulei de integrare un grad de valabilitate corespunzător.

Problema 4

Pentru formula de integrare:

$$\int_a^b f(x)w(x)dx \approx \sum_{i=0}^{N} a_i f(x_i),$$

să se arate că dacă integrarea se face prin cuadratură Gaussiană, atunci

$$a_i = \int_a^b w(x)l_i^2(x)dx,$$

în care l_i reprezintă multiplicatorii din formula de interpolare Lagrange.

Problema 5

Formula de integrare Gauss-Radau are forma:

$$\int_{-1}^{1} \frac{f(x)}{\sqrt{1-x^2}} \approx \sum_{i=0}^{N} a_i f(x_i),$$

este exactă pentru polinoame de grad $\leq N$ şi utilizează ca abscise x_i zerourile polinomului $T_{N+1}(x) - T_N(x)$.

a) Dezvoltaţi o formulă de integrare cu $N = 3$.

b) Scrieţi o funcţie OCTAVE care implementează metoda de integrare pentru N oarecare.

Metode Numerice @ CS-UPB

Echipa MN CS - 2015:

Profesori:

- Pantelimon George Popescu - Seria CA;
- Florin Pop - Seria CB, CC;

Asistenti:

PG Popescu, Clementin Cercel, Sorin Ciolofan, Bogdan Tiganoaia, Alexandru Tifrea, David Iancu, Bogdan Cristian Marchis, Radu Poenaru, Madalina Grosu, Andrei Vlad Postoaca, Radu Constantinescu, Roxana Istrate, Madalina Hristache, Adelina Vidovici;

Ecuatii diferentiale cu cond. initiale



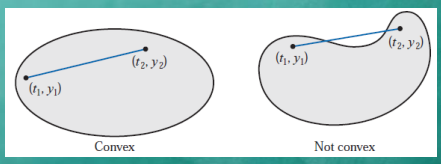
Ecuatii diferentiale cu cond. initiale

O functie f(t,y) satisface o conditie Lipschitz in variabila y pe o multime D din R^2 daca exista L>0 astfel incat $|f(t,y_1) - f(t,y_2,)| \leq L|y_1 - y_2|,$

oricare ar fi (t,y1) si (t,y2) din D. L se numeste constanta Lipschitz(Rudolf Otto Sigismund Lipschitz (1832-1903) mat. german).

Ecuatii diferentiale cu cond. initiale

D din R^2 este convex daca oricare ar fi (t1,y1) si (t2,y2) din D atunci ((1-alpha)t1+alpha t2, (1-alpha)y1+alpha y2), adica orice punct de pe dreapta care le uneste, apartine D oricare ar fi alpha in [0,1].



Ecuatii diferentiale cu cond. initiale

Fie f(t,y) definita pe multimea convexa D din R^2. Daca exista L>0 astfel incat $\left|\frac{\partial f}{\partial y}(t,y)\right| \leq L,$ oricare ar fi (t,y) din D, atunci f satisface conditia Lipschitz pe D in variabila y cu constanta Lipschitz L.

Fie D={(t,y)|a<=t<=b si -inf<y<inf}, iar functia f(t,y) este continua pe D. Daca f satisface o conditie Lipschitz pe D in y, atunci ecuatia diferentiala cu conditie initiala $y'(t) = f(t,y), \quad a \leq t \leq b, \quad y(a) = \alpha,$ are o unica solutie y(t) pentru a<=t<=b.

Ecuatii diferentiale cu cond. initiale

O problema, ODE $\frac{dy}{dt} = f(t,y), \quad a \leq t \leq b, \quad y(a) = \alpha,$ este "bine pusa" daca:

- Exista o solutie unica y(t);
- Exista constantele epsilon0>0 si k>0 astfel incat pentru orice epsilon, cu epsilon0>epsilon>0, oricare ar fi functia continua $|\delta(t)| < \varepsilon$ cu t din [a,b] si cand $|\delta_0| < \varepsilon$, problema $\frac{dz}{dt} = f(t,z) + \delta(t), \quad a \leq t \leq b, \quad z(a) = \alpha + \delta_0,$ are solutie unica z(t) ce satisface $|z(t) - y(t)| < k\varepsilon$ pt. orice t din [a,b].

Ecuatii diferentiale cu cond. initiale

Fie D={(t,y)|a<=t<=b si -inf<y<inf}, iar functia f(t,y) este continua pe D. Daca f satisface o conditie Lipschitz pe D in y, atunci ecuatia diferentiala cu conditie initiala este "bine pusa".

$$\frac{dy}{dt} = f(t,y), \quad a \le t \le b, \quad y(a) = \alpha$$

Ecuatii diferentiale cu cond. initiale

Metoda lui Euler. Isi propune sa afle aproximari ale solutiei ecuatiei diferentiale cu conditii initiale.

$$\frac{dy}{dt} = f(t,y), \quad a \le t \le b, \quad y(a) = \alpha.$$

De obicei se afla solutia unei astfel de ecuatii intr-un set de puncte, numit mesh points din intervalul [a,b] si apoi solutia intr-un punct diferit de mesh se va obtine prin interpolare.

Ecuatii diferentiale cu cond. initiale

Consideram punctele echidistante, ti=a+ih, i=0,1,…N. Folosim Taylor petru exprimarea solutiei y(ti) astfel

$$y(t_{i+1}) = y(t_i) + (t_{i+1} - t_i)y'(t_i) + \frac{(t_{i+1} - t_i)^2}{2}y''(\xi_i),$$

pt epsilon intre ti si ti+1, sau daca luam h=ti+1-ti

$$y(t_{i+1}) = y(t_i) + hy'(t_i) + \frac{h^2}{2}y''(\xi_i),$$

Ecuatii diferentiale cu cond. initiale

Iar pentru ca y'(ti) este solutie a ecuatiei diferentiale cu conditii initiale, avem

$$y(t_{i+1}) = y(t_i) + hf(t_i, y(t_i)) + \frac{h^2}{2}y''(\xi_i).$$

Metoda Euler construieste wi~=y(ti), renuntand la fiecare pas la rest, astfel

$$w_0 = \alpha,$$
$$w_{i+1} = w_i + hf(t_i, w_i),$$

Pentru i=0,1,…N-1

Ecuatii diferentiale cu cond. initiale

Metode Taylor

Daca la Euler am folosit Taylor cu n=1, aici ne dorim sa sporim convergenta prin derivare de ordin superior.

Deci daca presupunem ca solutia y(t) a ecuatiei

$$y' = f(t,y), \quad a \le t \le b, \quad y(a) = \alpha,$$

are (n+1) derivate continue, atunci

$$y(t_{i+1}) = y(t_i) + hy'(t_i) + \frac{h^2}{2}y''(t_i) + \cdots + \frac{h^n}{n!}y^{(n)}(t_i) + \frac{h^{n+1}}{(n+1)!}y^{(n+1)}(\xi_i),$$

pentru epsiloni din (ti,ti+1)

Ecuatii diferentiale cu cond. initiale

Si inlocuind $y^{(k)}(t) = f^{(k-1)}(t, y(t))$ in expresie obtinem

$$y(t_{i+1}) = y(t_i) + hf(t_i, y(t_i)) + \frac{h^2}{2}f'(t_i, y(t_i)) + \cdots + \frac{h^n}{n!}f^{(n-1)}(t_i, y(t_i)) + \frac{h^{n+1}}{(n+1)!}f^{(n)}(\xi_i, y(\xi_i)).$$

Iar ca metoda o vom considera fara restul cu epsilon

$$w_0 = \alpha,$$
$$w_{i+1} = w_i + hT^{(n)}(t_i, w_i), \quad \text{for each } i = 0, 1, \ldots, N-1,$$
$$T^{(n)}(t_i, w_i) = f(t_i, w_i) + \frac{h}{2}f'(t_i, w_i) + \cdots + \frac{h^{n-1}}{n!}f^{(n-1)}(t_i, w_i).$$

Ecuatii diferentiale cu cond. initiale

Metoda Runge-Kutta se bazeaza pe urmatoarea teorema Taylor in doua variabile si sunt la fel de bune ca metodele Taylor dar scapa de evaluarea derivatelor.

Daca f(t,y) si derivatele sale pana la ordinul n+1 inclusiv sunt continue pe $D = \{(t,y) \mid a \le t \le b, c \le y \le d\}$ si fie (t0,y0) din D, atunci pentru orice (t,y) din D exista epsilon intre t si t0 si mu intre y si y0 astfel incat

$$f(t,y) = P_n(t,y) + R_n(t,y),$$



Ecuatii diferentiale cu cond. initiale

unde

$$\begin{aligned} P_n(t,y) = f(t_0,y_0) &+ \left[(t-t_0)\frac{\partial f}{\partial t}(t_0,y_0) + (y-y_0)\frac{\partial f}{\partial y}(t_0,y_0)\right] \\ &+ \left[\frac{(t-t_0)^2}{2}\frac{\partial^2 f}{\partial t^2}(t_0,y_0) + (t-t_0)(y-y_0)\frac{\partial^2 f}{\partial t \partial y}(t_0,y_0) \right. \\ &\left. + \frac{(y-y_0)^2}{2}\frac{\partial^2 f}{\partial y^2}(t_0,y_0)\right] + \cdots \\ &+ \left[\frac{1}{n!}\sum_{j=0}^{n}\binom{n}{j}(t-t_0)^{n-j}(y-y_0)^j \frac{\partial^n f}{\partial t^{n-j}\partial y^j}(t_0,y_0)\right] \end{aligned}$$

si se numeste polinomul Taylor de ordin n in doua variabile, iar Rn este restul asociat polinomului,

$$R_n(t,y) = \frac{1}{(n+1)!}\sum_{j=0}^{n+1}\binom{n+1}{j}(t-t_0)^{n+1-j}(y-y_0)^j \frac{\partial^{n+1} f}{\partial t^{n+1-j}\partial y^j}(\xi,\mu).$$



Ecuatii diferentiale cu cond. initiale

RK de ordin 2. Ne propunem sa gasim coeficientii a1,alpha1 si beta1 astfel incat a1f(t+alpha1,y+beta1) sa aproximeze $T^{(2)}(t,y) = f(t,y) + \frac{h}{2}f'(t,y),$

Si pornim de la urmatoarea

$$f'(t,y) = \frac{df}{dt}(t,y) = \frac{\partial f}{\partial t}(t,y) + \frac{\partial f}{\partial y}(t,y)\cdot y'(t)$$

iar cum y'(t)=f(t,y) vom avea

$$T^{(2)}(t,y) = f(t,y) + \frac{h}{2}\frac{\partial f}{\partial t}(t,y) + \frac{h}{2}\frac{\partial f}{\partial y}(t,y)\cdot f(t,y).$$



Ecuatii diferentiale cu cond. initiale

Folosind polinomul Taylor de grad 1 in (t,y) vom avea

$$\begin{aligned} a_1 f(t+\alpha_1, y+\beta_1) &= a_1 f(t,y) + a_1\alpha_1\frac{\partial f}{\partial t}(t,y) \\ &+ a_1\beta_1\frac{\partial f}{\partial y}(t,y) + a_1 \cdot R_1(t+\alpha_1, y+\beta_1), \end{aligned}$$

cu $R_1(t+\alpha_1, y+\beta_1) = \frac{\alpha_1^2}{2}\frac{\partial^2 f}{\partial t^2}(\xi,\mu) + \alpha_1\beta_1\frac{\partial^2 f}{\partial t \partial y}(\xi,\mu) + \frac{\beta_1^2}{2}\frac{\partial^2 f}{\partial y^2}(\xi,\mu),$

pentru un epsilon intre t si t+alpha1 si un mu intre y si y+beta1.

Si deci coeficientii vor fi $a_1 = 1, \quad \alpha_1 = \frac{h}{2}, \qquad \beta_1 = \frac{h}{2}f(t,y),$



Ecuatii diferentiale cu cond. initiale

Deci $T^{(2)}(t,y) = f\left(t+\frac{h}{2}, y+\frac{h}{2}f(t,y)\right) - R_1\left(t+\frac{h}{2}, y+\frac{h}{2}f(t,y)\right)$

cu

$$\begin{aligned} R_1\left(t+\frac{h}{2}, y+\frac{h}{2}f(t,y)\right) &= \frac{h^2}{8}\frac{\partial^2 f}{\partial t^2}(\xi,\mu) + \frac{h^2}{4}f(t,y)\frac{\partial^2 f}{\partial t \partial y}(\xi,\mu) \\ &+ \frac{h^2}{8}(f(t,y))^2\frac{\partial^2 f}{\partial y^2}(\xi,\mu). \end{aligned}$$

Cu alte cuvinte in loc sa folosim aproximarea T(2), asa cum faceam la metodele Taylor, vom folosi o forma care nu implica derivate, adica metodele RK cu o eroare de O(h^p), unde p este ordinul metodei.



Ecuatii diferentiale cu cond. initiale

Un caz particular este Metoda Punct de mijloc

$$\begin{aligned} w_0 &= \alpha, \\ w_{i+1} &= w_i + hf\left(t_i + \frac{h}{2}, w_i + \frac{h}{2}f(t_i,w_i)\right) \end{aligned}$$

Un caz particular este Metoda Euler Modificata

$$\begin{aligned} w_0 &= \alpha, \\ w_{i+1} &= w_i + \frac{h}{2}[f(t_i,w_i) + f(t_{i+1}, w_i + hf(t_i,w_i))], \end{aligned}$$



Ecuatii diferentiale cu cond. initiale

RK de ordin 3 Adica o forma prin care il aproximam pe T3, eliminand derivatele, adica vrem sa aflam parametrii din urmatoarea forma $f(t+\alpha_1, y+\delta_1 f(t+\alpha_2, y+\delta_2 f(t,y)))$,

si sa ramanem cu o eroare de O(h^3). Parametrii se obtin analog cu cazul precedent. Si formula va fi, pt. i=0,1,...,N-1

$$w_0 = \alpha$$
$$w_{i+1} = w_i + \frac{h}{4}\left(f(t_i,w_i) + 3f\left(t_i+\frac{2h}{3}, w_i+\frac{2h}{3}f\left(t_i+\frac{h}{3}, w_i+\frac{h}{3}f(t_i,w_i)\right)\right)\right),$$

Metoda Heun-> cea mai cunoscuta metoda de ordin 3

$$w_0 = \alpha$$
$$w_{i+1} = w_i + \frac{h}{4}\left(f(t_i,w_i) + 3f\left(t_i+\frac{2h}{3}, w_i+\frac{2h}{3}f\left(t_i+\frac{h}{3}, w_i+\frac{h}{3}f(t_i,w_i)\right)\right)\right)$$



Ecuatii diferentiale cu cond. initiale

RK de ordin 4 Adica o forma prin care il aproximam pe T4, eliminand derivatele si sa avem oeroare de O(h^4).

t. i=0,1,...,N-1

$$w_0 = \alpha,$$
$$k_1 = hf(t_i, w_i),$$
$$k_2 = hf\left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_1\right),$$
$$k_3 = hf\left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_2\right),$$
$$k_4 = hf(t_{i+1}, w_i + k_3),$$
$$w_{i+1} = w_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4),$$



Ecuatii diferentiale cu cond. initiale

Metode multipas O metoda multipas cu m pasi pentru solutionarea unei ODE $y' = f(t,y), \quad a \le t \le b, \quad y(a) = \alpha,$ arata astfel(m>1)

$$w_{i+1} = a_{m-1}w_i + a_{m-2}w_{i-1} + \cdots + a_0 w_{i+1-m} + h[b_m f(t_{i+1}, w_{i+1}) + b_{m-1}f(t_i, w_i) + \cdots + b_0 f(t_{i+1-m}, w_{i+1-m})],$$

pentru i=m-1,m,...,N-1, cu h=(b-a)/N si se dau valorile initiale $w_0 = \alpha, \quad w_1 = \alpha_1, \quad w_2 = \alpha_2, \quad \ldots, \quad w_{m-1} = \alpha_{m-1}$

Daca bm=0 metoda se numeste explicita sau deschisa si invers implicita sau inchisa.



Ecuatii diferentiale cu cond. initiale

Metode multipas Adams-Bashforth explicite

- Doi pasi

$$w_0 = \alpha, \quad w_1 = \alpha_1,$$
$$w_{i+1} = w_i + \frac{h}{2}[3f(t_i,w_i) - f(t_{i-1}, w_{i-1})], \qquad \tau_{i+1}(h) = \frac{5}{12}y'''(\mu_i)h^2$$

- Trei pasi

$$w_0 = \alpha, \quad w_1 = \alpha_1, \quad w_2 = \alpha_2,$$
$$w_{i+1} = w_i + \frac{h}{12}[23f(t_i,w_i) - 16f(t_{i-1}, w_{i-1}) + 5f(t_{i-2}, w_{i-2})], \qquad \tau_{i+1}(h) = \frac{3}{8}y^{(4)}(\mu_i)h^3$$

- Patru pasi

$$w_0 = \alpha, \quad w_1 = \alpha_1, \quad w_2 = \alpha_2, \quad w_3 = \alpha_3,$$
$$w_{i+1} = w_i + \frac{h}{24}[55f(t_i,w_i) - 59f(t_{i-1}, w_{i-1}) + 37f(t_{i-2}, w_{i-2}) - 9f(t_{i-3}, w_{i-3})],$$
$$\tau_{i+1}(h) = \frac{95}{288}y^{(6)}(\mu_i)h^5$$



Ecuatii diferentiale cu cond. initiale

Metode multipas Adams-Moulton implicite

- Doi pasi

$$w_0 = \alpha, \quad w_1 = \alpha_1,$$
$$w_{i+1} = w_i + \frac{h}{12}[5f(t_{i+1}, w_{i+1}) + 8f(t_i,w_i) - f(t_{i-1}, w_{i-1})], \qquad \tau_{i+1}(h) = -\frac{1}{24}y^{(4)}(\mu_i)h^3$$

- Trei pasi

$$w_0 = \alpha, \quad w_1 = \alpha_1, \quad w_2 = \alpha_2, \qquad \tau_{i+1}(h) = -\frac{19}{720}y^{(5)}(\mu_i)h^4$$
$$w_{i+1} = w_i + \frac{h}{24}[9f(t_{i+1}, w_{i+1}) + 19f(t_i,w_i) - 5f(t_{i-1}, w_{i-1}) + f(t_{i-2}, w_{i-2})]$$

- Patru pasi

$$w_0 = \alpha, \quad w_1 = \alpha_1, \quad w_2 = \alpha_2, \quad w_3 = \alpha_3, \qquad \tau_{i+1}(h) = -\frac{3}{160}y^{(6)}(\mu_i)h^5$$
$$w_{i+1} = w_i + \frac{h}{720}[251f(t_{i+1}, w_{i+1}) + 646f(t_i,w_i) - 264f(t_{i-1}, w_{i-1}) + 106f(t_{i-2}, w_{i-2}) - 19f(t_{i-3}, w_{i-3})].$$



Bibliografie ODE

- Richard L. Burden, J. Douglas Faires, Numerical Analysis, 9th edition, Brooks/Cole, Cengage Learning, 2011;




VA MULTUMESC!
Mult succes in continuare!
26.05.2015,
PGPopescu
MN Primavara 2015 CS-UPB
ETHOS

Metode Runge-Kutta pentru rezolvarea ecuaţiilor diferenţiale

Noţiuni teoretice

Fiind date:

- intervalul $I = [x_0, x_0 + a] \subset R$
- funcţia continuă $f : I \times R \to R$ care asociază fiecarui punct (x, y) din domeniul de definiţie un număr real $f(x, y)$
- ecuaţia diferențială $y' = f(x, y)$

problema diferenţială de ordinul 1 constă în determinarea funcţiei $y : I \to R$ astfel încât pentru $\forall x \in I$ avem relaţia:

$$y'(x) = f(x, y(x))$$

Problema diferenţială de ordinul 1 cu condiţii iniţiale (numită şi problema Cauchy) constă în rezolvarea ecuaţiei diferenţiale $y'(x) = f(x, y(x))$ ştiind condiţia iniţială $y(x_0) = y_0$, $y_0 \in R$.

În cele ce urmează, presupunem că funcţia f satisface condiţia Lipschitz, fapt ce asigură existenţa şi unicitatea soluţiei problemei Cauchy:

$$\forall x \in I, \forall u, v \in R^n, \exists L > 0 \text{ astfel încât } |f(x, u) - f(x, v)| < L\,|u - v|$$

Metode de tip Runge-Kutta

O metodă numerică folosită pentru rezolvarea ecuţiilor diferenţiale este metoda Runge-Kutta. Această metodă este o *metodă cu paşi separaţi*, caracterizată prin

faptul că aproximaţia soluţiei la pasul următor $i+1$ ţine cont doar de informaţia de la pasul curent i, astfel:

$$\begin{cases} y_0 = \lambda_h \\ \\ y_{i+1} = y_i + hf_h(x_i, y_i), \quad i = 0, 1, ... \end{cases}$$

şi având condiţiile de consistenţă:

$$\lim_{h\to 0} \lambda_h = \lambda; \quad \lim_{h\to 0} f_h = f.$$

Funcţia $f_h(x, y)$ se determină urmând paşii:

- considerăm punctele distincte

$$x_{ij} = x_{i0} + u_j h$$

  care împart intervalul $I = [x_i, x_{i+1}]$ în q subintervale, unde $u_j \in [0, 1]$, $u_0 = 0$, $u_q = 1$;
- se calculează aproximaţiile soluţiei în punctele introduse x_{ij} folosind relaţiile:

$$\begin{cases} y_{i0} = y_i \\ \\ y_{ij} = y_i + h\sum_{l=0}^{j-1} K_{jl} f(x_{il}, y_{il}), \quad j = 1 : q \end{cases}$$

Pentru a determina punctele introduse x_{ij} şi constantele K_{jl} se impune condiţia ca în dezvoltarea Taylor a lui y_{ij} după puterile lui h, termenii astfel obtinuţi să coincidă cu cât mai mulţi termeni din dezvoltarea Taylor a soluției exacte. O metoda Runge-Kutta este de *ordin* p, dacă în cele două dezvoltări termenii coincid până la h^p inclusiv. Mai mult, numărul subintervalelor q defineşte *rangul* metodei Runge-Kutta.

Metoda Runge-Kutta de ordin 1 şi rang 1 este:

$$\begin{cases} y_{i0} = y_i \\ \\ y_{i1} = y_i + hu_1 f(x_{i0}, y_{i0}) \end{cases}$$

Metoda Runge-Kutta de ordin 2 şi rang 2 este:

$$\begin{cases} y_{i0} = y_i \\ \\ y_{i1} = y_i + hu_1 f(x_{i0}, y_{i0}) \\ \\ y_{i2} = y_i + h(1 - \frac{1}{2u_1}) f(x_{i0}, y_{i0}) + \frac{h}{2u_1} f(x_{i1}, y_{i1}) \end{cases}$$

Particularizând valoarea lui $u_1 \in [0,1]$ obţinem:

- metoda tangentei ameliorate, pentru $u_1 = \frac{1}{2}$:

$$\begin{cases} x_{i1} = x_{i0} + u_1 h = x_i + \frac{h}{2} \\ y_{i1} = y_i + \frac{h}{2} f(x_i, y_i) \\ y_{i+1} = y_i + h f(x_{i1}, y_{i1}) \end{cases}$$

- metoda Heun, pentru $u_1 = \frac{2}{3}$:

$$\begin{cases} x_{i1} = x_i + \frac{2}{3} h \\ y_{i1} = y_i + \frac{2}{3} h f(x_i, y_i) \\ y_{i+1} = y_i + \frac{h}{4} f(x_i, y_i) + \frac{3h}{4} f(x_{i1}, y_{i1}) \end{cases}$$

- metoda Euler-Cauchy, pentru $u_1 = 1$:

$$\begin{cases} x_{i1} = x_i + h \\ y_{i1} = y_i + h f(x_i, y_i) \\ y_{i+1} = y_i + \frac{h}{2}[f(x_i, y_i), f(x_{i1}, y_{i1})] \end{cases}$$

În mod uzual, se utilizează o metodă Runge-Kutta de ordin 4 pentru care avem relaţia:

$$y_{i+1} = y_i + \frac{K_1 + 2K_2 + 2K_3 + K_4}{6}$$

unde:

$K_1 = h f(x_i, y_i)$

$K_2 = h f(x_i + \frac{h}{2}, y_i + \frac{K_1}{2})$

$K_3 = h f(x_i + \frac{h}{2}, y_i + \frac{K_2}{2})$

$K_4 = h f(x_i + h, y_i + K_3)$

Probleme propuse

Problema 1

Să se scrie un program OCTAVE care să rezolve o ecuaţie diferenţială de ordin 1 folosind metoda tangentei ameliorate. Programul primeşte ca date de intrare: a, b - capătul superior, respectiv inferior al intervalului de integrare; n - numărul de puncte; $y0$ - condiţia iniţială; f - funcţia $f(x, y)$. Rezultatul programului va fi y, reprezentând vectorul aproximaţiilor soluţiei ecuaţiei diferenţiale.

Date de intrare:	**Date de ieşire:**
$a = 0, b = 3, n = 10, y0 = 0.5, @functie$	$y = \begin{bmatrix} 0.5 \\ 0.215691 \\ -0.074489 \\ -0.421686 \\ -0.883347 \\ -1.521975 \\ -2.390099 \\ -3.495294 \\ -4.751827 \\ -5.952477 \\ -6.811686 \end{bmatrix}$

unde am considerat funcţia $f(x, y)$ definită astfel:

```
function rez = functie(x, y)
   rez = y*sin(x)-1;
endfunction
```

Listing 1: Exemplu de funcţie de integrat.

Problema 2

Să se scrie program OCTAVE care să rezolve o ecuaţie diferenţială de ordin 1 folosind metoda Runge-Kutta de ordin 4. Programul primeşte ca date de intrare: a, b - capătul superior, respectiv inferior al intervalului de integrare; n - numărul de

puncte; $y0$ - condiţia iniţială; f - funcţia $f(x, y)$. Rezultatul programului va fi y, reprezentând vectorul aproximaţiilor soluţiei ecuaţiei diferenţiale.

Date de intrare:	**Date de ieşire:**
$a = 0, b = 3, n = 10, y0 = 0.5, @functie$	$y = \begin{bmatrix} 0.5 \\ 0.213770 \\ -0.079085 \\ -0.431524 \\ -0.902985 \\ -1.557599 \\ -2.447435 \\ -3.575427 \\ -4.847639 \\ -6.051285 \\ -6.905223 \end{bmatrix}$

Problema 3

Să se scrie un program OCTAVE care să rezolve o ecuaţie diferenţială de ordin 1 folosind metoda Euler-Cauchy. Programul primeşte ca date de intrare: a, b - capătul superior, respectiv inferior al intervalului de integrare; n - numărul de puncte; $y0$ - condiţia iniţială; f - funcţia $f(x, y)$. Rezultatul programului va fi y, reprezentând vectorul aproximaţiilor soluţiei ecuaţiei diferenţiale.

Problema 4

Să se scrie un program OCTAVE care să rezolve o ecuaţie diferenţială de ordin 1 folosind metoda Heun. Programul primeşte ca date de intrare: a, b - capătul superior, respectiv inferior al intervalului de integrare; n - numărul de puncte; $y0$ - condiţia iniţială; f - funcţia $f(x, y)$. Rezultatul programului va fi y, reprezentând vectorul aproximaţiilor soluţiei ecuaţiei diferenţiale.

Problema 5

Să se scrie un program OCTAVE care să rezolve un sistem de 2 ecuaţii diferenţiale de ordin 1 folosind metoda Runge-Kutta de ordin 4. Ambele ecuaţii diferenţiale se

rezolvă pe intervalul delimitat de parametrii a şi b, într-un număr de n puncte; $y10$, $y20$ reprezintă condiţia iniţială a primei ecuaţii diferenţiale, respectiv celei de-a doua; $f1$, $f2$ reprezintă prima funcţie a sistemului, respectiv cea de-a doua funcţie. Programul va avea ca rezultat vectorii $y1$ şi $y2$ (vectorul aproximaţiilor soluţiei asociată primei ecuaţii diferenţiale, respectiv celei de-a doua).

$function$ [$y1$ $y2$] = `Runge_Kutta4_sistem`(a, b, n, $y10$, $y20$, $f1$, $f2$)

Metode predictor-corector pentru integrarea ecuaţiilor diferenţiale

Noţiuni teoretice

Metodele cu paşi separaţi sunt utilizate datorită simplităţii lor şi a faptului că necesită puţine informaţii iniţiale. Ele au dezavantajul lipsei de precizie.

O altă categorie de metode folosite pentru rezolvarea ecuaţiilor diferenţiale şi a sistemelor de ecuaţii diferenţiale este reprezentată de *metodele cu paşi legaţi* (numite şi *metode multipas*). Metodele cu paşi legaţi folosesc mai multe informaţii iniţiale deci sunt mai precise decât metodele cu paşi separaţi. Există două tipuri de metode cu paşi legaţi: *metode explicite* şi *metode implicite*. În cele ce urmează, folosim notaţia: $y_k = y(x_k)$ şi $f_k = f(x_k, y_k)$.

O metodă explicită, cunoscută sub numele de metoda Adams-Bashforth, are forma:

$$y_{k+1} = y_k + h \sum_{j=0}^{r} c_j \nabla^j f_k$$

O metodă implicită, cunoscută sub numele de metoda Adams-Moulton, are forma:

$$y_{k+1} = y_k + h \sum_{j=0}^{r} d_j \nabla^j f_{k+1}$$

Numărul r defineşte ordinul metodei cu paşi legaţi. Dezvoltând diferenţele regresive se obţine o recurenţă liniară, general valabilă pentru metodele cu paşi legaţi, de forma:

$$y_{k+1} = \sum_{j=0}^{r-1} \alpha_j y_{k-j} + h \sum_{j=-1}^{r-1} \beta_j f_{k-j}$$

unde dacă $\beta_{-1} = 0$ se obţine relaţia pentru metoda Adams-Bashforth, altfel se obţine relaţia pentru metoda Adams-Moulton. Pentru a determina coeficienţii α_j, β_j, vom impune condiţia următoare: soluţia exactă $y(x)$ a relaţiei generale pentru metodele cu paşi legaţi să fie adevarată pentru polinoamele $1, x, x^2, ...$ Mai mult, se aleg punctele intermediare $x_{k-r+1}, ..., x_{k+1}$ echidistante cu $h = 1$ şi originea în x_{k-r+1}.

Prin îmbinarea unei metode explicite cu una implicită se obţin *metode predictor-corector*. Metodele predictor-corector presupun folosirea metodei explicite pentru predicţia unei valori y_{k+1} iar aceasta valoare se corecteză, ulterior, prin metoda implicită.

Astfel, putem aplica metoda Adams-Bashforth de ordin 3 pentru a calcula $y_{k+1}^{(p)}$ (y_{k+1} prezis, de unde provine şi superscriptul (p)):

$$y_{k+1}^{(p)} = y_k + \frac{h}{12}(23f_k - 16f_{k-1} + 5f_{k-2})$$

Mai departe, putem corecta valoarea $y_{k+1}^{(p)}$ folosind metoda Adams-Moulton de ordin 2 şi se obţine $y_{k+1}^{(c)}$ (y_{k+1} corectat, de unde provine şi superscriptul (c)):

$$y_{k+1}^{(c)} = y_k + \frac{h}{12}[5f(x_{k+1}, y_{k+1}^{(p)}) + 8f_k - f_{k-1}]$$

Probleme rezolvate

Problema 1

Determinaţi coeficienţii formulelor explicite şi implicite de tip Adams, de ordin 3.

Soluţie:

Metoda Adams-Bashforth de ordin 3 are forma:

$$y_{k+1} = y_k + h(\beta_0 f_k + \beta_1 f_{k-1} + \beta_2 f_{k-2})$$

Pentru început, alegem punctele: $x_{k-2} = 0, x_{k-1} = 1, x_k = 2, x_{k+1} = 3$ cu $h = 1$. Dorim ca formula să fie exactă pentru polinoamele $1, x, x^2, x^3$. Prin urmare, obţinem:

$y = 1 \Rightarrow f = y' = 0 \Rightarrow 1 = 1$

$y = x \Rightarrow f = y' = 1 \Rightarrow 3 = 2 + (\beta_0 + \beta_1 + \beta_2) \Rightarrow \beta_0 + \beta_1 + \beta_2 = 1$

$y = x^2 \Rightarrow f = y^{'} = 2x \Rightarrow 3^2 = 2^2 + (4\beta_0 + 2\beta_1) \Rightarrow 4\beta_0 + 2\beta_1 = 5$

$y = x^3 \Rightarrow f = y^{'} = 3x^2 \Rightarrow 3^3 = 2^3 + (12\beta_0 + 3\beta_1) \Rightarrow 12\beta_0 + 3\beta_1 = 19$

Se formează sistemul:

$$\begin{cases} \beta_0 + \beta_1 + \beta_2 = 1 \\ 4\beta_0 + 2\beta_1 = 5 \\ 12\beta_0 + 3\beta_1 = 19 \end{cases}$$

cu soluţia $\beta_0 = \frac{23}{12}, \beta_1 = \frac{-4}{3}, \beta_2 = \frac{5}{12}$.

Astfel, obţinem forma finală pentru metoda Adams-Bashforth de ordin 3:

$$y_{k+1} = y_k + \frac{h}{12}(23f_k - 16f_{k-1} + 5f_{k-2})$$

Metoda implicită Adams-Moulton de ordin 3 are forma:

$$y_{k+1} = y_k + h(\beta_{-1}f_{k+1} + \beta_0 f_k + \beta_1 f_{k-1} + \beta_2 f_{k-2})$$

Alegem punctele: $x_{k-2} = 0, x_{k-1} = 1, x_k = 2, x_{k+1} = 3$ cu $h = 1$. Dorim ca formula să fie exactă pentru polinoamele $1, x, x^2, x^3, x^4$. Prin urmare, obţinem:

$y = 1 \Rightarrow f = y^{'} = 0 \Rightarrow 1 = 1$

$y = x \Rightarrow f = y^{'} = 1 \Rightarrow 3 = 2 + (\beta_{-1} + \beta_0 + \beta_1 + \beta_2) \Rightarrow \beta_{-1} + \beta_0 + \beta_1 + \beta_2 = 1$

$y = x^2 \Rightarrow f = y^{'} = 2x \Rightarrow 3^2 = 2^2 + (6\beta_{-1} + 4\beta_0 + 2\beta_1) \Rightarrow 6\beta_{-1} + 4\beta_0 + 2\beta_1 = 5$

$y = x^3 \Rightarrow f = y^{'} = 3x^2 \Rightarrow 3^3 = 2^3 + (27\beta_{-1} + 12\beta_0 + 3\beta_1) \Rightarrow 27\beta_{-1} + 12\beta_0 + 3\beta_1 = 19$

$y = x^4 \Rightarrow f = y^{'} = 4x^3 \Rightarrow 3^4 = 2^4 + (108\beta_{-1} + 32\beta_0 + 4\beta_1) \Rightarrow 108\beta_{-1} + 32\beta_0 + 4\beta_1 = 65$

Se formează sistemul:

$$\begin{cases} \beta_{-1} + \beta_0 + \beta_1 + \beta_2 = 1 \\ 6\beta_{-1} + 4\beta_0 + 2\beta_1 = 5 \\ 27\beta_{-1} + 12\beta_0 + 3\beta_1 = 19 \\ 108\beta_{-1} + 32\beta_0 + 4\beta_1 = 65 \end{cases}$$

cu soluţia $\beta_{-1} = \frac{9}{24}, \beta_0 = \frac{19}{24}, \beta_1 = \frac{-5}{24}, \beta_2 = \frac{1}{24}$.

Atunci, forma finală pentru metoda Adams-Moulton de ordin 3 este:

$$y_{k+1} = y_k + \frac{h}{24}(9f_{k+1} + 19f_k - 5f_{k-1} - f_{k-2})$$

Problema 2

Pentru rezolvarea problemei diferenţiale cu condiţii iniţiale: $y' = f(x, y), y(x_0) = y_0$, se foloseşte următoarea formulă explicită, respectiv implicită:

$$y_{k+1} = y_{k-1} + h\sum_{j=0}^{2} \beta_j f_{k-j}$$

$$y_{k+1} = y_{k-1} + h\sum_{j=0}^{2} \beta_j f_{k+1-j}$$

a) Determinaţi coeficienţii β astfel încât formulele să aibă gradul de valabilitate maxim.

b) Definiţi o metodă predictor-corector folosind aceste formule.

Soluţie:

a) Pentru formula:

$$y_{k+1} = y_{k-1} + h(\beta_0 f_k + \beta_1 f_{k-1} + \beta_2 f_{k-2})$$

alegem punctele: $x_{k-2} = 0, x_{k-1} = 1, x_k = 2, x_{k+1} = 3$ cu $h = 1$. Dorim ca formula să fie exactă pentru polinoamele $1, x, x^2, x^3$. Avem:

$y = 1 \Rightarrow f = y' = 0 \Rightarrow 1 = 1$

$y = x \Rightarrow f = y' = 1 \Rightarrow 3 = 1 + (\beta_0 + \beta_1 + \beta_2) \Rightarrow \beta_0 + \beta_1 + \beta_1 = 2$

$y = x^2 \Rightarrow f = y' = 2x \Rightarrow 3^2 = 1^2 + (4\beta_0 + 2\beta_1) \Rightarrow 4\beta_0 + 2\beta_1 = 8$

$y = x^3 \Rightarrow f = y' = 3x^2 \Rightarrow 3^3 = 1^3 + (12\beta_0 + 3\beta_1) \Rightarrow 12\beta_0 + 3\beta_1 = 26$

Sistemul obţinut are soluţia $\beta_0 = \frac{7}{3}, \beta_1 = \frac{-2}{3}, \beta_2 = \frac{1}{3}$. Prin urmare, prima formulă are forma:

$$y_{k+1} = y_{k-1} + \frac{h}{3}(7f_k - 2f_{k-1} + f_{k-2})$$

Pentru formula:

$$y_{k+1} = y_{k-1} + h(\beta_0 f_{k+1} + \beta_1 f_k + \beta_2 f_{k-1})$$

alegem punctele: $x_{k-2} = 0, x_{k-1} = 1, x_k = 2, x_{k+1} = 3$ cu $h = 1$. Dorim ca formula să fie exactă pentru polinoamele $1, x, x^2, x^3$. Avem:

$y = 1 \Rightarrow f = y' = 0 \Rightarrow 1 = 1$

$y = x \Rightarrow f = y' = 1 \Rightarrow 3 = 1 + (\beta_0 + \beta_1 + \beta_2) \Rightarrow \beta_0 + \beta_1 + \beta_1 = 2$

$y = x^2 \Rightarrow f = y' = 2x \Rightarrow 3^2 = 1^2 + (6\beta_0 + 4\beta_1 + 2\beta_2) \Rightarrow 6\beta_0 + 4\beta_1 + 2\beta_2 = 8$

$y = x^3 \Rightarrow f = y' = 3x^2 \Rightarrow 3^3 = 1^3 + (27\beta_0 + 12\beta_1 + 3\beta_2) \Rightarrow 27\beta_0 + 12\beta_1 + 3\beta_2 = 26$

Sistemul obţinut are soluţia $\beta_0 = \frac{1}{3}, \beta_1 = \frac{4}{3}, \beta_2 = \frac{1}{3}$. Prin urmare, a doua formulă are forma:

$$y_{k+1} = y_{k-1} + \frac{h}{3}(f_{k+1} + 4f_k + f_{k-1})$$

b) Formula explicită furnizează predicţia:

$$y_{k+1}^{(p)} = y_{k-1} + \frac{h}{3}(7f_k - 2f_{k-1} + f_{k-2})$$

iar formula implicită corectează această valoare, astfel:

$$y_{k+1}^{(c)} = y_{k-1} + \frac{h}{3}[f(x_{k+1}, y_{k+1}^{(p)}) + 4f_k + f_{k-1}]$$

Probleme propuse

Problema 1

Scrieţi o funcţie OCTAVE care să rezolve o ecuaţie diferenţială de ordin 1 folosind metoda predictor-corector obţinută prin îmbinarea unei metode Adams-Bashforth de ordin 3 cu o metoda Adams-Moulton de ordin 2 (vezi secţiunea *Noţiuni teoretice*). Valorile iniţiale ale soluţiei se vor aproxima folosind metoda Runge-Kutta de ordin 4. Funcţia va avea următorii parametrii de intrare: a, b - capătul superior, respectiv inferior al intervalului de integrare; n - numărul de puncte; $y0$ - condiţia iniţială; f - funcţia de integrat $f(x, y)$. Rezultatul funcţiei va fi y, reprezentând vectorul aproximaţiilor soluţiei ecuaţiei diferenţiale.

Date de intrare:	**Date de ieşire:**
$a = 0, b = 3, n = 10, y0 = 0.5, @functie$	$y = \begin{bmatrix} 0.5 \\ 0.213770 \\ -0.079085 \\ -0.431703 \\ -0.903119 \\ -1.557205 \\ -2.445785 \\ -3.571678 \\ -4.841284 \\ -6.043362 \\ -6.899990 \end{bmatrix}$

unde am considerat funcţia $f(x, y)$ definită astfel:

```
function rez = functie(x, y)
   rez = y*sin(x)-1;
endfunction
```

Listing 1: Exemplu de funcţie de integrat.

Problema 2

Pentru problema diferenţială cu condiţii iniţiale: $y' = f(y, t), y(t_0) = y_0$, se consideră formula aproximativă de integrare:

$$y_{k+1} = y_k + h(\alpha_1 y'_{k+1} + \alpha_0 y'_k) + h^2(\beta_1 y''_{k+1} + \beta_0 y''_k)$$

Determinaţi $\alpha_0, \alpha_1, \beta_0$ şi β_1 astfel încât formula să aibă gradul de valabilitate cât mai mare.

Problema 3

Pentru problema diferenţială cu condiţii iniţiale: $y' = f(y, t)$, $y(t_0) = y_0$:

a) Calculaţi coeficienţii formulelor explicite şi implicite de tip Adams, de ordin 2.

b) Definiţi cu cele două formule o metodă predictor-corector şi scrieţi o funcţie în OCTAVE care implementează această metodă.